

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

Consumption Management System

2025.

Szakma:
SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:
Dobosi Gábor
Mák Luca
Orbán Barnabás

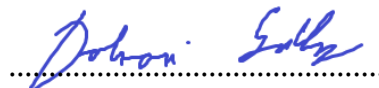
Tartalomjegyzék

1	Bevezetés	5
2	Témaválasztás indoklása	5
2.1	Indoklás és aktualitás	5
2.2	Kutatások, követelmények	6
2.3	Eszközök és eljárások	7
2.3.1	Általános fejlesztéshez használt eszközök	7
2.3.2	Adatbázis fejlesztéshez használt eszközök	8
3	Fejlesztői dokumentáció	8
3.1	A programmal kapcsolatos követelmény	8
3.2	Frontend	9
3.2.1	Specifikáció	9
3.2.2	Megjelenési követelmény	11
3.2.3	Frontendhez használt fejlesztői eszközök	42
3.2.4	Algoritmusok	44
3.2.5	Teljesítményoptimalizálás	48
3.2.6	Tesztelés	49
3.3	Backend	54
3.3.1	Specifikáció	54
3.3.2	Az alkalmazott fejlesztői eszközök	55
3.3.3	Adatmodell leírása	57
3.3.4	Táblák és modelljeik:	58
3.3.5	Dependency Injection (DI)	70
3.3.6	OnModelCreating Metódus és Entitás Kapcsolatok Konfigurálása	71
3.3.7	DTO-k (Data Transfer Object)	71
3.3.8	Controllerek	72

3.3.9	Tesztelés	88
3.4	Program futtatása fejlesztői módban.....	98
3.4.1	Backend (ASP.NET) indítása	98
3.4.2	Frontend (React + Vite) indítása	99
4	Felhasználói dokumentáció	100
4.1	A program általános specifikációja, célja	100
4.2	Rendszerkövetelmények	100
4.2.1	Szoftver követelmények	100
4.2.2	Minimális kliensoldai hardver követelmény	101
4.3	Program telepítése szerverre / kiszolgáló számítógépre	102
4.3.1	Rendszer futtatása (.exe esetén build használata).....	102
4.3.2	Hardverkövetelmény	104
4.4	A program használatának részletes leírása	105
5	Összegzés	120
5.1	Fejleszthetőségi lehetőségek	120
6	Ábrajegyzék.....	122
7	Irodalmi jegyzék, hivatkozásjegyzék	128

Alulírott **Dobosi Gábor** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 22.



Dobosi Gábor

Alulírott **Mák Luca** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 22.



Mák Luca

Alulírott **Orbán Barnabás** büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 22.



Orbán Barnabás

1 Bevezetés

Csapatunk egy valós problémára keresett megoldást, ami egy olyan szoftver kifejlesztéséhez vezetett, amely szállodákban, fürdőkben és egyéb komplex szolgáltatóhelyeken is alkalmazható. A program célja, hogy megkönnyítse az intézmény dolgozóinak munkáját, miközben kényelmesebb élményt biztosít a vendégek számára. A rendszer egy helyen gyűjti össze a felhasználók költségeit, amelyeket az alkalmazottak egy ID segítségével tudnak beazonosítani.

Példaként vegyünk egy fürdőt, amelyre a programunk is épül. Kis változtatásokkal azonban bármely más szolgáltatóhely igényeihez is könnyedén igazítható. A vendégek a belépéskor kapott karszalagjukhoz kapcsolódva vehetik igénybe a különböző szolgáltatásokat anélkül, hogy azonnali fizetésre lenne szükség. Ez úgy valósul meg, hogy a karszalagra terhelődnek az igénybe vett szolgáltatások.

Ez több szempontból is előnyös: először is, nem kell egész nap magukkal hordaniuk a pénztárcájukat vagy bankkártyájukat, ami kényelmi és biztonsági szempontból is praktikus. Emellett nem kell minden szolgáltatásért külön-külön fizetniük, hanem a látogatás végén kényelmesen, egy összegben rendezhetik a számlát. A fejlesztés során fizikai eszközök is rendelkezésünkre álltak, amelyeket beépítettünk a munkába, ezzel is szimulálva az éles környezetet.

2 Témaválasztás indoklása

2.1 Indoklás és aktualitás

A témaválasztás nem bizonyult egyszerű feladatnak. Szerettünk volna egy olyan alkalmazást létrehozni, amely valódi kihívást jelent, és nem csupán a minimum követelményszint teljesítésére irányul. Számos ötlet felmerült, például webshop vagy Rubik-kocka időmérő, végül azonban egy fogyasztáskezelő rendszer mellett döntöttünk.

Célunk az volt, hogy a megoldás minél jobban tükrözze az éles környezet működését, ezért fizikai eszközöket is bevontunk a fejlesztési folyamatba. Ennek részeként került sor karszalagok és RFID olvasók alkalmazására, amelyek lehetővé tették a szolgáltatások azonosítását és rögzítését.

A projekt aktualitása is indokolja a választásunkat, hiszen napjainkban egyre nagyobb igény mutatkozik a gyors, kényelmes és egyszerű szolgáltatások iránt. A programunk pontosan ezt a célt szolgálja.

A fejlesztés során a legkorszerűbb technológiákra alapoztunk: a projekt a *.NET 8* platformra és a *React* keretrendszerre épül. Ennek köszönhetően nemcsak a legújabb technológiai lehetőségeket aknáztuk ki, hanem követtük a modern, *API*-központú webfejlesztési irányelveket is, amelyek különösen fontosak a rendszer jövőbeli bővíthetősége szempontjából.

2.2 Kutatások, követelmények

Első lépésként áttekintettük a vizsgakövetelményeket annak érdekében, hogy azonosítsuk a kötelező elemeket. Az alkalmazásnak vagy weboldalnak tartalmaznia kellett egy frontend részt, amely a megjelenítésért és az adatok betöltéséért felel. Emellett szükség volt egy backend rendszerre is, amely biztosítja az adatok tárolását, a jogosultságkezelést, az adatbázis-táblák és kapcsolatok pontos definiálását, valamint az *API*-végpontok meghatározását, lehetővé téve a frontend és a backend közötti kommunikációt.

A hatékonyság érdekében frontend oldalon a *React* keretrendszert alkalmaztuk. A backend fejlesztéséhez az *ASP.NET Core Web API* technológiát választottuk, amely a *C#* programozási nyelvre épül.

A szoftver adatbázisa végül *SQLite* alapú lett, amelyet az *Entity Framework* segítségével integráltunk és kezeltünk. Ez a megoldás különösen előnyös a fejlesztés korai szakaszaiban és kisebb rendszerek esetében, mivel egyszerű, könnyűsúlyú és nem igényel külön szervertelepítést.

Programunk megfelel az alapvető adatkezelési műveleteknek (*CRUD*), amelyek magukban foglalják az adatok létrehozását (*Create*), lekérdezését (*Read*), módosítását (*Update*) és törlését (*Delete*). Ezek a backend oldalon a megfelelő *HTTP*-metódusokon (*GET*, *POST*, *PUT*, *DELETE*) keresztül valósulnak meg.

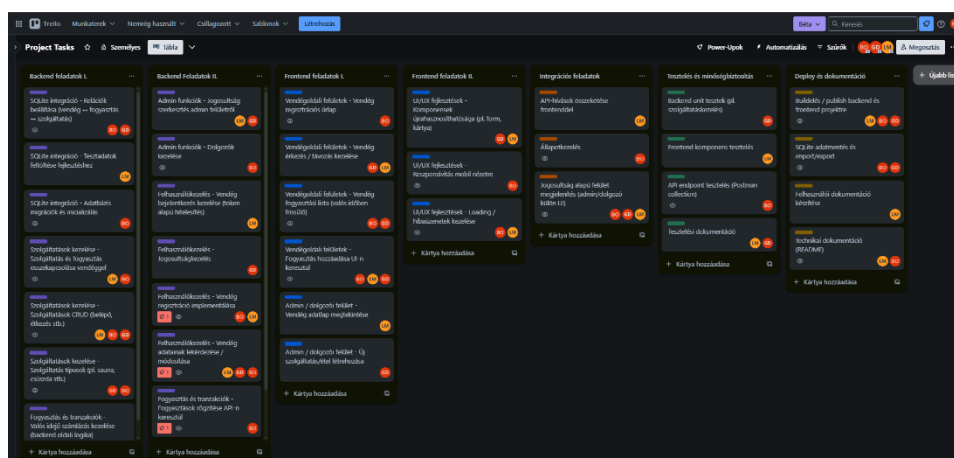
Törekedtünk a tiszta kód (*Clean Code*) elveinek alkalmazására, amelyek elősegítik a fejlesztés hatékonyságát, az átláthatóságot, valamint a későbbi módosítások egyszerűbb végrehajtását. Ennek érdekében például következetesen egyértelmű változóneveket használtunk a *PascalCase* elnevezési konvenció szerint, valamint logikus, jól strukturált mappaszerkezetet alakítottunk ki.

2.3 Eszközök és eljárások

2.3.1 Általános fejlesztéshez használt eszközök

Trello

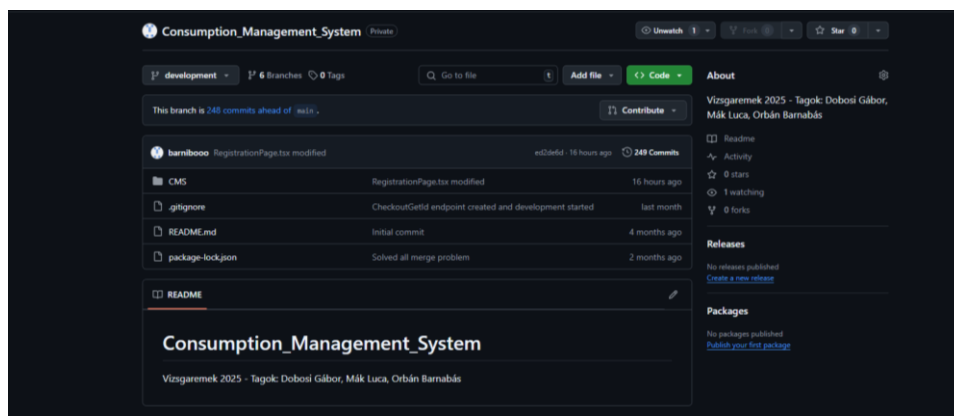
A *Trello* egy vizuális projektmenedzsment eszköz, amely *Kanban*-alapú táblák, listák és kártyák segítségével segíti a feladatok szervezését. Fejlesztőcsapatok számára kiválóan alkalmas sprint tervezésre, backlog kezelésre és feladatkövetésre. Integrálható *GitHub*bal, *Slack*kel és más fejlesztői eszközökkel.



1. ábra - Trello munkafelület

GitHub

A *GitHub* egy népszerű forráskód-kezelő és verziókövető platform, amely a *Git* rendszeren alapul. Lehetővé teszi a kód tárolását, verziókövetését, csapatmunkát. Nyílt forráskódú és privát projektekhez egyaránt használható.



2. ábra - Github repository development branch (2025.04.21. állapot szerint)

2.3.2 Adatbázis fejlesztéshez használt eszközök

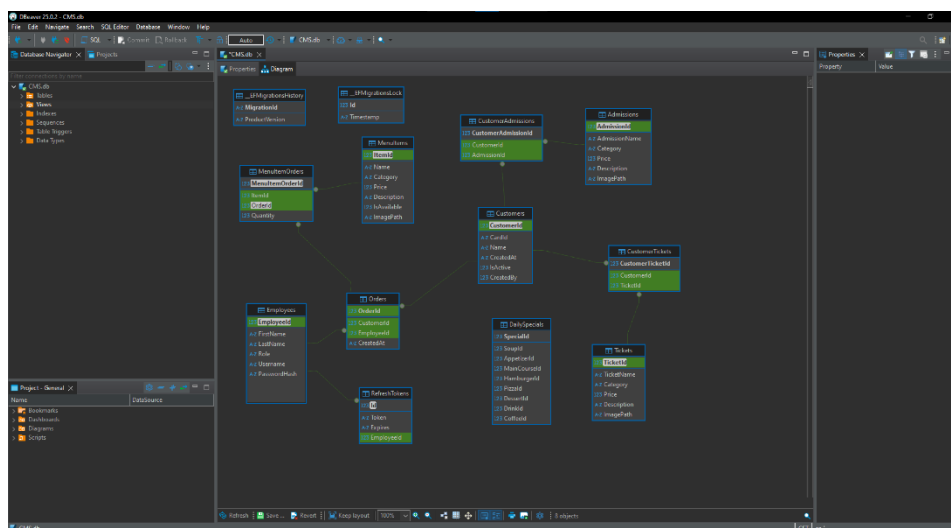
Dbeaver

Ingyenes, többplatformos adatbázis-kezelő eszköz, amely támogatja az *MSSQL*, *MySQL*, *PostgreSQL*, *SQLite* és más adatbázisokat, vizuális szerkesztővel és lekérdezési funkciókkal.

3 Fejlesztői dokumentáció

3.1 A programmal kapcsolatos követelmény

A program egy fürdő dolgozói számára lett kifejlesztve, figyelembe véve a felhasználói kör sajátosságait és igényeit. A fejlesztés során az összes lehetséges hibát és problémát feltérképeztük, amikor a felhasználó figyelmetlenségből vagy hibás döntés következtében súlyos problémákat okozhat. Ezek közé tartozhatnak kisebb működési hibáktól kezdve egészen a rendszer működését teljesen leállító vagy akár visszafordíthatatlan hibákig terjedő következmények. A kockázatok minimalizálása érdekében a programot alapos tesztelésnek vetettük alá, többféle tesztelési módszert alkalmazva annak érdekében, hogy a rendszer minden körülmény között megbízhatóan üzemeljen.



3. ábra - CMS.db adatbázis tábláink kapcsolatai DBeaver alkalmazásban

3.2 Frontend

3.2.1 Specifikáció

A *frontend* fejlesztése során elsődleges szempont volt a moduláris, átlátható kódbázis megteremtése, ezért komponens-alapú felépítést alkalmaztunk. A felhasználói élmény és a megjelenés egységességét modern, reszponzív dizájnelemek és gyors fejlesztői környezet biztosította.

Technológiák

- *React* és *TypeScript* alapú fejlesztés
- A felhasználói felület kialakításához *Material UI* komponensek alkalmazása
- A gyors fejlesztést biztosító *Vite* eszköz használata.

Funkcionalitások

- Felhasználói felület megjelenítése és interaktív kezelése
- Adatok megjelenítése, szűrése és keresése
- Űrlapok kitöltése és beküldése validációval
- Felhasználói hitelesítés és jogosultságkezelés
- *Backend API*-k hívása és az adatok dinamikus frissítése
- Hibakezelés és visszajelzés a felhasználók számára

Biztonsági szempontok

- Felhasználói adatok védelme és érzékeny információk bizalmas kezelése
- A hitelesítési és adatküldési folyamatok során titkosított kommunikáció (*HTTPS*) használata (kizárólag *buildelt* verziók esetén)
- A hibák visszajelzésekor nem pontosítjuk a hibát, (például bejelentkezésnél) csak általános hibaüzenetet jelenítünk meg.
- Az oldal frissítésekor a *token* automatikus frissítése és az inaktivitás miatti kijelentkeztetés megvalósítása

Teljesítmény-igények

- A *frontend* alkalmazás gyors betöltődési ideje és reszponzív működése biztosított
- Az oldalak és komponensek közötti navigáció zökkenőmentes, késleltetés nélküli élményt nyújt
- Az adatok lekérése és feldolgozása optimális, az API-hívások minimalizálása és gyors válaszidő biztosítása érdekében

Előre létrehozott felhasználók

Kizárólag a tesztelésre, létre lettek hozva felhasználók, melyek segítségével szimulálható a rendszer teljesértékű működése role-based alapon.

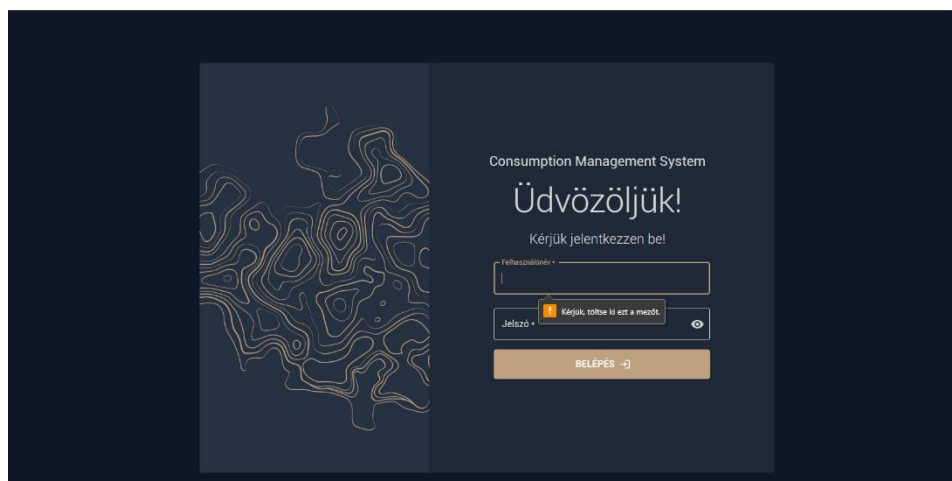
<u>Jogosultság</u>	<u>Felhasználónév</u>	<u>Jelszó</u>
Admin	admina	jelszo123
	adminb	
TicketAssistant	jegykezeloa	
	jegykezelob	
RestaurantAssistant	ettermesa	
	ettermesb	

3.2.2 Megjelenési követelmény

A tesztekhez használt képernyőképek 24 colos monitorokon készültek, míg a mobil nézetek iPhone 12 Pro *emulátor* segítségével. Ez a módszer biztosítja, hogy megfelelően ellenőrizhessük a felület reszponzív megjelenését különböző eszközökön.

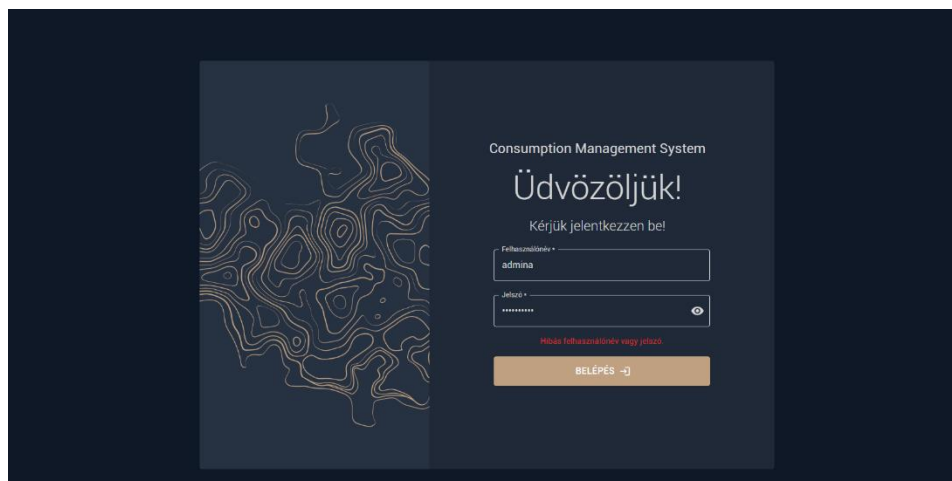
Bejelentkezési felület

A felület tartalmaz beviteli mezőket, ahol a felhasználónevet és a jelszót lehet megadni. Ezen kívül egy "Belépés" gomb is elérhető, amely lehetővé teszi a megadott adatok elküldését. Minden mező kitöltése kötelező, amennyiben anélkül próbálja a felhasználó elküldeni a bejelentkezési űrlapot, azt nem fogja engedni a program.



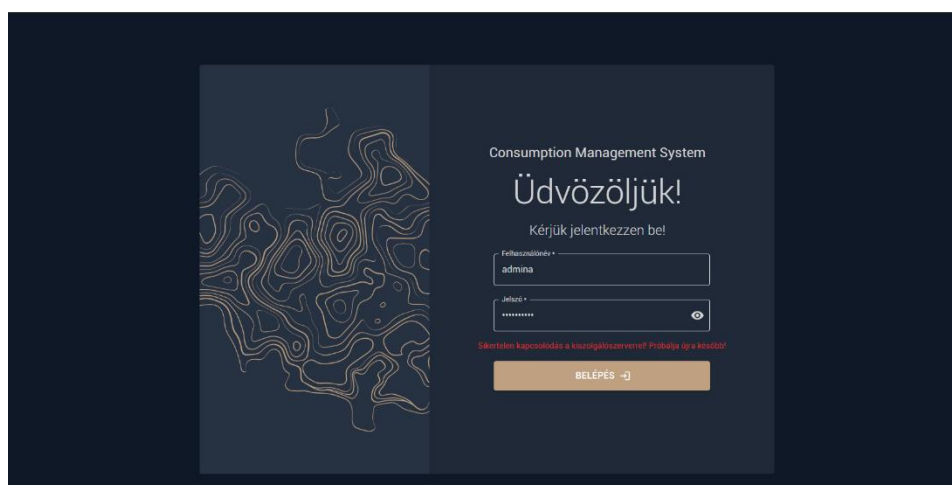
4. ábra - Bejelentkező oldal minden mező kitöltése kötelező

Amennyiben hibás vagy nem létező adat kerül megadásra, egy felugró figyelmeztető üzenet jelenik meg. „**Hibás felhasználónév vagy jelszó.**” Ezzel is támogatva, hogy nem közöljük a felhasználó részére, hogy mely adat volt pontosan a hibás, ezzel elkerülve egy esetleges bruteforce alapú támadást.



5. ábra - Bejelentkező oldal, rossz felhasználónév vagy jelszó hibakezelése

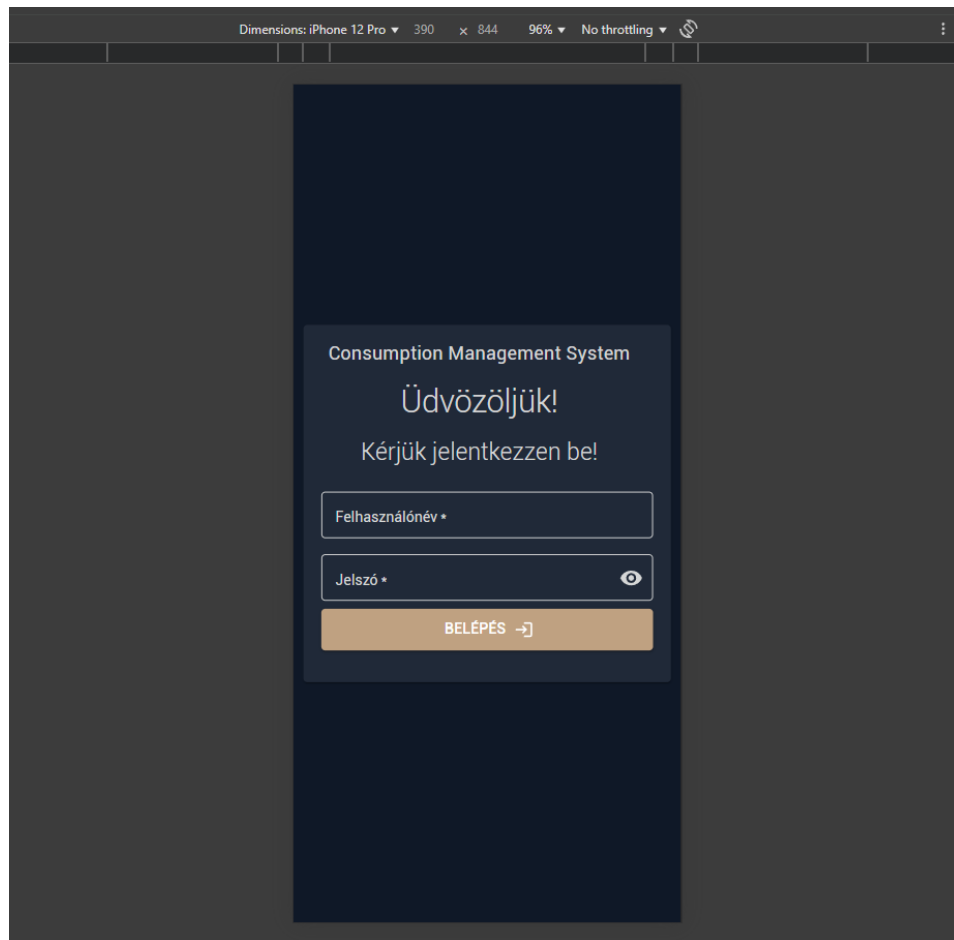
Azonban, ha a frontend valamilyen okból kifolyólag nem képes kapcsolatot létesíteni a *backend* kiszolgálóval, a bejelentkezési folyamat során specifikusabb hibaüzenet jelenik meg a felhasználó számára.



6. ábra – Bejelentkező oldal, Backenddel nem képes kialakítani kapcsolatot

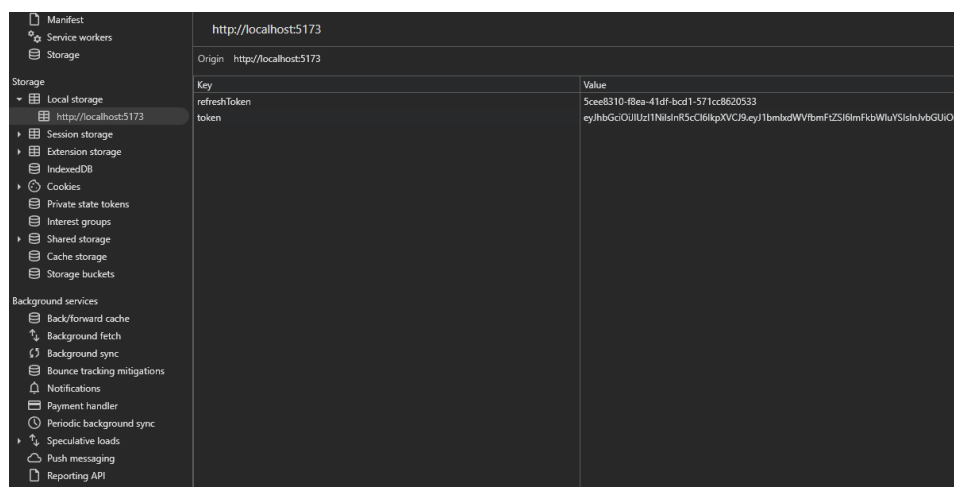
Kis képernyőfelbontású eszközökön, például mobiltelefonokon a bejelentkezési felület melletti kép nem kerül megjelenítésre, csupán maga a bejelentkező űrlap látható.

Sikeres bejelentkezést követően a *backend* válaszüzenetében lévő *token* és *refreshToken* értékeket eltárolja a *localStorage*-ba, mely lehetővé teszi a *token* mindenkor elérését, kijelentkezésig.



8. ábra - Bejelentkező oldal iPhone 12 Pro készüléken

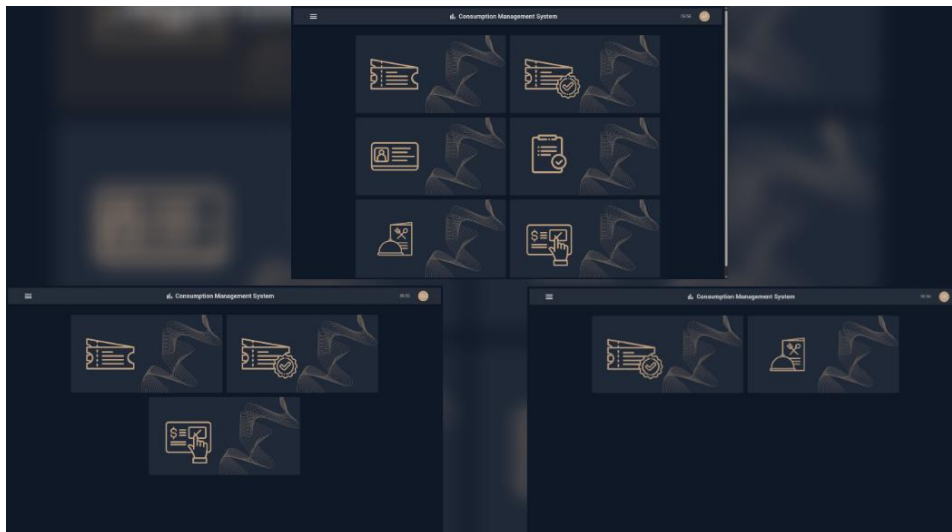
Sikeres bejelentkezést követően a *backend* válaszüzenetében lévő *token* és *refreshToken* értékeket eltárolja a *localStorage*-ba, mely lehetővé teszi a *token* mindenkor elérését, kijelentkezésig.



7. ábra - Böngésző local storage tartalma sikeres bejelentkezés után

Főoldal

Az oldalon, felhasználói jogosultságtól függően különböző kártyák jelennek meg, amelyek a többi felületre navigálnak. Amikor az egér a kártya fölé kerül, egy szöveg tűnik fel, amely az adott munkafelületet jelöli.



9. ábra - Főoldal megjelenési formái különféle jogosultságokkal

Annak érdekében, hogy a bejelentkezett felhasználók számára a számukra releváns kártyák jelenjenek meg, egy `useEffect` függvény kerül meghívásra, amely a komponens betöltődésekor eltárolja a `token`-ből kinyert felhasználói jogosultságot.

```
70 function App() {
71   const [role, setRole] = useState<string | null>(null);
72   const [isLoading, setIsLoading] = useState(true);
73
74   useEffect(() => {
75     const handleRole = async () => {
76       const token = localStorage.getItem("token");
77       if (token) {
78         const parsedToken = parseJwt(token);
79         setRole(parsedToken.role);
80         setIsLoading(false);
81       }
82     };
83
84     handleRole();
85   }, []);
86 }
```

10. ábra - Jogosultsági szint kinyerése a tokenből

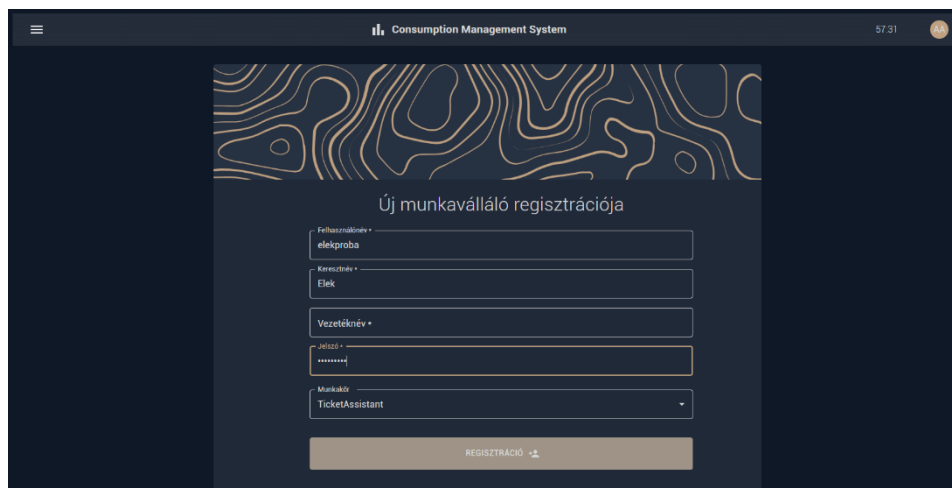
Például a következő komponens felel az Étterem kártya megfelelő megjelenítéséért attól függően, hogy a felhasználó szerepköre Admin vagy RestaurantAssistant. A kártya ezen felül tartalmaz egy képet és egy azonosítót, amely segítheti a további funkciók aktiválását. A megjelenítés reszponzívan alkalmazkodik a képernyő méretéhez, így mobil- és asztali nézetben is megfelelően elrendezett.

```
195 {role === "Admin" || role === "RestaurantAssistant" ? (  
196   <Box  
197     sx={{  
198       width: {  
199         xs: "100%",  
200         sm: "80%",  
201         md: "60%",  
202         lg: "45%",  
203         xl: "35%",  
204       },  
205       p: 1,  
206     }}  
207   >  
208     <Card sx={{ maxWidth: "100%", height: "auto" }}>  
209       {cardContent(  
210         "/img/landing/restaurant_temp.png",  
211         "Étterem",  
212         "restaurant"  
213       )}  
214     </Card>  
215   </Box>  
216 ) : null}
```

11. ábra - Az „Étterem” nevű kártya megjelenéséért felelős kódrészlet

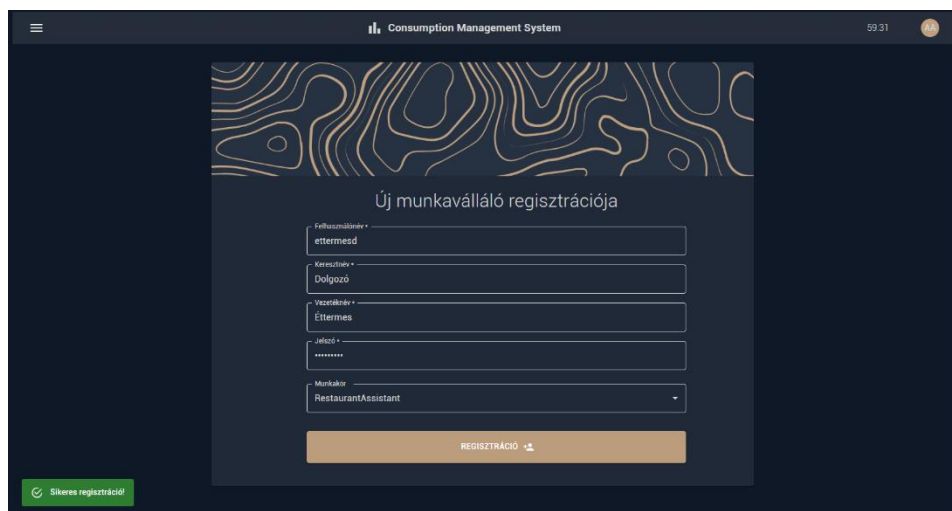
Regisztrációs felület

A felületen találhatóak felhasználónevet, vezetéknév, keresztnév, jelszót és feladatkört tartalmazó beviteli mezők. Ez a felület kizárólag az adminisztrátorok számára elérhető, azonban a kötelező mezők kitöltését a rendszer minden esetben. Ha bármely mező kitöltetlen a küldés gomb nem elérhető, ezzel elkerülve a hibás rögzítést.



12. ábra - Regisztrációs oldal, hibakezelése, üres mező esetén

A regisztrációs folyamat sikeres befejezését követően a rendszer egy felugró ablak (ún. *popup* értesítés) megjelenítésével tájékoztatja a felhasználót a regisztráció eredményességéről.



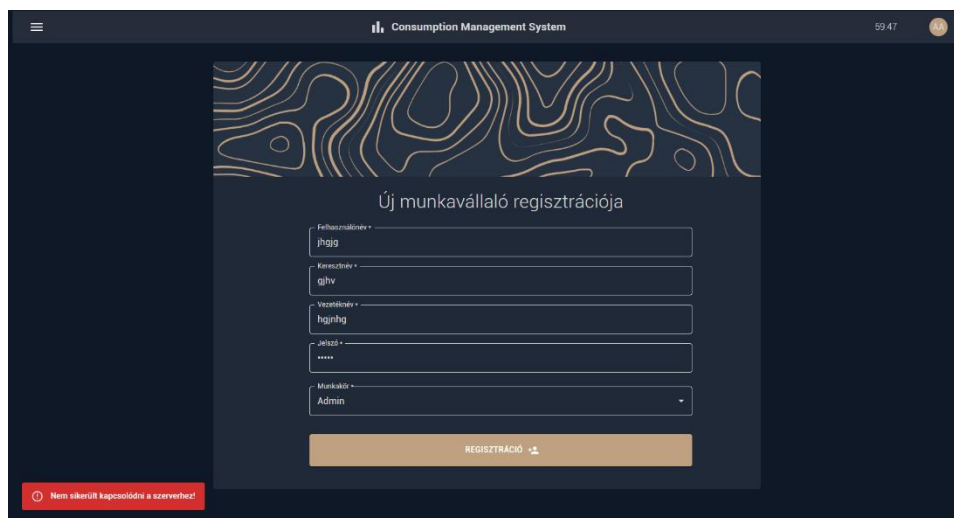
13. ábra - Regisztrációs oldal, Új dolgozó sikeresen létrejött

A regisztrációs folyamatot egy gomb indítja, amely elküldi a megadott adatokat az *API* végpontnak. A rendszer a *local storage*-ban tárolt *token*t használja a kérés hitelesítéséhez, amely *Bearer* típusú, valamint a fejlécben kerül elküldésre, ezzel is ellenőrizve a művelethez szükséges jogosultságot.

```
88     await axios.post(  
89         "https://localhost:5000/api/Auth/register",  
90         employeeData,  
91         {  
92             headers: {  
93                 Authorization: `Bearer ${localStorage.getItem("token")}`,  
94             },  
95         }  
96     );
```

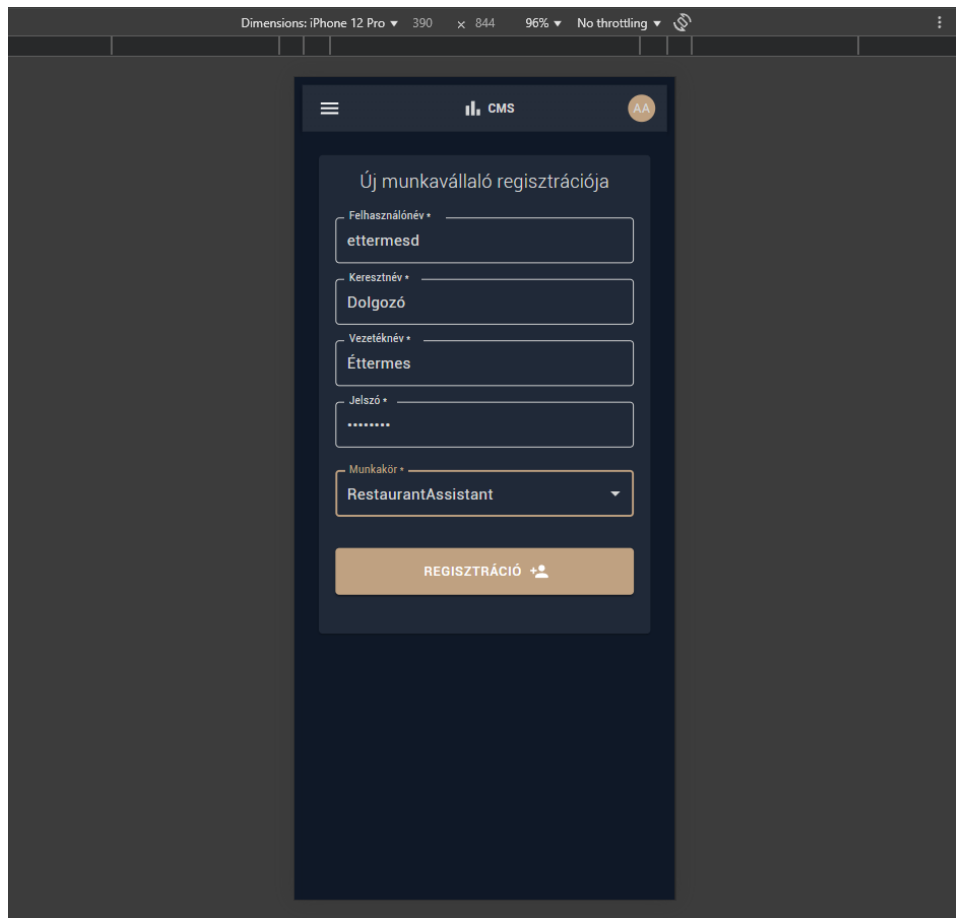
14. ábra - RegistrationPage.tsx-ben használt register végpont tokennel

Hiba esetén a rendszer megfelelően tájékoztatja a felhasználót a hiba forrásáról. Pl.: "Hiba történt a regisztráció során!" vagy "Nem sikerült kapcsolódni a szerverhez!".



15. ábra - Regisztrációs oldal, Backend elérésének hibája esetén

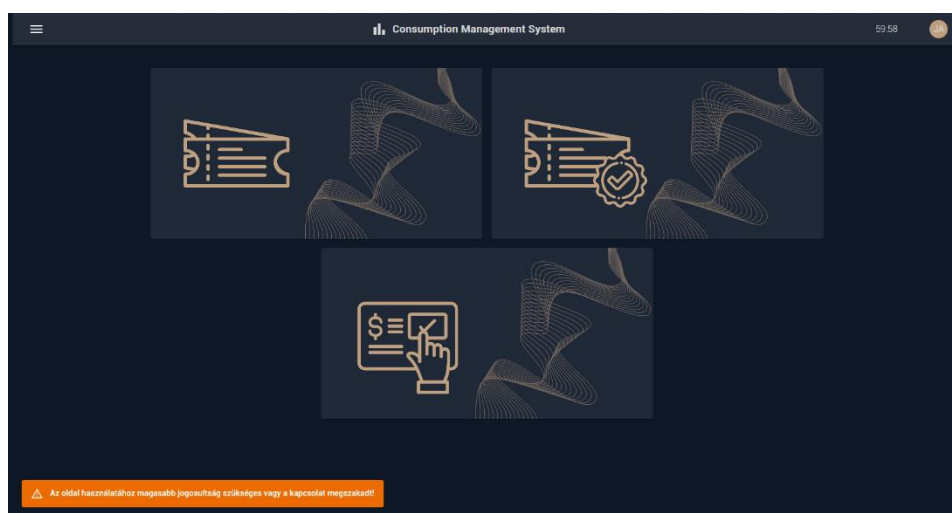
Kis képernyőn vagy mobil eszközön a regisztrációhoz tartozó kép nem jelenik meg, ezzel biztosítva a megfelelő nézetet kisebb képernyővel rendelkező eszközök számára.



16. ábra - Regisztrációs oldal, megjelenése iPhone 12 Pro készüléken

Étterem felület

Az oldal elérése kizárólag az Admin és a RestaurantAssistant szerepkörrel rendelkező felhasználók számára lehetséges. A végpontok védettek, így nem megfelelő felhasználói jogosultsággal semmilyen kérés nem küldhető az oldalra. Még abban az esetben sem, ha egy felhasználó ismeri a pontos *URL*-t. Amennyiben ilyen próbálkozás történik, a rendszer azonnal megjelenít egy figyelmeztető üzenetet „Magasabb felhasználói jogosultság szükséges vagy a kapcsolat megszakadt” szöveggel, majd automatikusan visszairányítja a felhasználót a főoldalra. Ez a megoldás garantálja, hogy az oldal funkciói és adatai csak az arra jogosult felhasználók számára legyenek elérhetők, fenntartva ezzel a rendszer biztonságát és integritását.



17. ábra - Jogosultság hiány miatti átirányítás a restaurant oldalról

A rendeléssávtól balra egy görgethető felület található, amely az ételek kártyáit jeleníti meg. Minden kártyán megtalálható az étel neve, annak típusa, egy kép az ételről, egy rövid leírás, az elérhetőségi státusz felirata, az ára, valamint egy plusz és mínusz gomb, amely lehetővé teszi a kosárban lévő darabszám növelését vagy csökkentését.

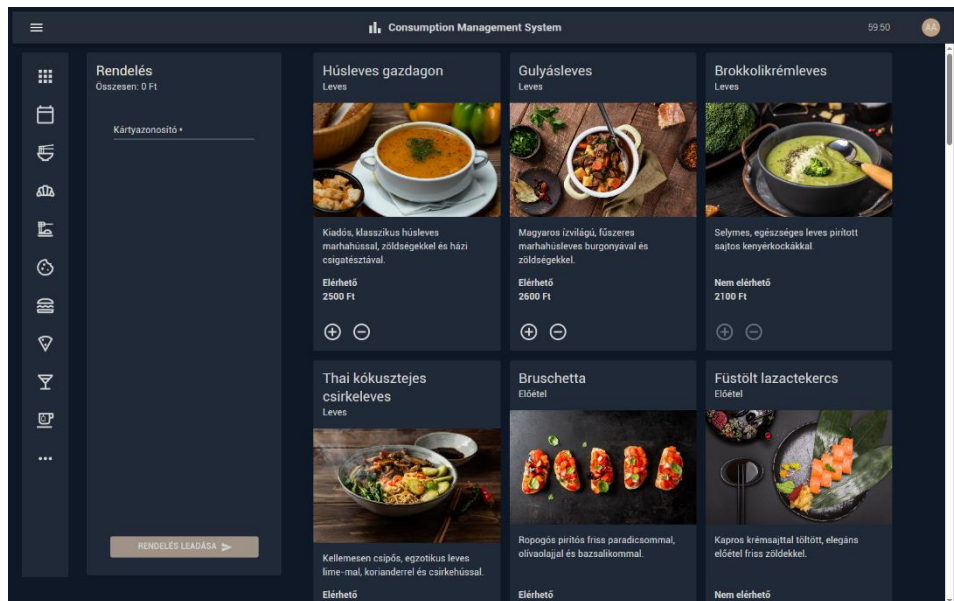


19. ábra - Éttermi oldal, Elérhető termékek kártyája



18. ábra - Éttermi oldal, Nem elérhető termék kártyája

Az éttermi felület célja, hogy hatékony módon biztosítsa az ételek böngészését és rendelések leadását. A bal oldalon található navigációs sáv, mely listázza az összes elérhető ételtípust. Abban az esetben, ha egy adott ételtípus nem kategorizálható, az bekerül az egyéb (három pont) gomb alá. Az ikonokra kattintva megjelennek szűrve a kategória elemei. A felhasználók számára rendelkezésre áll két speciális ikon: az egyik az összes ételt listázza, míg a másik az aktuális napi ajánlatot.



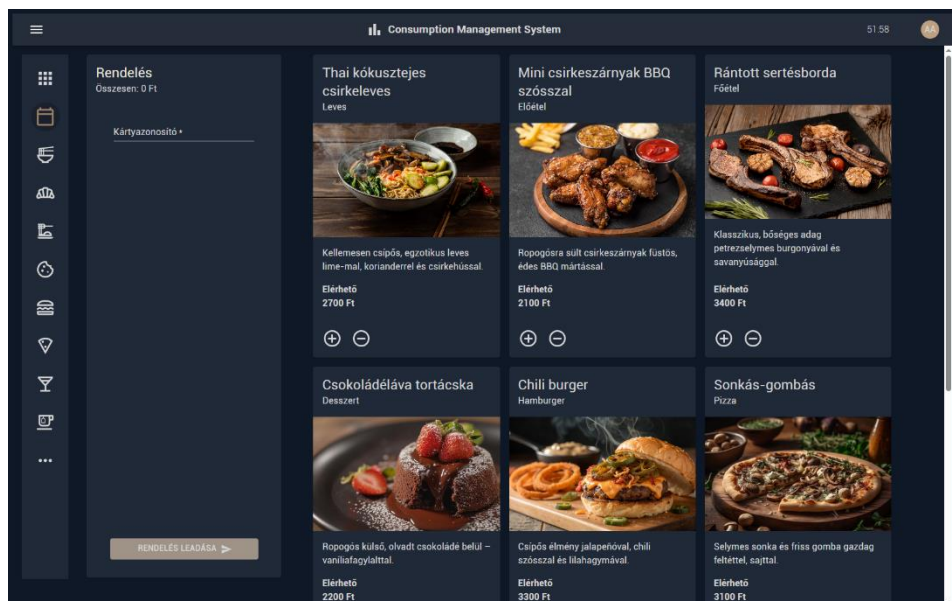
20. ábra - Éttermi oldal felépítése

Az oldal betöltésekor, a *token*ek validálása és megújítása után lefut egy *GET* függvény, amely az adatbázisból lekéri az összes elérhető menüelemet

```
179     try {
180       const decodedToken = jwt(token);
181       if (
182         !decodedToken ||
183         (decodedToken.role !== "Admin" &&
184          decodedToken.role !== "RestaurantAssistant")
185       ) {
186         console.error(
187           "Unauthorized access: Admin or RestaurantAssistant role required"
188         );
189         setIsUnauthorized(true);
190         return;
191       }
192
193       const isValidToken = await checkToken();
194       if (!isValidToken) {
195         console.error("Invalid token. Redirecting to login...");
196         setIsUnauthorized(true);
197         return;
198       }
199
200       await refreshToken();
201
202       const newToken = localStorage.getItem("token");
203       if (!newToken) {
204         throw new Error("Token refresh failed");
205       }
206
207       const menuResponse = await axios.get(
208         "https://localhost:5000/api/MenuItems",
209         {
210           headers: {
211             Authorization: `Bearer ${newToken}`,
212           },
213         }
214       );
```

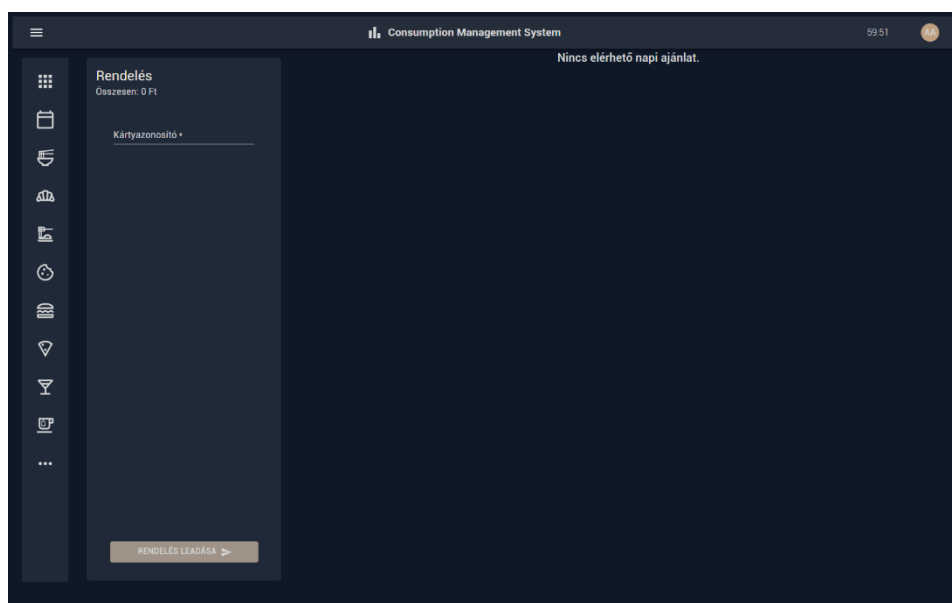
21. ábra – OrderPage.tsx, Menüelem végpont meghívása

Amennyiben a napi ajánlat opció kerül megnyitásra, akkor egy másik végpontról kerül lekérdezésre az Adminisztrátor által beállított napi ajánlat.



22. ábra - Éttermi oldal, Napi ajánlat fül, ha van beállítva ajánlat.

Amennyiben nem volt beállítva napi ajánlat a következő oldal jelenik meg a felhasználó részére.



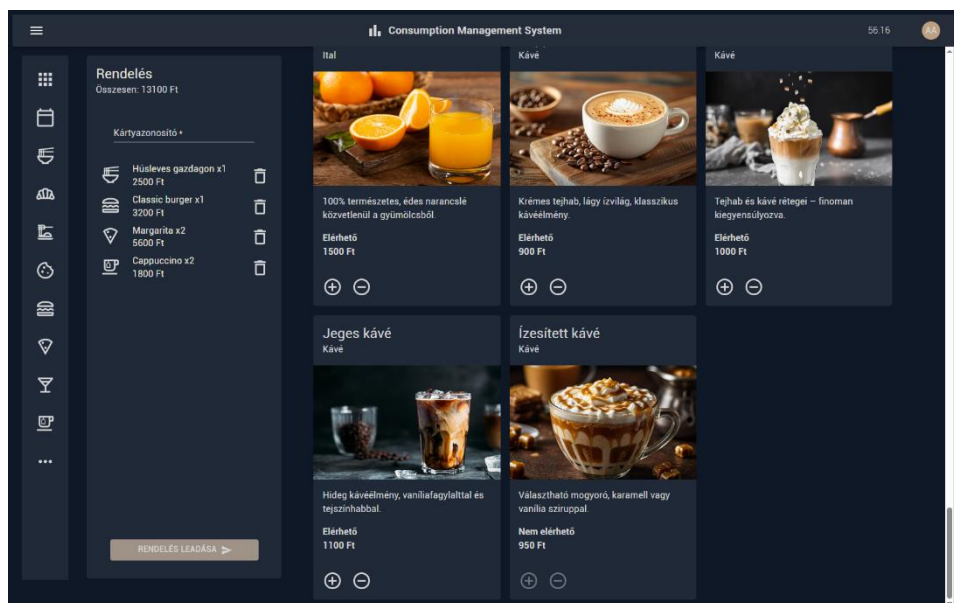
23. ábra - Éttermi oldal, Napi ajánlat fül, ha nincs beállítva ajánlat

A napi ajánlathoz tartozó kódrészlet, mely felel a megfelelő lekérdezésért, és hibakezelésért egyaránt.

```
89 const handleCategoryClick = (category: string | null) => {
90   setSelectedCategory(category);
91   if (category === "Napi ajánlat") {
92     axios
93       .get("https://localhost:5000/api/DailySpecials", {
94         headers: {
95           "Content-Type": "application/json",
96           Authorization: `Bearer ${localStorage.getItem("token")}`,
97         },
98       })
99
100     .then((response) => {
101       const specials = response.data;
102       const filteredSpecials = menuItems.filter((item) =>
103         [
104           specials[0].soupName,
105           specials[0].appetizerName,
106           specials[0].mainCourseName,
107           specials[0].hamburgerName,
108           specials[0].pizzaName,
109           specials[0].dessertName,
110           specials[0].drinkName,
111           specials[0].coffeeName,
112         ].includes(item.name)
113       );
114       setDailySpecials(filteredSpecials);
115     })
116     .catch((error) => {
117       console.error(
118         "Hiba történt a napi ajánlatok betöltése közben:",
119         error
120       );
121       setDailySpecials([]);
122     });
123   }
124 }
```

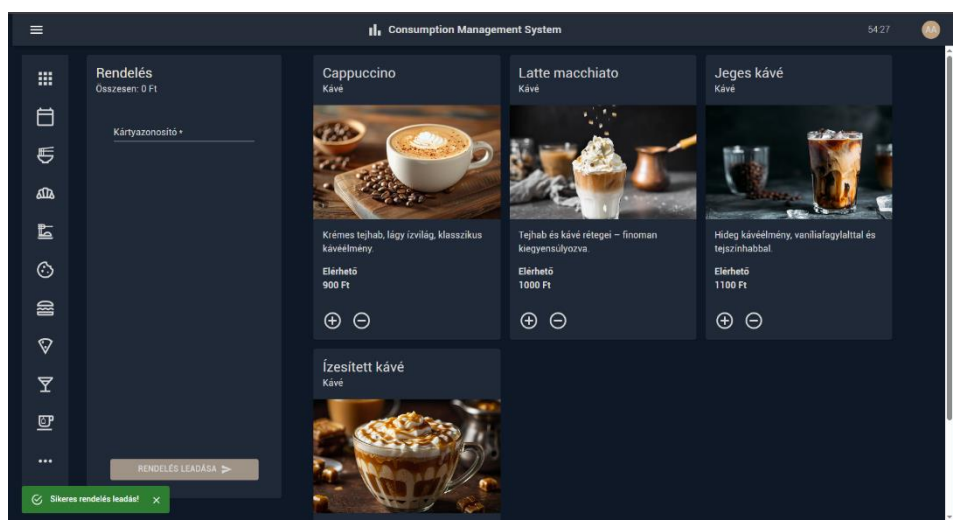
24. ábra - OrderPage.tsx, Napi ajánlat lekérdezése

A rendelés sáv a folyamatban lévő rendelés tételeit mutatja, beleértve a tételek árának összegét. Az árak alatt egy beviteli mező található, ahová a kártyaazonosítót kell megadni. Az aktuális rendelés tételei alatta jelennek meg. Egy rendelési elem a következő komponensekből áll: baloldalt az étel típusának ikonja, az étel neve, a darabszám (ami mutatja, hogy hány darabot rendeltek az adott ételből), az ár, valamint egy törlés gomb, amely lehetővé teszi az adott tétel törlését. A rendelés sáv alján található a „Rendelés leadása” gomb, amely addig inaktív marad, amíg nincs megadva a kártyaazonosító és legalább egy tétel nem szerepel a kosárban.



25. ábra - Éttermi oldal, Folyamatban lévő rendelés

A „Rendelés leadása” gomb megnyomásakor egy *POST* függvény kerül meghívásra, amely az adatbázisba továbbítja a rendelés adatait, visszajelzi a felhasználó számára, hogy sikeres volt-e a rendelés, majd a kosár automatikus ürítése történik.



26. ábra - Éttermi oldal, Sikeresen éttermi rendelés

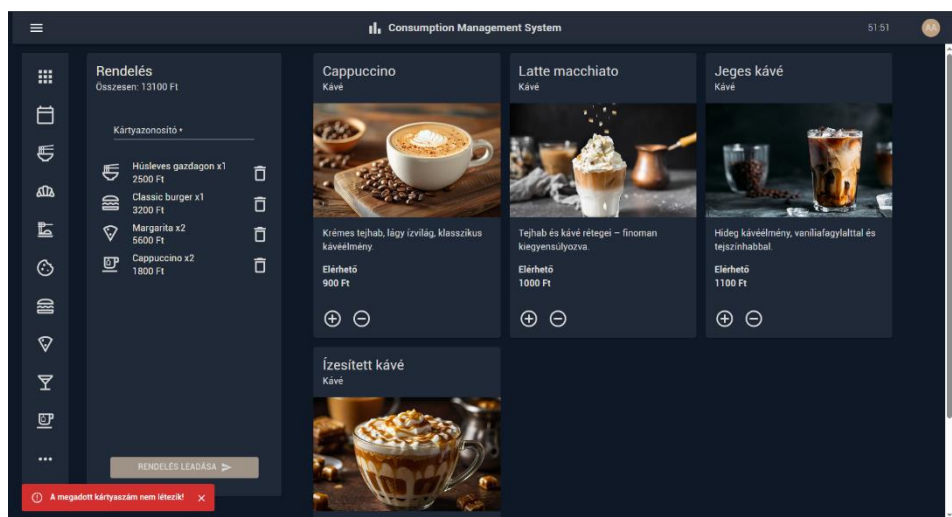
Rendelés leadása az alábbiak szerint valósul meg:

- Kártya ID visszakeresés *GET* kéréssel ID-ra,
- Kosár tartalmának elküldése *POST* segítségével


```
258     axios
259       .get("https://localhost:5000/api/Cards/GetCustomerIdByCardId/" + cardId, {
260         headers: {
261           Authorization: `Bearer ${localStorage.getItem("token")}`,
262         },
263       })
264       .then((response) => {
265         if (!isNaN(response.data)) {
266           const token = localStorage.getItem("token");
267           let decodedToken: { EmployeeId: number } | null = null;
268           if (token) {
269             decodedToken = parseJwt(token);
270           }
271           const orderData = {
272             customerId: response.data,
273             employeeId: decodedToken?.EmployeeId || "",
274             menuItems: orders.map((orderItem) => ({
275               menuItemId: orderItem.itemId,
276               quantity: orderItem.quantity,
277             })),
278           };
279
280           axios
281             .post("https://localhost:5000/api/Orders", orderData, {
282               headers: {
283                 Authorization: `Bearer ${localStorage.getItem("token")}`,
284               },
285             })
286             .then((response) => {
287               console.log(response.data);
288               if (response.status === 201) {
289                 setOrderSuccess(true);
290                 setOrders([]);
291                 setCardId("");
292                 setTimeout(() => {
293                   setOrderSuccess(false);
294                 }, 6000);
295               }
296             })
297             .catch((error) => {
298               setDataLoadError(error);
299             });
```

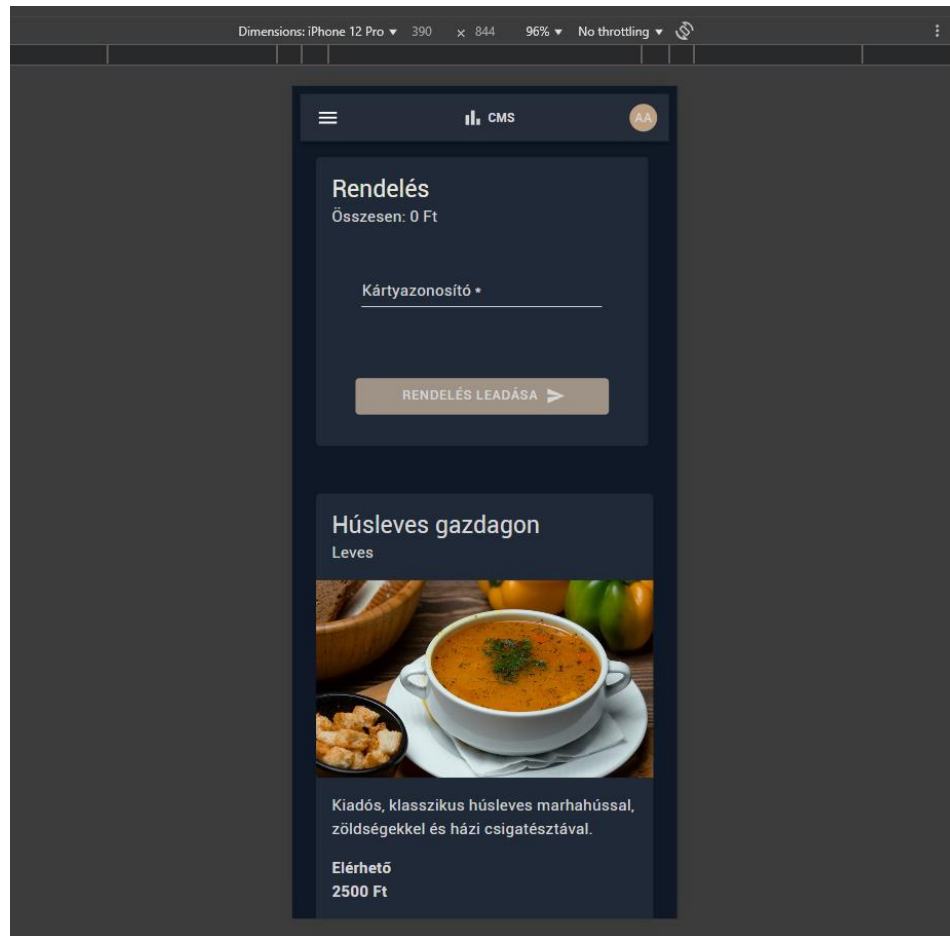
27. ábra - OrderPage.tsx, Rendelés elküldéséhez használt végpont

Hibás kártyaazonosító esetén csak a kártyaazonosító mezője törlődik, és jelzi a hibát. Sikertelen rendelés esetén szintén egy értesítés jelenik meg, amely a problémáról ad visszajelzést.



28. ábra - Éttermi oldal, Nem létező kártyaazonosító használta esetén

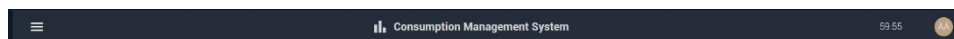
Kiseb kijeelzővel rendelkező eszközökön a szűrő opció eltűnik, a kártyák esztétikus megjelenése miatt, ezzel biztosítva a megfelelő nézetet.



29. ábra - Éttermi oldal, iPhone 12 Pro készüléken

Navigációs Sáv

A navigációs sáv, amely az oldal tetején található, minden oldalon megjelenik a bejelentkező oldal kivételével. A sáv bal oldalán található egy lenyíló lista, amely felsorolja az összes olyan oldalt, amelyek elérhetők a főképernyőn. Az elérhető oldalak listája a felhasználói jogosultságoktól függően változik, tehát csak azok a menüpontok jelennek meg, amelyekhez a felhasználónak van hozzáférése. A sáv középső részén látható az alkalmazás logója, valamint az alkalmazás neve.



30. ábra - Navigációs sáv nagy képernyős eszközökön



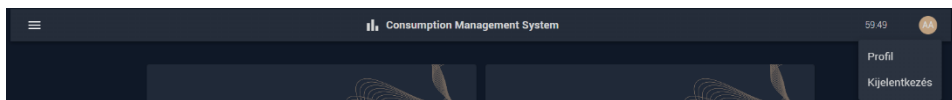
31. ábra - Navigációs sáv kisebb kijelzőkön

A navigációs sáv jobb oldalán egy visszaszámláló kap helyet, amely egy órától számol vissza. Amint ez az idő letelik, a felhasználót automatikusan kijelentkezteti a rendszer. Minden oldal frissítéskor (*token*) frissítésnél a visszaszámláló újraindul, és ismét egy órától kezdi a visszaszámlálást. Az időt a program a *tokenben* eltárolt lejáratí időből és az aktuális pontos időből számítja ki, az ehhez használt *JWT parser*.

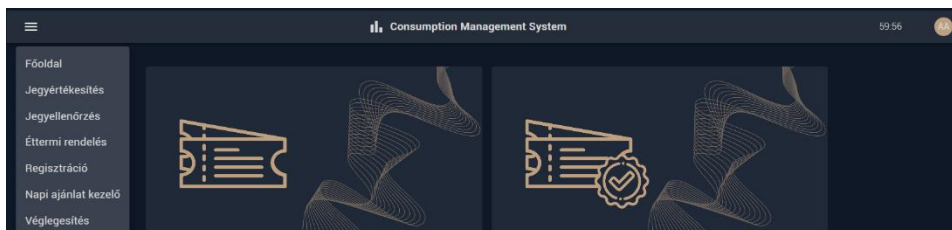
```
5 const CountdownTimer: React.FC = () => {  
6   const [timeLeft, setTimeLeft] = useState<number | null>(null);  
7  
8   useEffect(() => {  
9     const checkToken = () => {  
10      const token = localStorage.getItem("token");  
11      if (!token) {  
12        setTimeLeft(null);  
13        return;  
14      }  
15  
16      const decodedToken = parseJwt(token);  
17      if (!decodedToken || !decodedToken.exp) {  
18        setTimeLeft(null);  
19        return;  
20      }  
21  
22      const expirationTime = decodedToken.exp * 1000;  
23      const currentTime = Date.now();  
24      const remainingTime = expirationTime - currentTime;  
25      setTimeLeft(Math.max(remainingTime, 0));  
26    };  
27  
28    checkToken();  
29    const interval = setInterval(checkToken, 1000);  
30  
31    window.addEventListener("storage", checkToken);  
32  
33    return () => {  
34      clearInterval(interval);  
35      window.removeEventListener("storage", checkToken);  
36    };  
37  }, []);  
38  
39  const formatTime = (milliseconds: number): string => {  
40    const totalSeconds = Math.floor(milliseconds / 1000);  
41    const minutes = Math.floor(totalSeconds / 60);  
42    const seconds = totalSeconds % 60;  
43    return `${(minutes):${(seconds.toString().padStart(2, "0"))}}`;  
44  };  
45  
46  return (  
47    <Box sx={{ mr: 2, display: "flex", alignItems: "center" }}>  
48      <Typography  
49        variant="body2"  
50        sx={{  
51          color: timeLeft && timeLeft < 300000 ? "#ff4444" : "#d5d6d6",  
52          fontSize: "1rem",  
53          fontWeight: "300",  
54        }}>  
55    </Typography>  
56  </Box>);  
57
```

32. ábra - NavBar.tsx, Visszaszámláló funkció

A navigációs sáv jobb szélső részén egy profil ikon jelenik meg, amely a bejelentkezett felhasználó monogramját tartalmazza. Az ikonra kattintva egy lenyíló lista jelenik meg két elemmel: az egyik a kijelentkezés funkció, a másik pedig a profil elérése.



33. ábra - Navigációs sáv, Felhasználói funkciók



34. ábra - Navigációs sáv, Navigálási funkciók

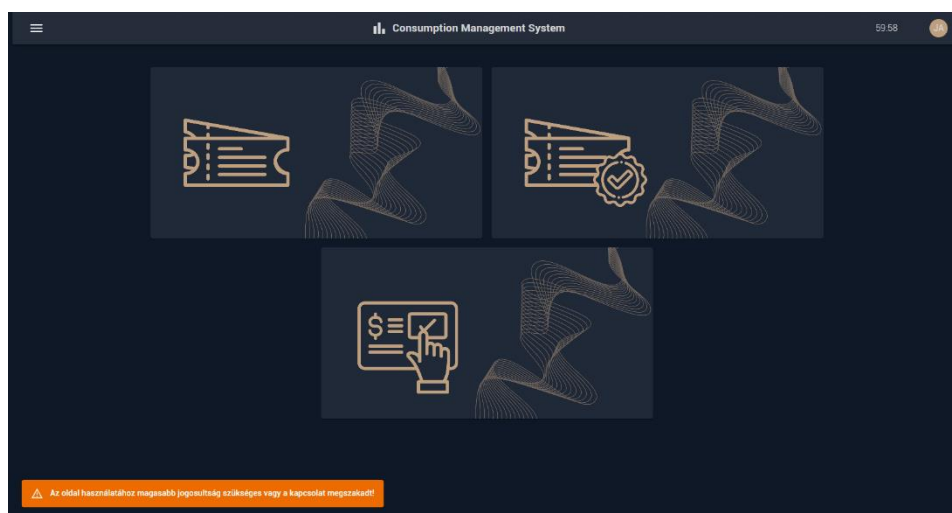
Kijelentkezéskor a rendszer elsőként meghívja a `/logout` végpontot, hogy érvénytelenítse a felhasználó `session`-jét, majd törli a `token` és `refreshToken` adatait a `localStorage`-ból.

```
53 const handleCloseUserMenu = async (setting: string) => {  
54   setAnchorElUser(null);  
55   if (setting === "Profil") {  
56     window.location.href = "/profileEmployee";  
57   }  
58   if (setting === "Kijelentkezés") {  
59     const token = localStorage.getItem("token");  
60     if (token) {  
61       try {  
62         await axios.post(  
63           "https://localhost:5000/api/Auth/logout",  
64           {},  
65           {  
66             headers: {  
67               "Content-Type": "application/json",  
68               Authorization: `Bearer ${token}`,  
69             },  
70           },  
71         );  
72       } catch (error) {  
73         console.error("Logout failed", error);  
74       }  
75     }  
76     localStorage.removeItem("token");  
77     localStorage.removeItem("refreshToken");  
78     window.location.href = "/login";  
79   }  
80 };
```

35. ábra - Navbar.tsx - Kijelentkeztetési algoritmus

Jegyértékesítő felület

A jegyértékesítési oldal elérése az Admin és TicketAssistant szerepkörhöz kötött, ugyanúgy, mint az éttermi felület esetében. Jogosultság hiányában semmilyen kérés nem küldhető az oldal irányába, még ismert *URL* esetén sem. Ilyen próbálkozáskor a rendszer figyelmeztető üzenetet jelenít meg, majd visszairányít a főoldalra, ezzel biztosítva a rendszer védelmét és az adatok biztonságát.



36. ábra - Jogosultság hiány miatti átirányítás a tickets oldalról

A rendelés sáv melletti görgethető felületen a különböző jegytípusok kártyái jelennek meg. Minden kártya tartalmazza a jegy nevét, típusát, képét, árát, valamint egy gombot, amely a jegy kosárba helyezésére szolgál. Alapértelmezett megjelenítéskor három kártya látható egymás mellett, a megjelenés azonban a képernyő méretéhez igazodva automatikusan módosul.

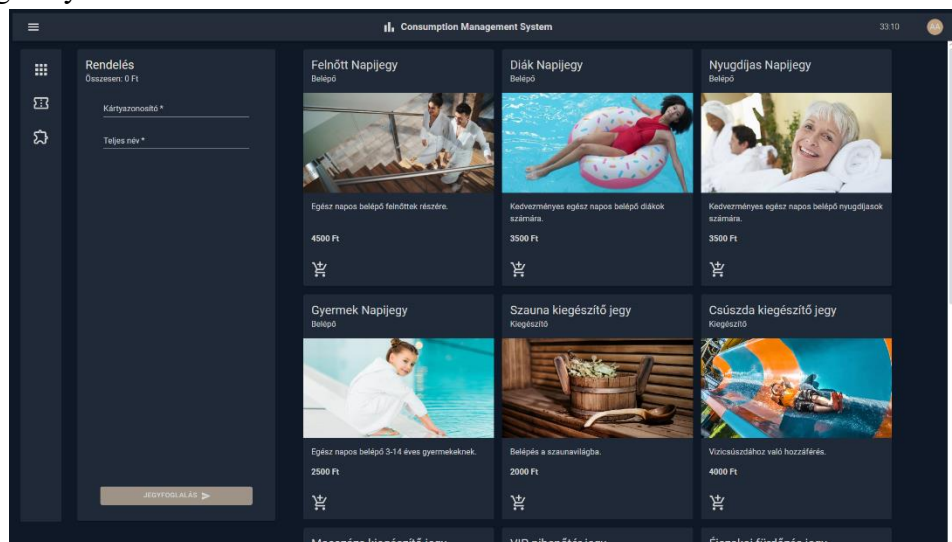


38. ábra - Jegyértékesítő oldal, mint a egy belépő típusú jegyre



37. ábra - Jegyértékesítő oldal, mint a egy kiegészítő típusú jegyre

A felület célja, hogy átlátható és egyszerű módot nyújtson a jegyek böngészésére és foglalására. A bal oldali navigációs sávban a felhasználók kategóriák szerint láthatják az elérhető jegytípusokat. Az ikonokra kattintva kizárólag az adott kategóriához tartozó jegyek jelennek meg a középső szolgáltatásfelületen. Egy felhasználó egyszerre csak egy darab belépőjegyet helyezhet a kosárba. Amennyiben megpróbál többet hozzáadni, a rendszer ezt nem engedélyezi.



39. ábra - Jegyértékesítő oldal, Felülete

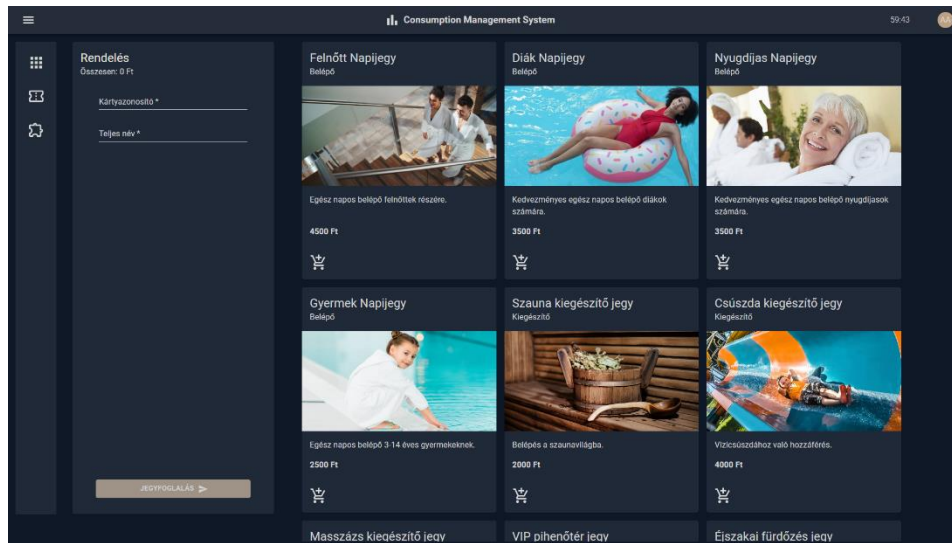
Az oldal betöltésekor – a jogosultságok ellenőrzése és a *token* érvényességének vizsgálata, illetve frissítése után – két *GET* kérés fut le az adatbázis felé. Az egyik lekéri az összes elérhető belépőjegyet (*/api/Tickets*), míg a másik a kiegészítő jegyeket (*/api/Admissions*). Mindkét lekérdezéshez szükséges a megfelelő jogosultság és érvényes *token* megléte.

```
177 const fetchTickets = axios.get("https://localhost:5000/api/Tickets", {
178   headers: {
179     Authorization: `Bearer ${token}`,
180   },
181 });
182
183 const fetchAdmissions = axios.get(
184   "https://localhost:5000/api/Admissions",
185   {
186     headers: {
187       Authorization: `Bearer ${token}`,
188     },
189   }
190 );
```

40. ábra – TicketPage.tsx, Jegyek lekérése a két GET végpontról

A rendelés sávban jelennek meg a folyamatban lévő rendelés tételei, beleértve a teljes fizetendő összeget is. Az árak alatt két beviteli mező található: az egyik a kártyaazonosító, a másik a teljes név megadására szolgál. Ezek alatt sorolódnak fel a kosárba helyezett tételek. Egy rendelési tétel a következőket tartalmazza: a jegy típusa, megnevezése, ára, valamint egy törlés gomb, amely lehetővé teszi az adott tétel eltávolítását.

A rendelés sáv legalján található a „Jegyfoglalás” gomb, amely mindaddig inaktív (*disabled*) marad, amíg a kosárban nincs legalább egy belépőjegy, és az adatmezők nincsenek kitöltve.



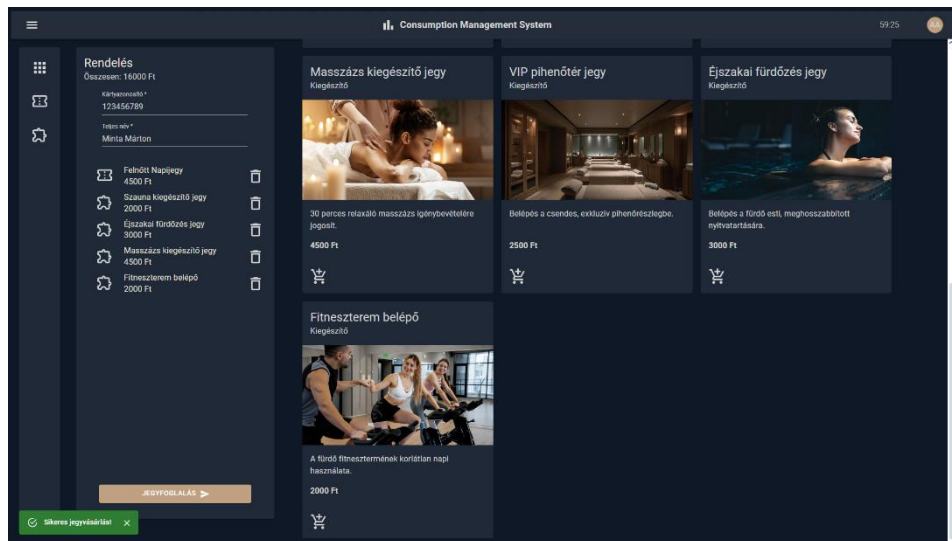
41. ábra – Jegyvásárló oldal felülete

A „Jegyfoglalás” gomb aktiválásakor egy *POST* kérés kerül elküldésre, amely továbbítja a rendelés adatait az adatbázis felé. A kérés fejlécében szerepel a jogosultságot igazoló *Bearer token* is. Az összeállított adatokat a rendszer elküldi a */api/Customers* végpontra egy *POST* kérés formájában.

```
116 const handleSubmitOrder = () => {  
117   const token = localStorage.getItem("token");  
118   let decodedToken: { EmployeeId: number } | null = null;  
119   if (token) {  
120     decodedToken = parseJwt(token);  
121   }  
122   const customerData = {  
123     cardId: cardId,  
124     name: fullName,  
125     createdBy: decodedToken?.EmployeeId || "",  
126     isActive: true,  
127     ticketsIds: orders.map((ticketItem) => ({  
128       ticketId: ticketItem.ticketId,  
129     })),  
130     admissions: orders.map((admissionItem) => ({  
131       admissionId: admissionItem.admissionId,  
132     })),  
133   };  
134  
135   axios  
136     .post("https://localhost:5000/api/Customers", customerData, {  
137       headers: {  
138         Authorization: `Bearer ${localStorage.getItem("token")}`,  
139       },  
140     })  
141     .then(() => {  
142       handleClick();  
143     })  
144     .catch((error) => {  
145       console.error("Error creating customer:", error);  
146       setError(error.message);  
147     })  
148     .finally(() => {  
149       setLoading(false);  
150     });  
151   };
```

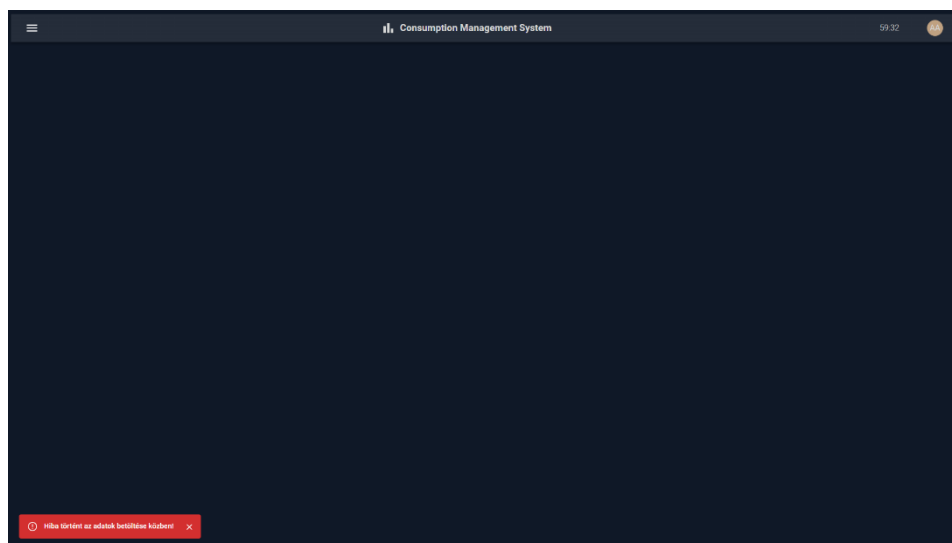
42. ábra - TicketsPage.tsx, Új vendég létrehozása

Sikeres vásárlás esetén a rendszer meghív egy visszajelző függvényt (*handleClick*), amelyre egy „Sikeres jegyvásárlás” alert jelenik meg.



43. ábra – Jegyértékesítő oldal, Sikeres jegy vásárlás

Sikertelen vásárlás esetén a hibát megjeleníti a felületen.

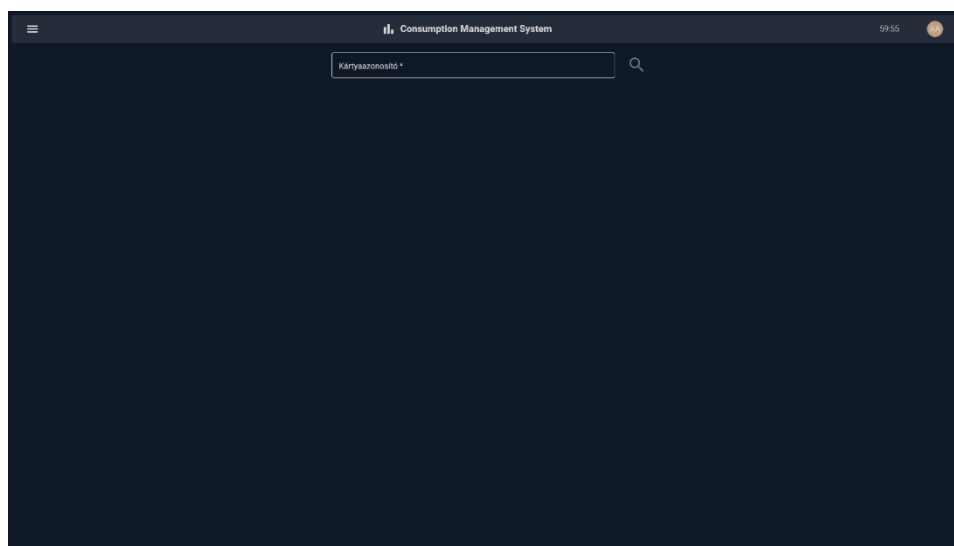


44. ábra - Jegyértékesítő oldal, Backendről az adatok nem érkeznek meg

Jegyellenőrzés

A felület minden belépett felhasználó számára elérhető. A célja, hogy ellenőrizhető legyen egy adott vendég belépőjegye és az ahhoz tartozó kiegészítő szolgáltatások. A felhasználónak be kell vinnie a vendég azonosítóját egy beviteli mezőbe, amely alapján a rendszer lekérdezi az adatbázisból a jegy információit.

Különösen hasznos funkció, ha egy vendég elveszíti a jegyét vagy talált jegyet szeretnének azonosítani - ilyenkor a rendszerben visszakereshető, hogy kihez tartozik az adott azonosító. Ezen felül lehetővé teszi a jogosultságok azonnali ellenőrzését is, így egyértelműen megállapítható, hogy a vendég milyen szolgáltatásokra jogosult (például szauna használata).



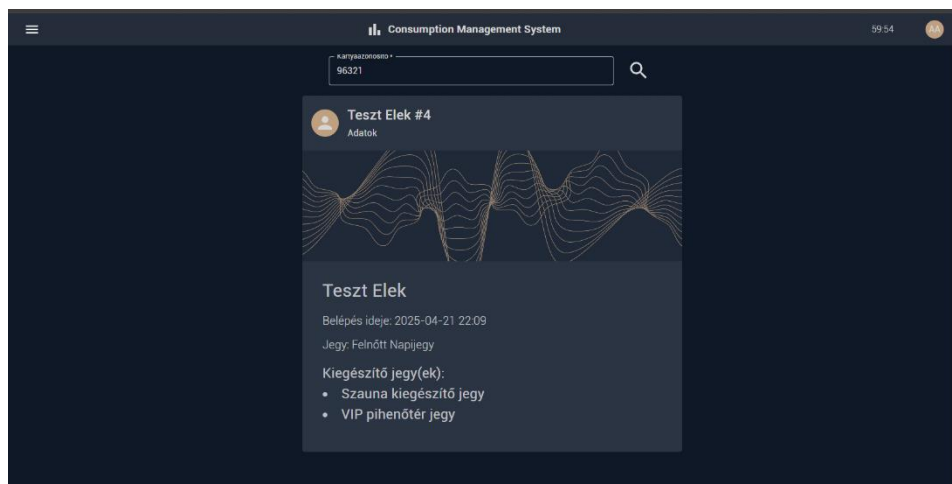
45. ábra - Jegyellenőrző oldal felülete

A lekérdezés során a rendszer egy védett *GET* végpontot hív meg, amely visszaadja az adott azonosítóhoz tartozó jegy adatokat. A képernyőn megjelenik a felhasználó neve, belépési ideje, jegytípusa, valamint a kiegészítő jegyek listája.

```
117 const fetchCustomerData = () => {
118   setLoading(true);
119   const token = localStorage.getItem("token");
120
121   axios
122     .get(`https://localhost:5000/api/Customers/${customerId}`, {
123       headers: {
124         Authorization: `Bearer ${token}`,
125       },
126     })
127     .then((response) => {
128       setCustomer(response.data);
129       setLoading(false);
130     })
131     .catch((error) => {
132       if (error.response && error.response.status === 404) {
133         setError("A megadott kártyaazonosító nem található!");
134       } else if (error.response && error.response.status === 401) {
135         setError("Az oldal használatához magasabb jogosultság szükséges!");
136       } else {
137         setError(error.message);
138       }
139     })
140 }
```

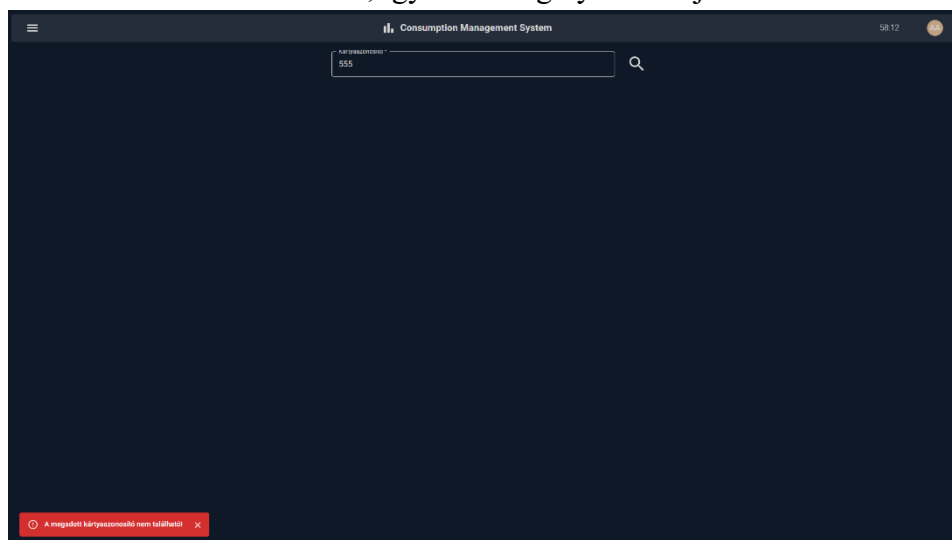
46. ábra - TicketValidationPage.tsx, Vendég adatainak lekérdezése *GET* végpont segítségével

A felületen a felhasználó azonosítója és profilja jól láthatóan megjelenik pl.: "Teszt Elek #4" formában.



47. ábra – Jegyellenőrző oldal, Kártya adatainak lekérdezése

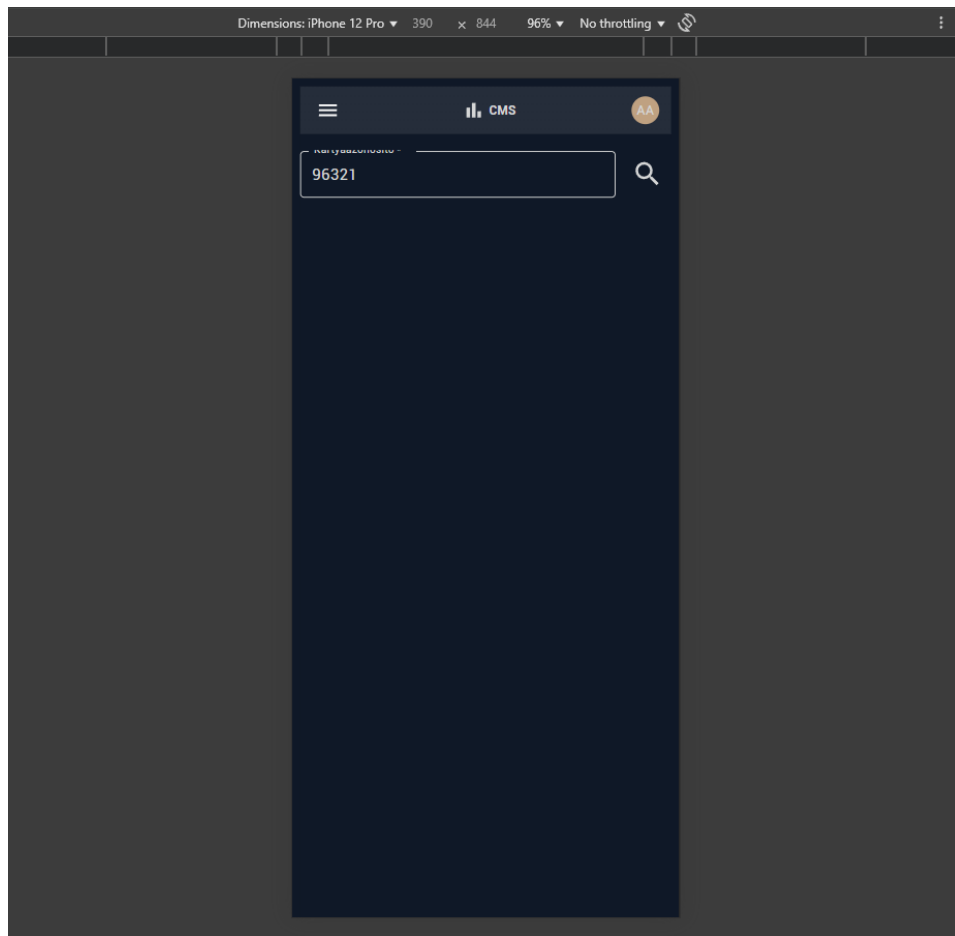
Ha a megadott kártyaazonosító nem található, a rendszer figyelmeztető üzenetet jelenít meg, majd automatikusan frissíti az oldalt, így lehetőség nyílik az újbóli keresésre.



48. ábra - Jegyellenőrző oldal, Kártya nem létezik

A felület követi a program arculati sémáját, sötét háttérrel és kontrasztos elemekkel, amelyek megkönnyítik az információk gyors áttekintését. A felhasználói felület középpontjában található adatkártya jól elkülönül a háttértől.

Mobil eszközökön és kisebb kijelzőkön a felület automatikusan átrendeződik: az elemek vertikális elrendezést vesznek fel, így minden információ jól olvasható marad és könnyedén hozzáférhető az érintőképernyő használat során is.



49. ábra - Jegyellenőrző oldal, iPhone 12 Pro készüléken

Napi ajánlat készítő felület

A felület kizárólag az adminisztrátorok számára elérhető. Itt van lehetőség a napi ajánlat beállítására, amelyet az éttermi dolgozók látnak, és ez alapján tudják ajánlani a vendégeknek az adott napi menüt. A felületen külön mezők találhatók a leves, főétel, előétel, pizza, hamburger, desszert, ital és kávé kiválasztására. A mezők legördülő listákként jelennek meg, és ezekből lehet kiválasztani a megfelelő elemet. Ezentúl lehetőség nyílik törölni a napi ajánlatot, így az éttermi rendszerben nem lesz elérhető napi ajánlat.

50. ábra - Napi ajánlat készítő oldal felülete

Az oldal képes figyelembe venni a készleten nem lévő termékeket is, így olyan nem lehet a napi ajánlat része, ami nem elérhető státuszban van. A `/api/MenuItems` végpontra egy `GET` kérést intéz az oldal betöltődésekor, majd a visszaérkező `json` alapját bekegategorizálja (`item.category`) kategória szerint és le ellenőrzi az elérhetőségét (`item.isAvailable`).

```
108 useEffect(() => {
109   const fetchData = async () => {
110     const token = localStorage.getItem("token");
111     if (!token) {
112       console.error("No token found. Redirecting to login...");
113       window.location.href = "/login";
114       return;
115     }
116   }
117   try {
118     const response = await axios.get(
119       "https://localhost:5000/api/MenuItems",
120       {
121         headers: {
122           Authorization: "Bearer " + token,
123         },
124       }
125     );
126     const levestItems = response.data.filter(
127       (item: { category: string; isAvailable: boolean }) =>
128         item.category === "leves" && item.isAvailable === true
129     );
130     setLevestOptions(levestItems.map((item: { name: string }) => item.name));
131   }
132   const eloetelItems = response.data.filter(
133     (item: { category: string; isAvailable: boolean }) =>
134       item.category === "előétel" && item.isAvailable === true
135   );
136   setEloetelOptions(
137     eloetelItems.map((item: { name: string }) => item.name)
138   );
139   const foetelItems = response.data.filter(
140     (item: { category: string; isAvailable: boolean }) =>
141       item.category === "főétel" && item.isAvailable === true
142   );
143   setFoetelOptions(foetelItems.map((item: { name: string }) => item.name));
144 }
```

51. ábra – Ételek lekérdezése `GET` végpont segítségével

A beállítások véglegesítésére egy **KÉSZ** feliratú gomb szolgál, amely elküldi a megadott adatokat. A küldés során a rendszer minden esetben először meghív egy védett `DELETE`

végpontot, amely teljesen kiüríti az ajánlatokat tartalmazó adatbázistáblát, így mindig csak egy, azaz az aktuális napi ajánlat marad bent, végül, ha az sikeresen lefutott, akkor *POST* végponttal rögzíti az új napi ajánlatot, és visszaállítja a menüpontok értékeit alapértelmezettre. Ezen végpontok hitelesítéssel védettek, csak érvényes adminisztrátori jogosultsággal érhetőek el.

```
884 onlick(async () => {  
885   const payload = {  
886     soupName: foodSelections[0],  
887     appetizerName: foodSelections[1],  
888     mainCourseName: foodSelections[2],  
889     hamburgerName: foodSelections[3],  
890     pizzaName: foodSelections[4],  
891     dessertName: foodSelections[5],  
892     drinkName: foodSelections[6],  
893     coffeeName: foodSelections[7],  
894   };  
895  
896   try {  
897     await axios.delete(  
898       "https://localhost:5000/api/DailySpecials",  
899       {  
900         headers: {  
901           "Content-Type": "application/json",  
902           "Authorization": "Bearer ${localStorage.getItem(  
903             "token"  
904           )}",  
905         },  
906       },  
907     );  
908  
909     await axios.post(  
910       "https://localhost:5000/api/DailySpecials",  
911       payload,  
912       {  
913         headers: {  
914           "Content-Type": "application/json",  
915           "Authorization": "Bearer ${localStorage.getItem(  
916             "token"  
917           )}",  
918         },  
919       },  
920     );  
921  
922     setFoodSelections(["", "", "", "", "", "", "", ""]);  
923   }  
924 }
```

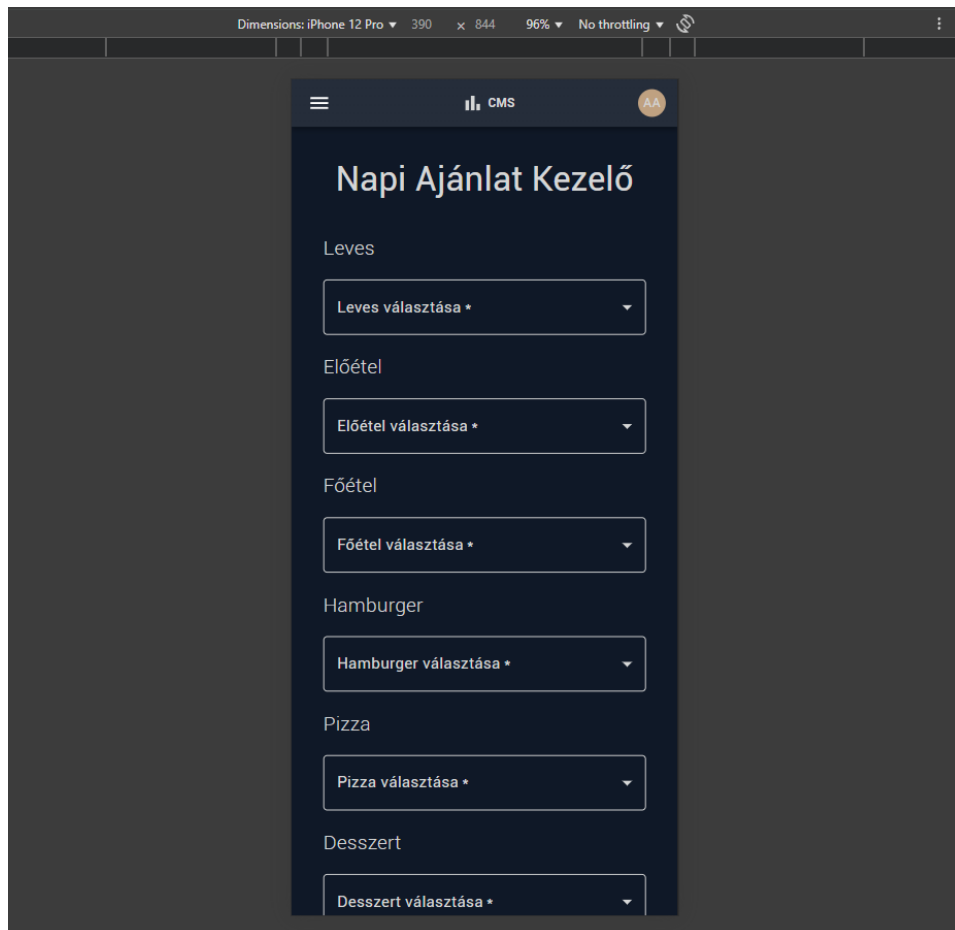
52. ábra – Adatok törlése DELETE végpont segítségével

Sikeres módosítás esetén a sikerességet üzenet jelzi, hogy a módosítás megtörtént, és visszaállítja a listákat az alapértelmezett értékekre.

53. ábra – Napi ajánlat kezelő visszajelző, Sikeres módosítás

A felület modern, sötét témájú megjelenést kapott, amely jól illeszkedik a rendszer többi részéhez. Az egyes mezők tágasak, jól elkülönülnek egymástól. A címek és legördülő listák bal és jobb oldalra rendezve helyezkednek el, ezáltal logikus és átlátható struktúrát alkotnak.

Kis képernyőn vagy mobil eszközön a mezők egymás alá rendeződnek, és a küldés gomb a képernyő alsó részén, középen helyezkedik el a jobb felhasználói élmény érdekében. Ez biztosítja, hogy az űrlap könnyen áttekinthető és használható legyen mobilon is, valamint megkönnyíti a navigációt és az adatok bevitelét.

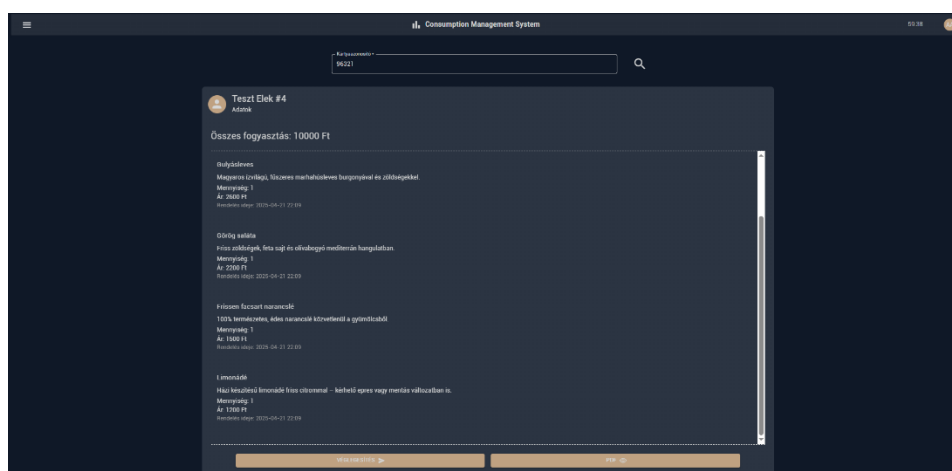


54. ábra – Napi ajánlat kezelő oldal, iPhone 12Pro készüléken

Véglegesítés

A felület kizárólag az adminisztrátorok és jegykezelők számára érhető el. A célja, hogy ellenőrizhető legyen egy adott vendég fogyasztása az étteremben, és az alapján a végösszeg kiszámításra, valamint a fizetés dokumentálásra kerüljön. A felhasználónak be kell vinnie a vendég azonosítóját egy beviteli mezőbe, amely alapján a rendszer lekérdezi az adatbázisból, hogy van-e rögzített fogyasztása.

A felületen a felhasználó azonosítója és profilja jól láthatóan megjelenik pl.: "Teszt Elek #4" formában.

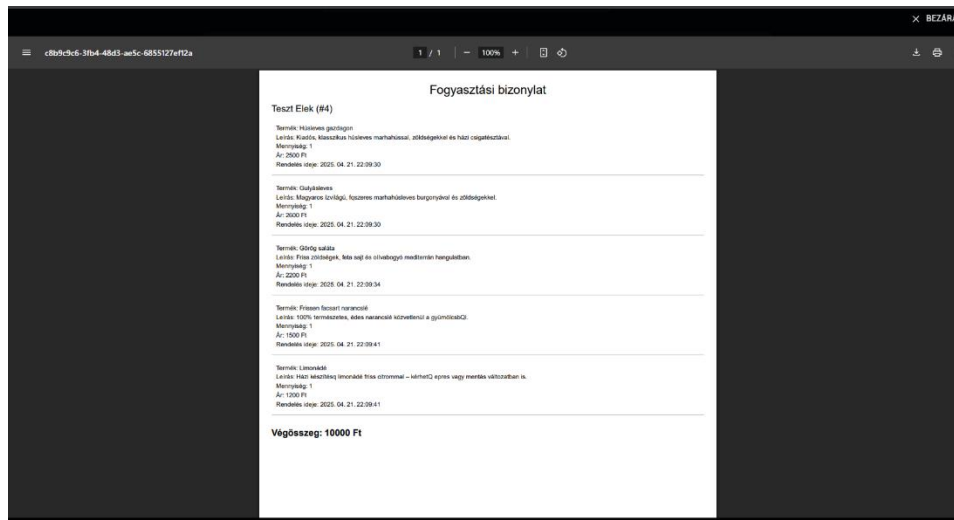


55. ábra – Véglegesítés oldal felülete

A rendszer minden esetben csak érvényes azonosító esetén enged továbblépni. A háttérben minden végpont (fogyasztás lekérdezés, fizetés véglegesítés) hitelesítéssel védett, kizárólag megfelelő jogosultsággal rendelkezők számára érhető el.

A felület alján két gomb található:

- **VÉGLEGESÍTÉS** – amely a fizetési folyamat lezárására szolgál. A gomb megnyomása meghív egy védett *PUT* végpontot, amely deaktiválja az adott kártyát (a háttérben az *isActive* mezőt *false* értékre állítja, és a kártya azonosító értékét törli), így az adatok naplózhatók maradnak, de a kártya nem lesz aktív.
- **PDF** – Lehetőség nyílik a vendég részére számla kiállítására, amely egyrészt lehetővé teszi a fogyasztások papíralapú archiválását, másrészt biztosítja a dokumentum kinyomtatását a vendég részére.



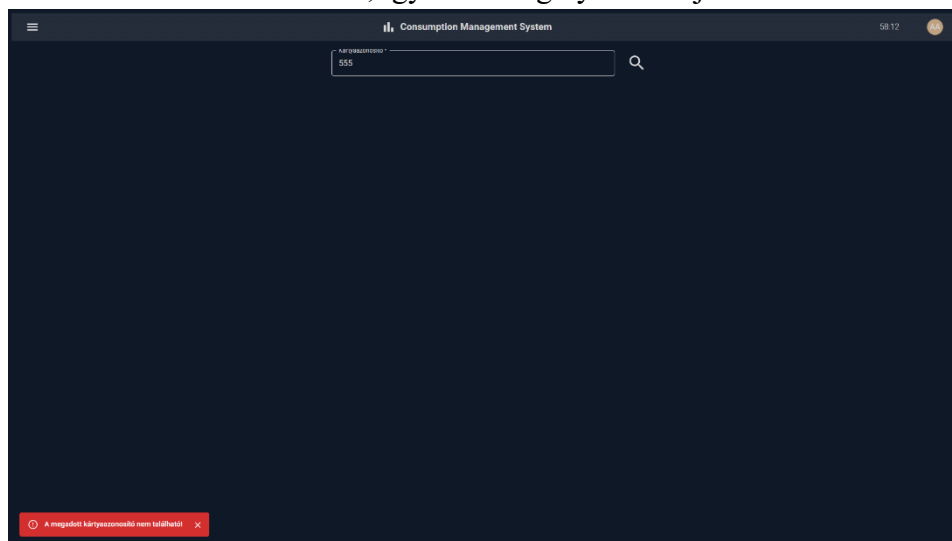
56. ábra – Fogyasztási bizonylat, PDF formátumban

Ennek a funkciónak a működését a *React-PDF* komponens biztosítja, amely egy külön, dedikált service-ben kapott helyet a projektfájlok között. Ez a megoldás lehetővé teszi a dinamikus *PDF*-generálást, megkönnyítve a karbantartást és a további bővítést is.

```
10 const ConsumptionPDF = ({
11   customer,
12   consumption,
13   totalAmount,
14 }) => {
15   const { styles } = useStyles();
16   return (
17     <Document>
18       <Page size="A" style={styles.page}>
19         <div style={styles.container}>
20           <div style={styles.title}>Fogyasztási bizonylat</div>
21           <div style={styles.header}>
22             <div style={styles.customer}>{customer.name}</div>
23             <div style={styles.customerId}>{customer.customerId}</div>
24           </div>
25           <div style={styles.items}>
26             {consumption.map((item, index) => {
27               <div key={index} style={styles.item}>
28                 <div style={styles.itemText}>{item.description}</div>
29                 <div style={styles.itemText}>{item.quantity}</div>
30                 <div style={styles.itemText}>{item.price}</div>
31                 <div style={styles.itemText}>{item.orderDate}</div>
32               </div>
33             ))}
34           <div style={styles.total}>Végösszeg: {totalAmount} Ft</div>
35         </div>
36       </Page>
37     </Document>
38   );
39   export default ConsumptionPDF;
```

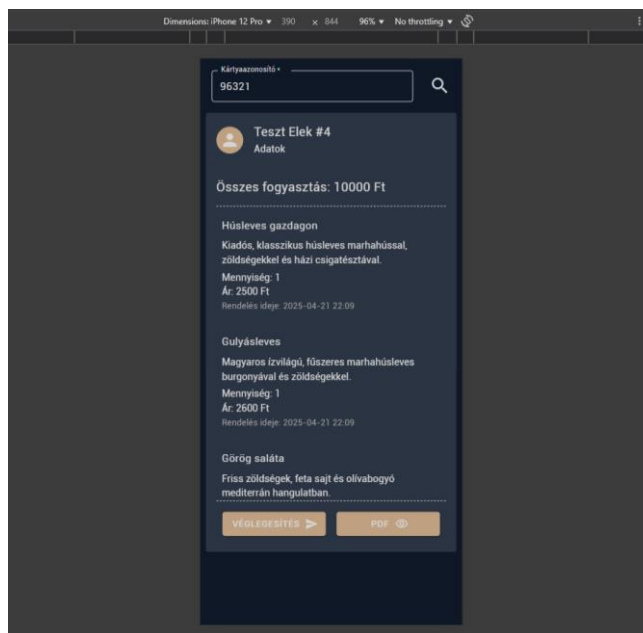
57. ábra – PDF generálás *React-PDF* segítségével

Ha a megadott kártyaazonosító nem található, a rendszer figyelmeztető üzenetet jelenít meg, majd automatikusan frissíti az oldalt, így lehetőség nyílik az újbóli keresésre.



58. ábra – Véglegesítés oldal, Kártya nem létezik

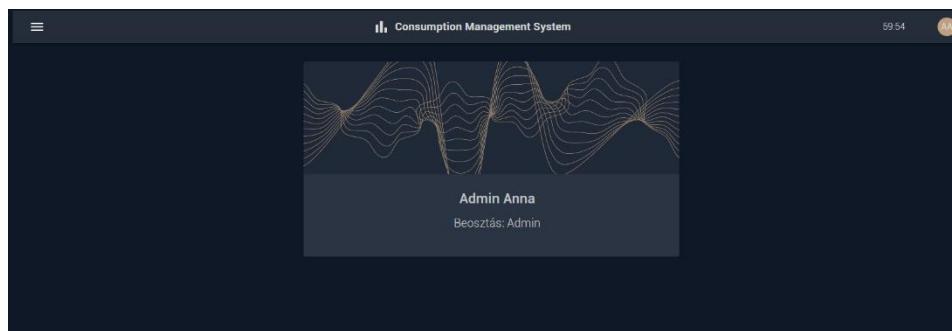
Mobil eszközökön és kisebb kijelzőkön minden információ jól olvasható marad és könnyedén hozzáférhető az érintőképernyő használat során is.



59. ábra – Véglegesítés oldal, iPhone 12Pro készüléken

Profil

A felület minden rendszerbe belépett alkalmazott számára elérhető. A célja, hogy az alkalmazott megtekinthesse saját alapvető adatait és beosztását.



60. ábra – Profil oldal, Felülete

Úgynevezett „token parse-olásával” kerül kinyerésre a bejelentkezett felhasználó adatai. Ez a megoldás biztonságos és gyors hozzáférést biztosít az alapvető felhasználói információkhoz anélkül, hogy újabb végpontot kellene meghívni.

```
useEffect(() => {
  const initialize = async () => {
    const isValidToken = await checkToken();
    if (!isValidToken) {
      window.location.href = "/login";
      return;
    }

    const tokenRefreshed = await refreshToken();
    if (tokenRefreshed) {
      const token = localStorage.getItem("token");
      if (token) {
        const parsedToken = parseJwt(token);
        if (parsedToken) {
          setEmployee({
            firstName: parsedToken.FirstName,
            lastName: parsedToken.LastName,
            username: parsedToken.Username,
            role: parsedToken.role,
          });
        }
      }
    }
    setLoading(false);
  };

  initialize();
}, []);
```

61. ábra – EmployeeProfilPage.tsx Adatok kinyerése tokenből

3.2.3 Frontendhez használt fejlesztői eszközök

Visual Studio Code

A *VSCode* egy ingyenes, könnyű, mégis erőteljes kódszerkesztő, amelyet a *Microsoft* fejlesztett. Beépített támogatást nyújt *JavaScript*, *TypeScript* és *React* fejlesztéshez, valamint számos bővítmény (pl. *ESLint*, *Prettier*) elérhető hozzá a fejlesztési élmény javítása érdekében.

Vite

A *Vite* egy modern fejlesztési szerver és *builder* eszköz, amely gyorsabb fejlesztői élményt nyújt a *React* és *TypeScript* alapú alkalmazások számára. Használata lehetővé teszi a gyorsabb *hot-reloadot* és optimalizált végső kódot.

Material-UI (MUI)

A *Material-UI* egy népszerű *React UI* komponenskönyvtár, amely a *Google Material Design* elveit követi. Segítségével modern, reszponzív és könnyen testreszabható felhasználói felületeket lehet fejleszteni.

Axios

Az *Axios* egy ígéreteken (*Promises*) alapuló *HTTP* kliens, amelyet adatok lekérésére és küldésére használnak *API*-khoz. Különösen népszerű a *React* alkalmazásokban, mivel egyszerűsíti az aszinkron hálózati kéréseket és kezelést.

React & React-DOM

A *React* egy népszerű *JavaScript* könyvtár interaktív felhasználói felületek fejlesztésére. A *React-DOM* a *React* komponensek *DOM*-ba való rendereléséért felelős, biztosítva a hatékony frissítéseket és a komponens-alapú fejlesztési modellt.

TypeScript

A *TypeScript* egy *JavaScript* alapú programozási nyelv, amely statikus típusellenőrzést biztosít. A *React* alkalmazásokban használva segít a hibák csökkentésében és a kód karbantarthatóságának javításában.

ESLint

Az *ESLint* egy *JavaScript* és *TypeScript* kódellenőrző eszköz, amely segít fenntartani a kódminőséget, betartani a kódolási szabványokat és azonosítani a lehetséges hibákat már fejlesztés közben.

Prettier

A *Prettier* egy automatikus kódformázó eszköz *JavaScript*, *TypeScript*, *HTML*, *CSS* és sok más nyelv számára. Segítségével egységes kódstílust tarthatunk fenn a projektben anélkül, hogy manuálisan kellene figyelni a formázásra.

Tailwind CSS

A *Tailwind CSS* egy *utility-first* szemléletű *CSS* keretrendszer, amely előre definiált, kis méretű és jól elkülöníthető osztályokat biztosít a felhasználó számára. Ezek az osztályok lehetővé teszik a felhasználói felület gyors és hatékony, ugyanakkor magas szinten testreszabható kialakítását.

Figma

A *Figma* egy felhőalapú tervezőeszköz, amely lehetővé teszi felhasználói felületek, *wireframe*-ek és prototípusok létrehozását és megosztását. Valós idejű együttműködési funkciói révén ideális csapatmunkára. Intuitív felületének és beépített komponenseinek köszönhetően hatékonyan támogatja a *UX/UI* tervezést.



62. ábra – Wireframe-ek tervezéshez

3.2.4 Algoritmusok

Kategória Kiválasztás Algoritmus

- Cél: A felhasználó által kiválasztott ételkategória beállítása.
- Működés:
 - A *handleCategoryClick* függvény kap egy kategória nevet (*string* | *null*), és frissíti a *selectedCategory* állapotot.
 - Ezáltal a felhasználó által kiválasztott kategóriához tartozó ételek jelennek meg a listában.

```

89 const handleCategoryClick = (category: string | null) => {
90   setSelectedCategory(category);
91   if (category === "Napi ajánlat") {
92     axios
93       .get("https://localhost:5000/api/DailySpecials", {
94         headers: {
95           "Content-Type": "application/json",
96           "Authorization": "Bearer ${localStorage.getItem("token")}",
97         },
98       })
99       .then((response) => {
100         const specials = response.data;
101         const filteredSpecials = menuItems.filter((item) =>
102           [
103             specials[0].soupName,
104             specials[0].appetizerName,
105             specials[0].mainCourseName,
106             specials[0].hamburgerName,
107             specials[0].pizzaName,
108             specials[0].dessertName,
109             specials[0].drinkName,
110             specials[0].coffeeName,
111             ].includes(item.name)
112         );
113         setDailySpecials(filteredSpecials);
114       })
115       .catch((error) => {
116         console.error(
117           "Hiba történt a napi ajánlatok betöltése közben.",
118           error
119         );
120         setDailySpecials([]);
121       });
122   }
123 }
124

```

63. ábra – OrderPage.tsx Napi ajánlat lekérése GET végponttal

Kosárhoz Adás Algoritmus

- Cél: Az adott étel mennyiségének növelése a rendelésben.
- Működés:
 - A *handleAddToOrder* függvény kap egy *MenuItem* objektumot.
 - Ellenőrzi, hogy az adott tétel már szerepel-e az *orders* listában.
 - Ha igen, növeli a *quantity* értéket.
 - Ha nem, hozzáadja az új tételt *quantity: 1* értékkel.

```

126 const handleAddToOrder = (item: MenuItem) => {
127   setOrders((prevOrders) => {
128     const existingOrder = prevOrders.find(
129       (orderItem) => orderItem.itemId === item.itemId
130     );
131     if (existingOrder) {
132       return prevOrders.map((orderItem) =>
133         orderItem.itemId === item.itemId
134           ? { ...orderItem, quantity: (orderItem.quantity || 0) + 1 }
135           : orderItem
136       );
137     } else {
138       return [...prevOrders, { ...item, quantity: 1 }];
139     }
140   });
141 }

```

64. ábra – OrderPage.tsx Elem hozzáadási algoritmus

Kosárból Tétel Eltávolítása Algoritmus

- Cél: Egy adott tétel mennyiségének csökkentése a kosárban, vagy teljes eltávolítása, ha a mennyiség 0 lesz.
- Működés:
 - A *handleRemoveFromOrder* függvény végigiterál az *orders* listán.
 - Ha az adott elem *itemId*-ja megegyezik a csökkentendő tétellel, csökkenti a *quantity* értékét.
 - Ha az új mennyiség 0 lesz, az adott tételt eltávolítja a listából.

```
143 const handleRemoveFromOrder = (item: MenuItem) => {  
144   setOrders(prevOrders) => {  
145     return prevOrders  
146       .map((orderItem) => {  
147         orderItem.itemId === item.itemId  
148           ? { ...orderItem, quantity: (orderItem.quantity || 0) - 1 }  
149           : orderItem  
150       })  
151       .filter((orderItem) => orderItem.quantity > 0);  
152   };  
153 }
```

65. ábra – OrderPage.tsx Elem mennyiség csökkentő algoritmus

Kosárból Tétel Teljes Törlése Algoritmus

- Cél: Egy adott étel teljes eltávolítása a rendelésből.
- Működés:
 - A *handleDeleteFromOrder* függvény kiszűri az adott *itemId*-val rendelkező tételt az *orders* listából.

```
155 const handleDeleteFromOrder = (item: MenuItem) => {  
156   setOrders(prevOrders) => {  
157     prevOrders.filter((orderItem) => orderItem.itemId !== item.itemId)  
158   };  
159 }
```

66. ábra – OrderPage.tsx Elem törlő algoritmus

Teljes Ár Számítási Algoritmus

- Cél: A rendelés teljes árának kiszámítása.
- Működés:
 - A *calculateTotalPrice* függvény végigiterál az *orders* listán.
 - Minden rendelési tétel *price* értékét megszorozza a *quantity* értékével.
 - Az összegeket összeadja, majd visszatér az eredménnyel.

```
161 const calculateTotalPrice = () => {  
162   return orders.reduce(  
163     (total, orderItem) => total + orderItem.price * orderItem.quantity,  
164     0  
165   );  
166 };  
167
```

67. ábra – OrderPage.tsx Végösszeg kiszámító algoritmus

Rendelés Leadási Algoritmus

- Cél: Az összegyűjtött rendelés adatainak elküldése a backend felé.
- Működés:
 - A *handleSubmitOrder* függvény kap egy *cId* értéket, amely a vendég azonosítója.
 - Először az `axios.get (" /api/Cards/GetCustomerIdByCardId/ " + cId)` segítségével lekérdezi a felhasználó azonosítóját.
 - Ha érvényes azonosítót kap, az `axios.post("/api/Orders")` hívással elküldi a rendelést a szervernek.
 - Ha sikeres a rendelés, elmenti a rendelési azonosítót (*orderId*) és megnyitja a visszaigazoló párbeszédablakot (*setDialogOpen(true)*).
 - Ha hiba történik, naplózza a hibát.

```
253 const handleSubmitOrder = () => {  
254   if (!cardId || orders.length === 0) {  
255     return;  
256   }  
257
```

68. ábra – OrderPage.tsx Rendelés leadó algoritmus

Jogosultság kezelés JWT token segítségével

A jogosultságkezelés központi eleme a *JWT token* alapú azonosítás, amely lehetővé teszi a felhasználók biztonságos hitelesítését és jogosultsági szintjük egyértelmű meghatározását.

```
1 function parseJwt(token: string): any {  
2   try {  
3     const base64Url = token.split(".")[1];  
4     const base64 = base64Url.replace(/-/g, "+").replace(/_/g, "/");  
5     const jsonPayload = decodeURIComponent(  
6       atob(base64)  
7         .split("")  
8         .map((c) => "%" + ("00" + c.charCodeAt(0).toString(16)).slice(-2))  
9         .join("")  
10    );  
11  
12    return JSON.parse(jsonPayload);  
13  } catch (error) {  
14    console.error("Failed to parse JWT:", error);  
15    return false;  
16  }  
17 }  
18  
19 export { parseJwt };  
20
```

69. ábra – Jogosultság kezelés JWT token segítségével

- Cél:
 - A felhasználók biztonságos azonosítása és jogosultsági szintjük kezelése.
 - A folyamatos bejelentkezési állapot és az érvényes jogosultságok biztosítása.
 - A felhasználói élmény és adatbiztonság egyensúlyának fenntartása.
- Működés:
 - A *backend* által generált úgynevezett „*access token*”-t a böngésző *localStorage*-ében tároljuk, amely legfeljebb egy órán keresztül érvényes.
 - A *refresh token* minden új oldalra történő navigáláskor automatikusan frissül, így a felhasználó aktív marad.
 - A felületen egy időzítő jelzi, mennyi ideig érvényes még az aktuális *access token*.
 - A *JWT token* tartalmát egy „*parser*” segítségével *JSON* formátumba alakítjuk, innen nyerjük ki a felhasználó monogramját és jogosultsági szintjét.
 - Minden oldalfrissítéskor a backend is ellenőrzi a *token* érvényességét, így mindig naprakész és valós jogosultsági adatokkal dolgozunk.

3.2.5 Teljesítményoptimalizálás

Állapotkezelés (State Management)

Az alkalmazás belső állapotkezelése a *React* könyvtárban elérhető *Hook*-ok alkalmazásával történik.

- *useState* hook: az összes dinamikus adatállapotot kezeli, például:
 - *menuItems*: a menüben lévő ételek listája.
 - *orders*: az aktuális rendelésben lévő tételek listája.
 - *selectedCategory*: a kiválasztott kategória.
 - *loading* és *error*: az adatbetöltés állapotát követik.
- *useEffect* hook: az adatbetöltést végzi el az *API* hívás során.

API Kommunikáció

Az alkalmazás *Axios* könyvtárat használ a backend *API*-val történő adatkommunikációhoz. Többek között az alábbi végpontokat hívja meg:

- *GET /api/MenuItems* – az étlap adatainak lekérése.
- *GET /api/Cards/GetCustomerIdByCardId/{cId}* – egy adott kártya ID alapján a vendég azonosítójának lekérdezése.
- *POST /api/Orders* – a rendelés leadása, amely tartalmazza a felhasználó azonosítóját, az alkalmazott ID-ját és a megrendelt ételeket.

Hibakezelés

- Minden *API* hívás tartalmaz *catch* ágat, amely hibás válasz esetén naplózza az esetleges problémát.
- Az *error* állapot frissítésével a hibák *UI* szinten is megjeleníthetők (pl. hibaüzenetek formájában).

UI Interakciók és Felhasználói Élmény

- **Betöltés közbeni állapot**
 - Az oldal megjelenít egy *CircularProgress* komponenst (*size*={120}, *sx*={{ color: "#bfa181" }}), amíg az adatok nem töltődtek be.
 - Hiba esetén egy *Alert* komponens jelenik meg.

- **Dinamizmus az ikonoknál**

- Az egyes ételkategóriákhoz tartozó ikonokat egy objektumtérkép (*categoryIcons*) tárolja, amely egyértelműen kapcsolja össze a kategóriákat a megfelelő *Material UI* ikonokkal.

- **Minimalizált újrenderelés**

- Az állapotok (*useState*) és az effektusok (*useEffect*) használata biztosítja, hogy az újrenderelések csak akkor történjenek meg, ha ténylegesen szükségesek.
- A *menuItems* listát csak egyszer tölti be az alkalmazás, és az csak akkor frissül, ha új *API* hívás történik.

- **Kosárkezelés hatékonysága**

- Az *orders* listában történő keresés (*find*, *map*, *filter*) $O(n)$ komplexitású művelet, de mivel a lista mérete jellemzően kicsi (néhány tétel), ez nem okoz teljesítményproblémát.

3.2.6 Tesztelés

A *Jest* egy *JavaScript*-alapú tesztelési keretrendszer, amelyet a *Facebook* fejlesztett. Elsősorban a *React* alkalmazások tesztelésére használják, de más *JavaScript* keretrendszerek és könyvtárak tesztelésére is alkalmas. A *Jest* népszerű a gyorsasága, rugalmassága és egyszerű használata miatt.

Főbb jellemzők:

1. **Egyszerű használat:**

- Könnyen telepíthető és integrálható a *JavaScript*-projektekbe.
- Nem szükséges külön konfiguráció az alapvető használatához.

2. **Gyors végrehajtás:**

- Párhuzamosan futtatja a teszteket, így gyors eredményt biztosít.

3. **Mocking támogatás:**

- Lehetővé teszi függvények, modulok és *API*-k mockolását, így izolált teszteket végezhetünk.

4. Snapshot tesztelés:

- Egyedi funkciója, amely megkönnyíti a felhasználói felület változásainak követését.

5. Kódlefedettség ellenőrzése:

- Jelentést készít arról, hogy a kód mely részeit fedik le a tesztek.

6. Környezetfüggetlen:

- Böngészőfüggetlen környezetet biztosít a tesztek futtatásához.

Teszt típusok Jestben:**1. Egységteszt (Unit Test):**

- A kód kis egységeit (pl. funkciókat, osztályokat) teszteli.

2. Integrációs teszt (Integration Test):

- Ellenőrzi, hogy a különböző kódmodulok együttműködnek-e.

3. Snapshot teszt:

- A React komponensek megjelenését ellenőrzi. A snapshotok segítségével összevethetjük a komponens aktuális állapotát az előző állapottal.

Jest működési elve

A Jest általában három kulcsfogalmat használ

1. Test Suite:

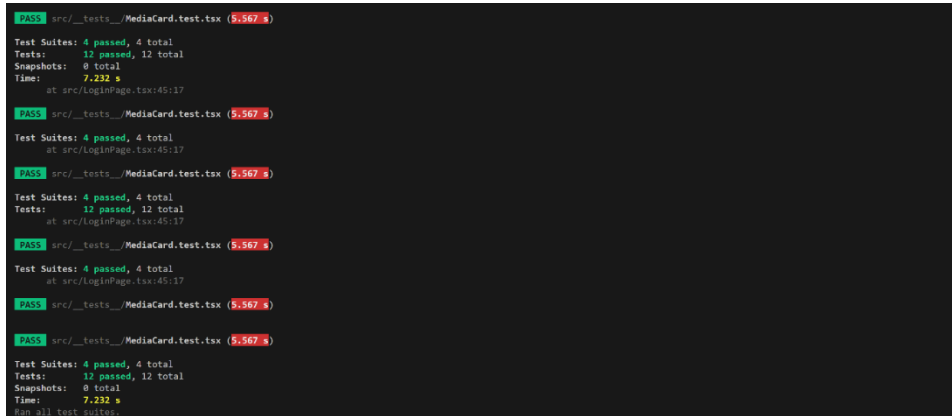
- Egy tesztescsoport, amely egy adott funkció vagy modul tesztelését foglalja magában.
- Példa: *describe()* metódus.

2. Teszt:

- Egy egyedi teszteset, amely meghatározza, hogy egy adott funkció mit vár el.
- Példa: *test()* vagy *it()* metódus.

3. Assertion:

- Kijelentések, amelyek ellenőrzik, hogy egy teszt megfelel-e az elvárt eredménynek.
- Példa: `expect()` metódus.



70. ábra – Tesztelés Jest keretrendszer segítségével

A Jest tesztelési keretrendszer beállítása és használata

A projektkörnyezet kialakításához a következő fejlesztői függőségek telepítése szükséges: „`npm install --save-dev jest @testing-library/react @testing-library/jest-dom @types/jest ts-jest`”

- `jest`: A tesztmotor, amely futtatja a teszteseteket.
- `@testing-library/react`: A *React* komponensek tesztelését segítő eszközkészlet.
- `@testing-library/jest-dom`: *DOM*-specifikus *matcherek*, amelyek például a `toBeInTheDocument()` függvényt biztosítják.
- `@types/jest`: Típusdefiníciók *TypeScript*-alapú projektekhez.
- `ts-jest`: Fordítómotor, amely lehetővé teszi a *TypeScript* fájlok használatát *Jest* környezetben.

Konfiguráció

A *Jest* tesztelési környezete konfigurálható *CommonJS* (`jest.config.js`) vagy *ECMAScript* modul (*ESM*) formátumú (`jest.config.mjs`) fájl segítségével. A projektben az *ESM* formátum került alkalmazásra, ennek megfelelően a konfigurációs beállítások a `jest.config.mjs` fájlban találhatók. *ESM* formátum használata esetén az alábbi követelmények érvényesek: A konfigurációs fájlban az `export default` szintaxis kötelező.

A projekt `package.json` fájljában a `type` mező értékét `module`-ra kell állítani, vagy alternatív megoldásként `.mjs` kiterjesztésű fájl használata javasolt. A *Jest* automatikusan felismeri a

következő elnevezési konvenciókat: Minden fájl, amely *.test.ts*, *.test.tsx*, *.spec.ts*, *.spec.tsx* néven végződik. A tesztfájlok elhelyezhetők dedikált `__tests__` mappában vagy a komponenssel azonos mappában.

```
2 export default {
3   testEnvironment: 'jest-environment-jsdom',
4   transform: {
5     '^.+\\.tsx?$': 'ts-jest',
6   },
7   moduleFileExtensions: ['ts', 'tsx', 'js', 'jsx'],
8 }
```

71. ábra – *jest.config.mjs*, default export

Egy teszt leírása példával szemléltetve

1. Komponens sikeres renderelése

A cél annak ellenőrzése, hogy a *MediaCard* komponens helyesen jelenik-e meg, és tartalmazza a várt feliratokat:

```
23 test("renders the MediaCard component", () => {
24   render(<MediaCard />);
25   expect(
26     screen.getByText("Consumption Management System")
27   ).toBeInTheDocument();
28   expect(screen.getByText("Üdvözlök!")).toBeInTheDocument();
29   expect(screen.getByText("Kérjük jelentkezze be!")).toBeInTheDocument();
30 });
```

72. ábra – *Jest teszt, MediaCard komponens helyes megjelenése*

2. Hibakezelés: Sikertelen bejelentkezés

Ez a teszt azt ellenőrzi, hogy ha a bejelentkezés sikertelen (a szerver 401-es hibát ad vissza), akkor megjelenik-e a megfelelő hibaüzenet a felhasználónak. A *mockedAxios.post* segítségével szimuláljuk a hibás válaszkódot, majd a felhasználói interakciók után ellenőrizzük, hogy megjelenik-e a hibaüzenet az oldalon.

```
32 test("shows error on failed login", async () => {
33   mockedAxios.post.mockRejectedValueOnce({
34     response: { status: 401 },
35   });
36
37   render(<MediaCard />);
38   const usernameInput = screen.getByLabelText(/felhasználónev/i);
39   const passwordInput = screen.getByLabelText(/jelszo/i);
40   const submitButton = screen.getByRole("button", { name: /belepés/i });
41
42   fireEvent.change(usernameInput, { target: { value: "wronguser" } });
43   fireEvent.change(passwordInput, { target: { value: "wrongpassword" } });
44   fireEvent.click(submitButton);
45
46   await waitFor(() => {
47     expect(
48       screen.getByText("Hibás felhasználónev vagy jelszo.")
49     ).toBeInTheDocument();
50   });
51 });
```

73. ábra – *Jest teszt, Hibakezelés sikertelen bejelentkezés esetén*

3. Sikeres bejelentkezés kezelése

A sikeres bejelentkezés szimulálása során ellenőrizzük, hogy az *axios.post* helyesen lett meghívva a megadott adatokkal, és hogy a *localStorage* megfelelő értékeket tartalmaz a válasz alapján:

```
53 test('submit the form successfully', async () => {
54   mockedAxios.post.mockResolvedValueOnce({
55     data: { token: 'mockToken', refreshToken: 'mockRefreshToken' },
56   });
57
58   render(<MediaCard />);
59   const usernameInput = screen.getByLabelText(/felhasználónev/i);
60   const passwordInput = screen.getByLabelText(/jelszó/i);
61   const submitButton = screen.getByRole('button', { name: /bejelentkezés/i });
62
63   fireEvent.change(usernameInput, { target: { value: 'testuser' } });
64   fireEvent.change(passwordInput, { target: { value: 'password123' } });
65   fireEvent.click(submitButton);
66
67   await waitFor(() => {
68     expect(mockedAxios.post).toHaveBeenCalledWith(
69       'https://localhost:5000/api/Auth/login',
70       {
71         username: 'testuser',
72         password: 'password123',
73       },
74     );
75   });
76
77   expect(localStorage.getItem('token')).toBe('mockToken');
78   expect(localStorage.getItem('refreshToken')).toBe('mockRefreshToken');
79 });
80 });
```

74. ábra – Jest teszt, Sikeres bejelentkezés kezelése

A teljes projekten végig futatott teszt eredménye:

A projekt során automatizált tesztelés került végrehajtásra a rendszer megbízhatóságának ellenőrzése céljából. A tesztek célja a funkciók helyes működésének validálása és az esetleges hibák korai kiszűrése volt. Az összes teszt sikeresen lefutott, ami a rendszer stabilitását támasztja alá.

```
console.log
{ data: { token: 'mockToken', refreshToken: 'mockRefreshToken' } }

console.log
{ data: { token: 'mockToken', refreshToken: 'mockRefreshToken' } }

console.log
{ data: { token: 'mockToken', refreshToken: 'mockRefreshToken' } }
at src/LoginPage.tsx:45:17

console.log
{ data: { token: 'mockToken', refreshToken: 'mockRefreshToken' } }
at src/LoginPage.tsx:45:17

console.log
{ data: { token: 'mockToken', refreshToken: 'mockRefreshToken' } }
at src/LoginPage.tsx:45:17

at src/LoginPage.tsx:45:17

at src/LoginPage.tsx:45:17

Test Suites: 4 passed, 4 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 8.887 s
Test Suites: 4 passed, 4 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Test Suites: 4 passed, 4 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Test Suites: 4 passed, 4 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 8.887 s
Done all test suites.
```

75. ábra – A Jest teszt sikeres

3.3 Backend

3.3.1 Specifikáció

A projekt kiszolgálója egy *ASP.NET 8* alapokra épülő backend alkalmazás, amely megbízhatóan és biztonságosan szolgáltatja a fogyasztásnyilvántartó rendszer adatait a frontend számára. Az alkalmazás biztosítja a különböző fogyasztási adatok kezelését, elérését és naprakészen tartását, valamint támogatja a felhasználói jogosultságkezelést és a hitelesítési folyamatokat is. A rendszer célja, hogy hatékony adatforgalmat és integrációs lehetőségeket kínáljon a felhasználói felület és a háttérrendszer között, miközben megfelel a korszerű biztonsági elvárásoknak.

Technológiák

- *RESTful API*, hitelesítési folyamatok *JWT (JSON Web Token)* használata.
- *Entity Framework Core* az adatkezeléshez és *ORM*-hez
- *SQL* alapú relációs adatbázisok
- *xUnit* tesztek

Funkcionalitások

- A rendszer lehetőséget biztosít alapvető *CRUD (Create, Read, Update, Delete)* műveletek végrehajtására, melyek elérhetősége és használata a felhasználó jogosultsági szintjétől függ.
- A felhasználók csak a számukra engedélyezett műveleteket hajthatják végre, így garantált a biztonságos és szabályozott adatkezelés.
- A jogosultsági szintek dinamikusan szabályozzák, hogy ki hozhat létre, módosíthat, törölhet vagy tekinthet meg adatokat a rendszerben.

Biztonsági szempontok

- Az érzékeny adatok, például a jelszavak, kizárólag *hashed* formában kerülnek eltárolásra az adatbázisban, így megakadályozva azok visszafejtését és illetéktelen hozzáférését.
- Az adatok átviteléhez és megjelenítéséhez *DTO-k (Data Transfer Object)* használata biztosítja, hogy kizárólag a szükséges és megfelelő információk legyenek elérhetőek a felhasználók számára, megelőzve az érzékeny adatok kiszivárgását.

- A rendszer minden adatforgalma *HTTPS* protokollon keresztül történik, amely garantálja a kommunikáció titkosságát és védelmét az illetéktelen lehallgatásokkal szemben
- Az *API* végpontokhoz való hozzáférés kizárólag hitelesített és jogosult felhasználók számára engedélyezett.

Teljesítmény-igények

- A rendszer biztosítja a gyors válaszidőt, így a felhasználók zökkenőmentesen és késedelem nélkül érhetik el az adatokat és funkciókat.
- Képes több felhasználó egyidejű, párhuzamos kiszolgálására, így nagyobb terhelés esetén is megbízható és stabil működést garantál.
- *CORS (Cross-Origin Resource Sharing)* szabályok szigorú beállítása annak érdekében, hogy csak megbízható források érhessék el az *API*-t.

3.3.2 Az alkalmazott fejlesztői eszközök

Visual Studio 2022

A Visual Studio 2022 egy széles körben használt fejlesztőkörnyezet, amely támogatja az *ASP.NET Web API* fejlesztését *C#* nyelven. Beépített *NuGet* csomagkezelője lehetővé teszi a szükséges csomagok, például az *Entity Framework Core*, a *Swashbuckle (Swagger)* és egyéb *API*-fejlesztést támogató könyvtárak egyszerű hozzáadását és frissítését.

Általunk alkalmazott NuGet csomagok (Dependency Management)

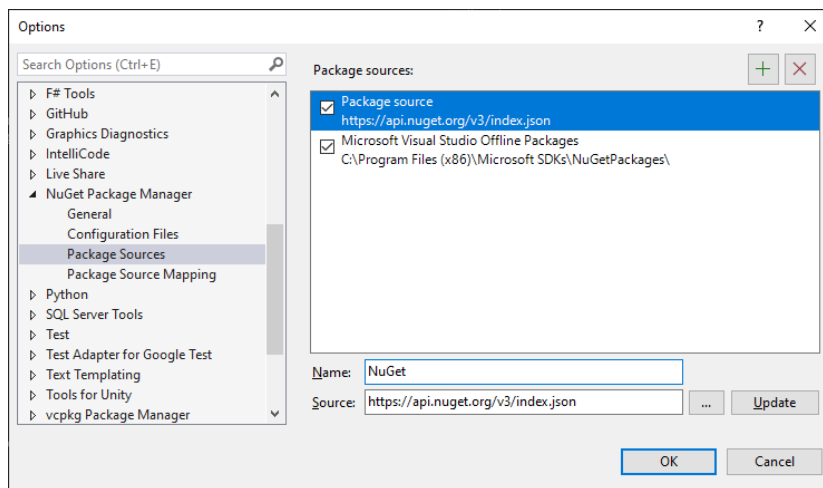
A *NuGet* csomagokat *NuGet Package Manager* segítségével telepítettük és használtuk a programunkban. Alapértelmezetten a *Visual Studio* nem mindig tartalmazza a szükséges *NuGet* csomagforrást, ezért ezt manuálisan kell beállítani.

A beállítás elérhető a következő útvonalon:

Visual Studio → *Tools* → *Options* → *NuGet Package Manager* → *Package Sources*

Itt adjunk hozzá egy új csomagforrást a következő értékekkel:

- **Name:** *NuGet*
- **Source:** *https://api.nuget.org/v3/index.json*



76. ábra – Nuget csomag hozzáadása Visual Studiohoz

- **BCrypt.Net-Next (by Chris McKee, Ryan D. Emerl, Damien Miller) – 4.0.3:**
 - Egy *.NET*-ben használható könyvtár, amely a *BCrypt* hash-elési algoritmus implementációját biztosítja. Lehetővé teszi a *jelszóhash*-ek generálását és ellenőrzését, erős védelmet nyújtva a *bruteforce* támadások ellen.
- **Microsoft.AspNetCore.Authentication.JwtBearer (by Microsoft) – 8.0.13:**
 - Az *Entity Framework Core* egy modern objektum-relációs adatbázis leképző *.NET* keretrendszerhez, ami támogatja a *LINQ* lekérdezéseket, módosításokat, sémamigrációkat.
- **Microsoft.EntityFrameworkCore (by Microsoft) – 9.0.2:**
 - Ez a *middleware* lehetővé teszi a *JWT* tokenek érvényesítését *HTTP*-kérések során, biztosítva a felhasználók hozzáféréseinek biztonságos kezelését.
- **Microsoft.EntityFrameworkCore.Design (by Microsoft) – 9.0.2**
 - Az *Entity Framework Core* eszközeihez szükséges fejlesztési időben használt komponensek.
- **Microsoft.EntityFrameworkCore.Sqlite (by Microsoft) – 9.0.2**
 - *SQLite* adatbázis kezelő *Entity Framework Core*-hoz.
- **Microsoft.EntityFrameworkCore.Tools (by Microsoft) – 9.0.2**
 - Az *Entity Framework Core* eszközeihez szükséges *NuGet* csomag.

- **Microsoft.VisualStudio.Web.CodeGeneration.Design (by Microsoft) – 8.0.7:**
 - Kódgenerálás az ASP.NET Core-hoz, tartalmazza a *dotnet-aspnet-codegenerator* parancsot vezérlők és nézetek generálásához.
- **Swashbuckle.AspNetCore (by domaindrivendev) – 6.6.2**
 - Kizárólag fejlesztői környezethez használt *Swagger* eszköz, mely képes a végpontok tesztelésére.

Postman

A *Postman* egy népszerű *API*-fejlesztő eszköz, amely lehetővé teszi *HTTP*-kérések küldését, *API*-k tesztelését és dokumentálását, támogatva a projektben is használt *JWT* hitelesítési módszert.

Swagger UI

A *Swagger UI* egy könnyen használható eszköz, amely lehetővé teszi a *RESTful* *API*-k dokumentációját. A fejlesztők megtekinthetik, tesztelhetik és kipróbálhatják az *API* végpontokat közvetlenül egy böngészőből, mely fejlesztési módban a backend szerver indításával automatikusan indul.

Package Management Console

A *Package Management Console* a *Visual Studio* eszköze, amellyel *NuGet* csomagokat kezelhetünk és migrációkat futtathatunk, például *Add-Migration* és *Update-Database* parancsokkal.

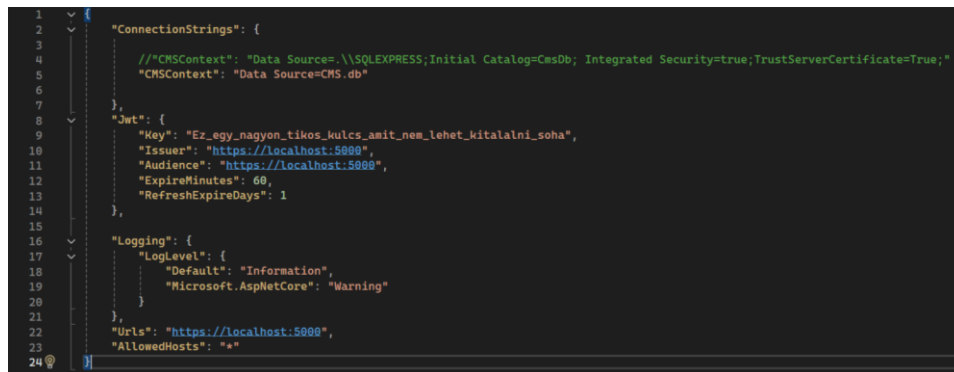
3.3.3 Adatmodell leírása

Az *Entity Framework (EF)* egy *ORM (Object-Relational Mapping)* keretrendszer, amely lehetővé teszi az adatbázis és az alkalmazás objektumai közötti leképezést. Az *EF* segítségével a fejlesztők közvetlenül objektumokkal dolgozhatnak *SQL* lekérdezések helyett, ami csökkenti a hibalehetőségeket és növeli a fejlesztési hatékonyságot.

Az *EF* alapelvei közé tartoznak:

- **Code First:** Az adatbázis az osztályokból generálódik.
- **Database First:** Már meglévő adatbázisból generálódnak az osztályok.
- **Model First:** Modellező eszköz segítségével hozható létre az adatbázis.

Az *EF* támogatja az automatikus migrációkat, így az adatbázis változásai könnyen kezelhetők. A backend legfontosabb beállításait, mint például a *ConnectionString* és a *JWT* konfiguráció, az *appsettings.json* fájlban tároljuk.



```
1  {
2    "ConnectionStrings": {
3      //CMSContext: "Data Source=\\SQLSERVER\\Initial Catalog=CmsDb; Integrated Security=true;TrustServerCertificate=True;"
4      "CMSContext": "Data Source=CMS.db"
5    },
6    "Jwt": {
7      "Key": "Ez egy nagyon titkos kulcs amit nem lehet kitalálni soha",
8      "Issuer": "https://localhost:5000",
9      "Audience": "https://localhost:5000",
10     "ExpireMinutes": 60,
11     "RefreshExpireDays": 1
12   },
13   "Logging": {
14     "LogLevel": {
15       "Default": "Information",
16       "Microsoft.AspNetCore": "Warning"
17     }
18   },
19   "Urls": "https://localhost:5000",
20   "AllowedHosts": "*"
21 }
```

77. ábra – *appsettings.json*, *ConnectionString* és *JWT* konfiguráció

A modellek kialakításánál a következő annotációkat használtuk:

- **[Key]:** Az attribútumot az entitás azon tulajdonságának meghatározására használjuk, amely az elsődleges kulcsot képviseli az adatbázisban.
- **[Required]:** Arra használjuk, hogy megadjuk, hogy egy adott tulajdonság kötelező (nem lehet null). Az adatbázis-kezelő ennek megfelelően nem engedélyezi a *NULL* értékek tárolását.
- **[InverseProperty("táblanév")]:** A kapcsolódó entitások közötti viszonyok meghatározására használjuk. Az adott entitás kapcsolatát az összefüggő másik entitással jelöli. Ebben az esetben a táblanév a másik entitás tulajdonsága.

3.3.4 Táblák és modelljeik:

A backend struktúrájában a modellek elnevezése angol nyelven, egyes számban történik (pl. Ticket), mivel ezek egyetlen entitást reprezentálnak. Az adatbázisban a *DbSet* típusú tulajdonságok már többes számban jelennek meg (pl. Tickets), jelezve, hogy ezek a táblákban tárolt rekordok gyűjteményei. Ez a megnevezési konvenció elősegíti a kód strukturáltságát és a logikai felépítés egyértelműségét.

Admission / Admissions

Ebben a táblában a különböző kiegészítő jegytípusok találhatóak, melyek az alapjegyhez vásárolhatók. Minden jegyhez tartozik egy egyedi azonosító, név, kategória, leírás, kép elérési útja, elérhetőség és ár. A tábla felel a kiegészítő jegyek kezeléséért.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3  using System.Text.Json.Serialization;
4
5  namespace CMS.Models
6  {
7      7 references
8      public class Admission
9      {
10         [Key]
11         3 references
12         public int AdmissionId { get; set; }
13         [Required]
14         0 references
15         public string? AdmissionName { get; set; }
16
17         [Required]
18         0 references
19         public string Category { get; set; }
20
21         [Required]
22         0 references
23         public int Price { get; set; }
24
25         [Required]
26         0 references
27         public string Description { get; set; }
28
29         [Required]
30         0 references
31         public bool IsAvailable { get; set; }
32
33         [Required]
34         0 references
35         public string ImagePath { get; set; }
36         [JsonIgnore]
37         [InverseProperty("Admissions")]
38         0 references
39         public List<CustomerAdmission>? CustomerAdmissions { get; set; }
40     }
41 }

```

78. ábra – Admission.cs, Kiegészítő jegy tábla felépítése

Admissions			
	Típus	Mezőnév	Megjegyzés
	INTEGER	AdmissionId	Auto increment
	TEXT	AdmissionName	A kiegészítő jegy típusa
	TEXT	Category	A kiegészítő jegy kategóriája
	TEXT	Description	A kiegészítő jegy rövid megnevezése
	TEXT	ImagePath	A kiegészítő jegy képének relatív útvo- nala
	INTEGER	IsAvailable	Elérhető-e az adott kiegészítő jegy
	INTEGER	Price	A kiegészítő jegy ára

Customer / Customers

A táblában a vendégek adatai kerülnek tárolásra, mint például a kártyaazonosító, név, létrehozási dátum és aktivitási státusz. Ezen információk alapján történik a vendég azonosítása, valamint a vásárlások és rendeléseik nyomon követése. Az *IsActive* mező jelzi, hogy a vendég aktív-e a rendszerben.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3  using System.Text.Json.Serialization;
4  using CMS.Models;
5
6  namespace CMS.Models
7  {
8      7 references
9      public class Customer
10     {
11         [Key]
12         3 references
13         public int CustomerId { get; set; }
14         [Required]
15         4 references
16         public string CardId { get; set; }
17         [Required]
18         2 references
19         public string Name { get; set; }
20         [Required]
21         2 references
22         public DateTime CreatedAt { get; set; }
23         [Required]
24         3 references
25         public bool IsActive { get; set; }
26         [Required]
27         1 reference
28         public int CreatedBy { get; set; }
29         [JsonIgnore]
30         0 references
31         public List<Order>? Orders { get; set; }
32         [JsonIgnore]
33         [InverseProperty("Customers")]
34         0 references
35         public List<CustomerAdmission>? CustomerAdmissions { get; set; }
36         [JsonIgnore]
37         [InverseProperty("Customers")]
38         4 references
39         public List<CustomerTicket>? CustomerTickets { get; set; }
40     }
41 }

```

79. ábra – Customer.cs, Vendégek tábla felépítése

Customers			
	Típus	Mezőnév	Megjegyzés
	INTEGER	CustomerId	Auto increment
	TEXT	CardId	A vendéghez hozzárendelt kártya azonosítója
	TEXT	Name	Vendég keresztnéve a megszólítás miatt
	TEXT	CreatedAt	A vendég rekord létrehozásának dátuma
	INTEGER	IsActive	A vendég engedélyezett-e

CustomerAdmission / CustomerAdmissions

A *CustomerAdmissions* táblában tároljuk az *Admissions* és a *Customers* tábla közötti kapcsolatot. A *CustomerId* és az *AdmissionId* mezők hivatkoznak az egyes vendégek és kiegészítő jegyek azonosítójára, ezzel lehetővé téve, hogy egy vendég több kiegészítő jegyet is vásároljon. A *CustomerAdmissionId* mező egyedi azonosítót biztosít a rekordok számára.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3  using System.Text.Json.Serialization;
4
5  namespace CMS.Models
6  {
7      8 references
8      public class CustomerAdmission
9      {
10         [Key]
11         public int CustomerAdmissionId { get; set; }
12         [Required]
13         public int CustomerId { get; set; }
14         [ForeignKey(nameof(CustomerId))]
15         public Customer Customers { get; set; }
16         [Required]
17         public int AdmissionId { get; set; }
18         [ForeignKey(nameof(AdmissionId))]
19         public Admission Admissions { get; set; }
20     }
21 }
22

```

80. ábra – CustomerAdmission.cs, Kapcsolótábla a Customer és az Admission táblához

CustomerAdmissions			
	Típus	Mezőnév	Megjegyzés
🔑	INTEGER	CustomerAdmissionId	Auto increment
	INTEGER	CustomerId	A „Customers” tábla azonosítója
	INTEGER	AdmissionId	Az „Admissions” tábla azonosítója

CustomerTicket / CustomerTickets

A *CustomerTickets* tábla a *Customers* és a *Tickets* táblák közötti kapcsolatot tárolja. Egy vendég több jegyet is vásárolhat, így a tábla lehetővé teszi a vendégek és jegyek közötti kapcsolatok nyomon követését. A *CustomerId* és *TicketId* mezők az adott vásárló és jegy azonosítójára hivatkoznak.

```
1 using System.ComponentModel.DataAnnotations.Schema;
2 using System.ComponentModel.DataAnnotations;
3
4 namespace CMS.Models
5 {
6     10 references
7     public class CustomerTicket
8     {
9         [Key]
10         public int CustomerTicketId { get; set; }
11         [Required]
12         public int CustomerId { get; set; }
13         [ForeignKey(nameof(CustomerId))]
14         1 reference
15         public Customer Customers { get; set; }
16         0 references
17         [Required]
18         public int TicketId { get; set; }
19         [ForeignKey(nameof(TicketId))]
20         2 references
21         public Ticket Tickets { get; set; }
22     }
23 }
```

81. ábra – *CustomerTicket.cs*, Kapcsolótábla a *Customer* és a *Ticket* táblához

CustomerTickets			
	Típus	Mezőnév	Megjegyzés
🔑	INTEGER	CustomerTicketId	Auto increment
	INTEGER	CustomerId	A „Customers” tábla azonosítója
	INTEGER	TicketId	Az „Ticket” tábla azonosítója

DailySpecial / DailySpecials

Ezen táblában tartalmazza az adott napi ajánlat azonosítóját, melyet kizárólag adminisztrátori hozzáféréssel lehet rögzíteni és törlni. Az éttermi minden egyes kategóriájából kell egyet kiválasztani, melyet az éttermi dolgozó, az rendelési felületen lát.

```

1  using System.ComponentModel.DataAnnotations;
2
3  namespace CMS.Models
4  {
5      [Table("DailySpecial")]
6      public class DailySpecial
7      {
8          [Key]
9          [Required]
10         public int SpecialId { get; set; }
11         [Required]
12         public int SoupId { get; set; }
13         [Required]
14         public int AppetizerId { get; set; }
15         [Required]
16         public int MainCourseId { get; set; }
17         [Required]
18         public int HamburgerId { get; set; }
19         [Required]
20         public int PizzaId { get; set; }
21         [Required]
22         public int DessertId { get; set; }
23         [Required]
24         public int DrinkId { get; set; }
25         [Required]
26         public int CoffeeId { get; set; }
27     }
28 }

```

82. ábra – DailySpecial.cs, Napi menü tábla felépítése

DailySpecials			
	Típus	Mezőnév	Megjegyzés
	INTEGER	SpecialId	Auto increment
	INTEGER	AppetizerId	A napi előétel azonosítója
	INTEGER	MainCourseId	A napi főétel azonosítója
	INTEGER	HamburgerId	A napi hamburger azonosítója
	INTEGER	PizzaId	A napi pizza azonosítója
	INTEGER	DessertId	A napi desszert azonosítója
	INTEGER	DrinkId	A napi ital termék azonosítója
	INTEGER	CoffeeId	A napi kávé termék azonosítója

Employee / Employees

Ebben a táblában a dolgozók adatai vannak tárolva, amelyek a rendszerben való azonosításhoz és jogosultságkezeléshez szükségesek. A *EmployeeId* egyedi azonosítót ad, a *FirstName*, *LastName*, *Role*, *Username* és *PasswordHash* mezők pedig a dolgozók személyes adatait és hitelesítési információit tárolják.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3  using System.Text.Json.Serialization;
4
5  namespace CMS.Models
6  {
7      public class Employee
8      {
9          [Key]
10         public int EmployeeId { get; set; }
11         [Required]
12         public string FirstName { get; set; }
13         [Required]
14         public string LastName { get; set; }
15         [Required]
16         public string Role { get; set; }
17         [Required]
18         public string Username { get; set; }
19         [Required]
20         public string PasswordHash { get; set; }
21         [JsonIgnore]
22         public List<Order>? Orders { get; set; }
23     }
24 }
25
26

```

83. ábra – Employee.cs, Dolgozó tábla felépítése

Employees			
	Típus	Mezőnév	Megjegyzés
	INTEGER	EmployeeId	Auto increment
	TEXT	FirstName	A dolgozó keresztnéve
	TEXT	LastName	A dolgozó vezetéknéve
	TEXT	Role	A dolgozó munkaköre, ez alapján tud belépni a különböző felületekre
	TEXT	Username	A dolgozó felhasználóneve

MenuItem / MenuItems

Ebben a táblában kerülnek tárolásra az étteremben elérhető termékek adatai. A *ItemId* mező egyedi azonosítót biztosít minden termék számára. A *Name* mező a termék nevét, míg a *Category* a termék kategóriáját tartalmazza. A *Price* mező a termék árát forintban, a *Description* pedig egy rövid leírást ad a termékről. Az *IsAvailable* mező jelzi, hogy a termék elérhető-e, míg az *ImagePath* a termék képének relatív elérési útját tárolja.

```
1 using System.ComponentModel.DataAnnotations;
2 using System.ComponentModel.DataAnnotations.Schema;
3 using System.Text.Json.Serialization;
4
5 namespace CMS.Models
6 {
7     [Table("MenuItems")]
8     public class MenuItem
9     {
10         [Key]
11         public int ItemId { get; set; }
12
13         [Required]
14         public string Name { get; set; }
15
16         [Required]
17         public string Category { get; set; }
18
19         [Required]
20         public int Price { get; set; }
21
22         [Required]
23         public string Description { get; set; }
24
25         [Required]
26         public bool IsAvailable { get; set; }
27
28         [Required]
29         public string ImagePath { get; set; }
30
31         [JsonIgnore]
32         public List<Order> Orders { get; set; }
33
34         [JsonIgnore]
35         [InverseProperty("MenuItems")]
36         public List<MenuItemOrder> MenuItemOrders { get; set; }
37     }
38 }
39
40
```

84. ábra – MenuItem.cs, Termékek tábla felépítése

MenuItems			
	Típus	Mezőnév	Megjegyzés
	INTEGER	ItemId	Auto increment
	TEXT	Name	A termék neve
	TEXT	Category	A termék kategóriája
	INTEGER	Pice	A termék ára forintban
	TEXT	Description	Egy rövid leírás a termékről
	INTEGER	IsAvailable	Elérhető-e az adott termék
	TEXT	ImagePath	A termék képének relatív útvonala

MenuItemOrder / MenuItemOrders

A *MenuItemOrders* táblát kapcsolatként használjuk a *MenuItems* és az *Orders* táblák között, hogy egy étlap elem több rendelésben is szerepelhessen, illetve egy rendelés több étlap elemet is tartalmazhasson. Ezen felül a táblában szerepel egy *Quantity* mező, amely tárolja, hogy egy adott termék hányszor található meg a rendelésben.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3
4  namespace CMS.Models
5  {
6      11 references
7      public class MenuItemOrder
8      {
9          [key]
10         public int MenuItemOrderId { get; set; }
11         [Required]
12
13         1 reference
14         public int ItemId { get; set; }
15         [ForeignKey(nameof(ItemId))]
16         2 references
17         public MenuItem MenuItems { get; set; }
18         [Required]
19         1 reference
20         public int OrderId { get; set; }
21         [ForeignKey(nameof(OrderId))]
22         1 reference
23         public Order Orders { get; set; }
24         2 references
25         public int Quantity { get; set; }
26     }

```

85. ábra – *MenuItemOrder.cs*, Kapcsolótábla a *MenuItem* és az *Order* táblához

MenuItemOrders			
	Típus	Mezőnév	Megjegyzés
	INTEGER	MenuItemOrderId	Auto increment
	INTEGER	ItemId	A „MenuItem” tábla azonosítója
	INTEGER	OrderId	Az „Orders” tábla azonosítója
	INTEGER	Quantity	Mennyiséget meghatározó mező

Order / Orders

Az *Order* osztály a vendégek rendeléseit tárolja. Ez az osztály pontosan meghatározza, hogy egy adott rendelés mikor történt, ki a vendég, aki a rendelést leadta, és ki a dolgozó, aki rögzítette azt. Az *Orders* táblában tárolt adatok segítenek nyomon követni a rendeléseket az étterem vagy szolgáltatás számára.

```

1  using System.ComponentModel.DataAnnotations;
2  using System.ComponentModel.DataAnnotations.Schema;
3  using System.Text.Json.Serialization;
4  using CMS.Models;
5
6  namespace CMS.Models
7  {
8      public class Order
9      {
10         [Key]
11         public int OrderId { get; set; }
12         [Required]
13         public int CustomerId { get; set; }
14         [Required]
15         public int EmployeeId { get; set; }
16         [Required]
17         public DateTime CreatedAt { get; set; }
18
19         public List<MenuItem> MenuItemItems { get; set; }
20         [JsonIgnore]
21         [InverseProperty("Orders")]
22         public List<MenuItemOrder> MenuItemOrders { get; set; }
23         [JsonIgnore]
24         [ForeignKey("CustomerId")]
25         public Customer? Customer { get; set; }
26         [JsonIgnore]
27         [ForeignKey("EmployeeId")]
28         public Employee? Employee { get; set; }
29     }
30

```

86. ábra – Order.cs, Rendelés tábla felépítése

Orders			
	Típus	Mezőnév	Megjegyzés
	INTEGER	OrderId	Auto increment
	INTEGER	CustomerId	A vendég azonosítója
	INTEGER	EmployeeId	A dolgozó azonosítója
	TEXT	CreatedAt	Rögzíti, hogy mikor került rögzítésre a rendelés

RefreshToken / RefreshTokens

A RefreshToken osztály feladata a tokenek tárolása, amelyek biztosítják a rendszert használó felhasználók bejelentkezését és hitelesítését. Minden token egy adott felhasználóhoz (dolgozóhoz) tartozik, és meghatározott idő után lejár.

```
1 namespace CMS.Models
2 {
3     3 references
4     public class RefreshToken
5     {
6         0 references
7         public int Id { get; set; }
8
9         5 references
10        public string Token { get; set; }
11
12        2 references
13        public DateTime Expires { get; set; }
14
15        2 references
16        public int EmployeeId { get; set; }
17
18        2 references
19        public Employee Employee { get; set; }
20    }
21 }
```

87. ábra – RefreshToken.cs, Token tábla felépítése

RefreshTokens			
	Típus	Mezőnév	Megjegyzés
🔑	INTEGER	Id	Auto increment
	TEXT	Token	A token kulcs
	TEXT	Expires	A token lejárat dátuma
	INTEGER	EmployeeId	Itt tárolódik, hogy az adott token melyik felhasználóhoz tartozik

Ticket / Tickets

A *Ticket* osztály a fő kategóriába tartozó jegyek tárolására szolgál. Minden vendégnek először egy ilyen típusú jeggyel kell rendelkeznie a belépéshez.

```

1  using System.ComponentModel.DataAnnotations.Schema;
2  using System.ComponentModel.DataAnnotations;
3  using System.Text.Json.Serialization;
4
5  namespace CMS.Models
6  {
7      public class Ticket
8      {
9          [Key]
10         public int TicketId { get; set; }
11         [Required]
12         public string? TicketName { get; set; }
13         [Required]
14         public string Category { get; set; }
15         [Required]
16         public int Price { get; set; }
17         [Required]
18         public string Description { get; set; }
19         [Required]
20         public bool IsAvailable { get; set; }
21         [Required]
22         public string ImagePath { get; set; }
23         [JsonIgnore]
24         [InverseProperty("Tickets")]
25         public List<CustomerTicket> CustomerTickets { get; set; }
26     }
27 }

```

88. ábra – Ticket.cs, Jegy tábla felépítése

Tickets			
	Típus	Mezőnév	Megjegyzés
	INTEGER	TicketId	Auto increment
	TEXT	TicketName	A jegy pontos megnevezése
	TEXT	Category	A jegy kategóriája
	TEXT	Description	A jegy rövid megnevezése
	TEXT	ImagePath	A jegy képének relatív útvonala
	INTEGER	IsAvailable	Elérhető-e az adott jegy
	INTEGER	Price	A jegy ára

Kontextus osztály

A *CMSContext* osztály az Entity Framework Core alapértelmezett adatbázis-kapcsolat kezelője, amely a *DbContext* osztályból származik. A *DbContext* osztály biztosítja az adatbázis műveletek kezelését, és a *CMSContext* osztály egyéni alkalmazás-specifikus implementációját tartalmazza.

```
1  using Microsoft.EntityFrameworkCore;
2  using CMS.Models;
3
4  namespace CMS.Models
5  {
6      // context
7      public class CMSContext : DbContext
8      {
9          // constructor
10         public CMSContext(DbContextOptions<CMSContext> options) : base(options)
11         {
12         }
13
14         // DbSet<Model> ModelName { get; set; } = null;
15         public DbSet<MealItem> MealItems { get; set; } = null;
16         public DbSet<MealItemOrder> MealItemOrders { get; set; } = null;
17         public DbSet<Order> Orders { get; set; } = null;
18
19         // DbSet<Customer> Customers { get; set; } = null;
20         public DbSet<Customer> Customers { get; set; } = null;
21
22         // DbSet<Employee> Employees { get; set; } = null;
23         public DbSet<Employee> Employees { get; set; } = null;
24
25         // DbSet<Admission> Admissions { get; set; } = null;
26         public DbSet<Admission> Admissions { get; set; } = null;
27
28         // DbSet<CustomerAdmission> CustomerAdmissions { get; set; } = null;
29         public DbSet<CustomerAdmission> CustomerAdmissions { get; set; } = null;
30
31         // DbSet<RefreshToken> RefreshTokens { get; set; } = null;
32         public DbSet<RefreshToken> RefreshTokens { get; set; } = null;
33
34         // DbSet<Ticket> Tickets { get; set; } = null;
35         public DbSet<Ticket> Tickets { get; set; } = null;
36
37         // DbSet<CustomerTicket> CustomerTickets { get; set; } = null;
38         public DbSet<CustomerTicket> CustomerTickets { get; set; } = null;
39
40         // DbSet<DailySpecial> DailySpecials { get; set; } = null;
41         public DbSet<DailySpecial> DailySpecials { get; set; } = null;
42
43         // protected override void OnModelCreating(ModelBuilder modelBuilder)
44         protected override void OnModelCreating(ModelBuilder modelBuilder)
45         {
46             modelBuilder.Entity<MealItem>()
47                 .HasMany(m => m.Orders)
48                 .WithMany(o => o.MealItems)
49                 .UsingEntity<MealItemOrder>();
50
51             modelBuilder.Entity<Employee>().HasIndex(e => e.Username).IsUnique();
52         }
53     }
54 }
```

89. ábra – *CMSContext.cs*, Adatbázis kapcsolatok kezelése

3.3.5 Dependency Injection (DI)

A *Dependency Injection (DI)* egy tervezési minta, amely segít az alkalmazás modulárisabbá és tesztelhetőbbé tételében azáltal, hogy a szükséges függőségeket külső forrásból injektálja az osztályokba, ahelyett, hogy azok saját maguk hozzák létre. *CMSContext.cs* tartalmazza a függőségi befecskendezést (*DI*).

```
9
10
11     public CMSContext(DbContextOptions<CMSContext> options) : base(options)
12     {
13     }
14 }
```

90. ábra – *CMSContext.cs*, Függőségi befecskendezés

DbSet tulajdonságok

A *DbSet* tulajdonságok az Entity Framework Core-ban az adatbázis táblákat reprezentálják. Itt definiáljuk az adatbázisban szereplő táblák neveit. Minden *DbSet* tulajdonság egy-egy entitást reprezentál, amely az adatbázis egy-egy táblájához kapcsolódik. Ezek a tulajdonságok a *CMSContext* osztályban találhatóak, és az adatbázis műveletek végrehajtásához szükségesek.

```

17 public DbSet<MenuItem> MenuItem { get; set; } = null!;
18 //reference
19 public DbSet<MenuItemOrder> MenuItemOrders { get; set; } = null!;
20 //reference
21 public DbSet<Order> Orders { get; set; } = null!;
22
23 //reference
24 public DbSet<Customer> Customers { get; set; } = null!;
25
26 //reference
27 public DbSet<Employee> Employees { get; set; } = null!;
28
29 //reference
30 public DbSet<Admission> Admissions { get; set; } = null!;
31
32 //reference
33 public DbSet<CustomerAdmission> CustomerAdmissions { get; set; } = null!;
34
35 //reference
36 public DbSet<RefredIdadems> RefredIdadems { get; set; } = null!;
37
38 //reference
39 public DbSet<Ticket> Tickets { get; set; } = null!;
40
41 //reference
42 public DbSet<CustomerTicket> CustomerTickets { get; set; } = null!;
43
44 //reference
45 public DbSet<DailySpecial> DailySpecials { get; set; } = null!;

```

91. ábra – *CMSContext.cs*, Táblák reprezentálása *DbSet* segítségével

3.3.6 OnModelCreating Metódus és Entitás Kapcsolatok Konfigurálása

Az *OnModelCreating* metódus lehetővé teszi az entitások közötti kapcsolatok és egyedi szabályok konfigurálását az adatbázisban.

Itt kerül beállításra egy több a többhöz kapcsolat is a *MenuItem* és *Order* entitások között egy köztes *MenuItemOrder* entitás segítségével:

```

30 protected override void OnModelCreating(ModelBuilder modelBuilder)
31 {
32     modelBuilder.Entity<MenuItem>()
33         .HasMany(e => e.Orders)
34         .WithMany(e => e.MenuItemOrders)
35         .UsingEntity<MenuItemOrder>();
36
37     modelBuilder.Entity<Employee>().HasIndex(u => u.Username).IsUnique();
38 }

```

92. ábra – *CMSContext.cs*, Metódus és Entitás kapcsolatok konfigurálása

3.3.7 DTO-k (Data Transfer Object)

A *DTO* (*Data Transfer Object*) egy egyszerű adatobjektum, amelyet az adatok különböző rendszerek vagy alkalmazásrétegek közötti továbbítására használnak. Az *API* hívásokban alkalmazva lehetővé teszi az adatok strukturált, egyszerűsített és biztonságos kezelését. A *DTO*-k segítenek csökkenteni a rendszer bonyolultságát, és biztosítják, hogy csak a szükséges információk kerüljenek továbbításra, miközben támogatják a hatékony kommunikációt a frontend és a backend között.

Példa a DTO használatára

A *MenuItemPostDto* osztály az étterem rendelési rendszerében új menü tételek hozzáadására szolgál. A *DTO* mezői a következő adatokat tartalmazzák:

```
1 namespace CMS.DTOs
2 {
3     [reference]
4     public class MenuItemPostDto
5     {
6         [reference]
7         public string Name { get; set; }
8         [reference]
9         public string Category { get; set; }
10        [reference]
11        public int Price { get; set; }
12        [reference]
13        public string Description { get; set; }
14        [reference]
15        public bool IsAvailable { get; set; }
16        [reference]
17        public string ImagePath { get; set; }
18    }
19 }
```

93. ábra – *MenuItemPostDto.cs*, *POST* kérés során megjelenő adatok listája

Tulajdonságok

- **Name:** Az étel vagy ital neve.
- **Category:** Az étel vagy ital kategóriája (pl. főétel, desszert).
- **Price:** Az étel ára.
- **Description:** Az étel leírása.
- **IsAvailable:** Az étel elérhetősége (elérhető vagy nem).
- **ImagePath:** Az ételhez tartozó kép elérési útja.

A DTO előnyei

- **Karbantarthatóság:** A DTO-k segítenek a kód karbantartásában, mivel minden adat egy jól meghatározott objektumban van, így az alkalmazás komponensei közötti kommunikáció egyszerűbbé válik.
- **Teljesítmény:** A DTO-k csak a szükséges adatokat továbbítják, csökkentve ezzel a hálózati terhelést és optimalizálva az adatforgalmat.
- **Biztonság:** Az érzékeny adatokat nem szükséges közvetlenül küldeni vagy tárolni, mivel a DTO-k lehetőséget biztosítanak az adatok biztonságos kezelésére.

3.3.8 Controllerek

A **Controller** egy olyan osztály az *ASP.NET Core Web API* alkalmazásokban, amely az ügyfelektől érkező HTTP-kérések kezeléséért felel.

Útvonalkezelés (Routing)

- A **[Route]** attribútum meghatározza, hogy egy adott végpont (API endpoint) milyen URL-cím alatt érhető el.

- Példa: **[Route("api/[controller]")]** → Automatikusan az osztály nevéből (pl. *CustomersController*) generálódik az útvonal: **api/Customers**.

HTTP-műveletek kezelése

- **[HttpGet]** – Adatok lekérése
- **[HttpPost]** – Új adatok létrehozása
- **[HttpPut]** – Meglévő adatok frissítése
- **[HttpDelete]** – Rekord törlése az adatbázisból

Adatkezelés és a szűrés végrehajtása, fontosabb végpontok

Az *Entity Framework Core* segítségével adatbázis-műveleteket hajt végre (pl. adatok lekérése, módosítása, törlése).

Segédfüggvényeket tartalmazhat (pl. létezik-e az adott rekord).

Válaszok visszaadása a kliensnek

- **200 OK** – Sikeres kérés esetén.
- **201 Created** – Új adat sikeres létrehozásakor.
- **400 Bad Request** – Hibás kérés esetén.
- **401 Unauthorized** – Amennyiben token nélkül kerül a végpont meghívásra
- **404 Not Found** – Ha a keresett adat nem található.
- **406 Not Acceptable** – Ha az érték nem elfogadható
- **500 Internal Server Error** – Ha szerverhiba történik.

Példa a Controllerek használatára

Jogosultság jelölések	
Jogosultság nélkül	
Bármely belépett felhasználó	
Admin	 A
Admin & TicketAssistant	 A&TA
Admin & RestaurantAssistant	 A&RA

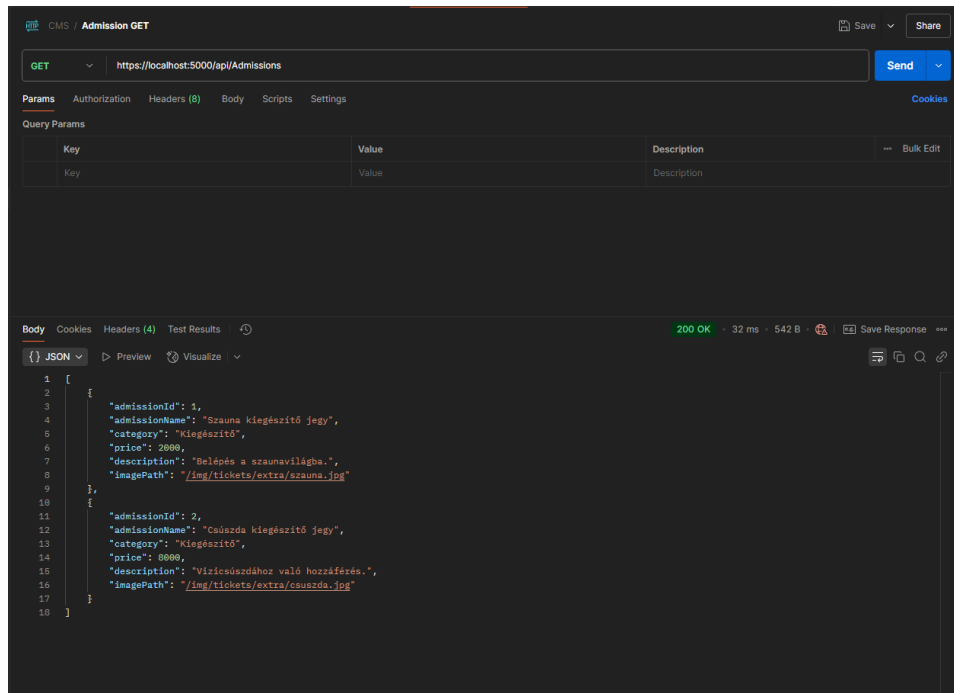
A tesztelés során a védett végpontok eléréséhez *JWT token* használatára van szükség. A hitelesítést a már említett *Postman* eszköz segítségével végeztük, ahol a *token* manuálisan került beállításra az egyes kérésekhez az „*Header*” résznél. Ez lehetővé teszi, hogy az autentikációt igénylő végpontokat is teszteljük. A JWT biztosítja, hogy csak érvényes jogosultsággal rendelkező felhasználók férjenek hozzá az adott erőforrásokhoz. A most következő példánál egy Adminisztrátor szerepkörrel rendelkező felhasználó *JWT tokenje* került felhasználásra.

AdmissionController

A jegyvásárlás kezelő oldal ezen a végponton keresztül kéri le az elérhető kiegészítő jegytípusokat, amelyek közül az értékesítés lebonyolítható.

Elérhető végpont:

-  **GET** /api/Admissions: Lekéri az összes kiegészítő jegyet



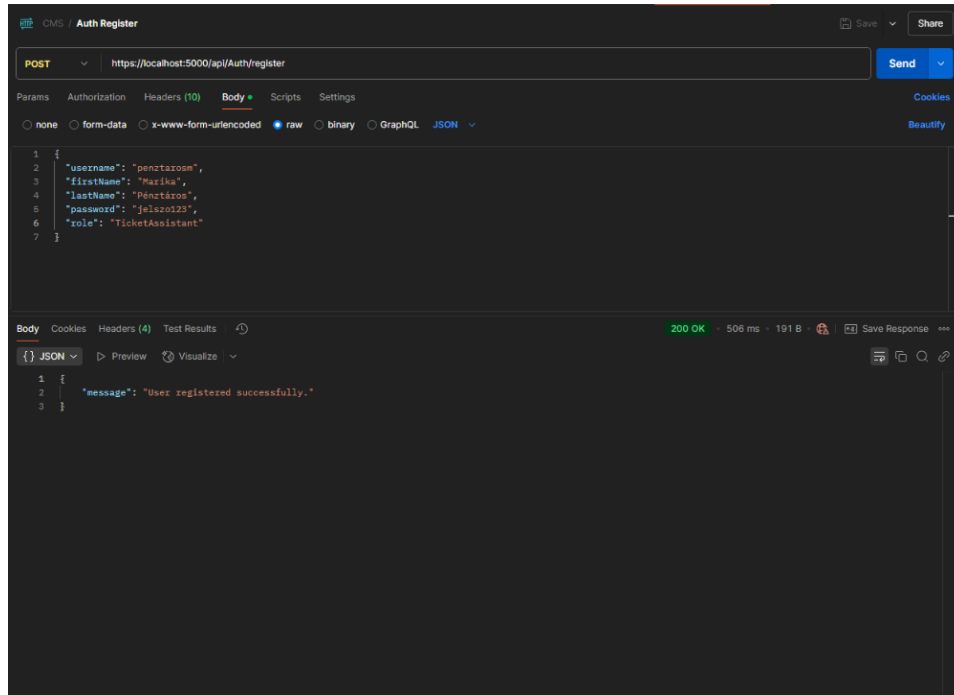
94. ábra – Admissions GET végpont visszatérő értékei

AuthController

Az controller a felhasználói hitelesítést és jogosultságokat kezeli, beleértve a regisztrációt, bejelentkezést, refresh token kezelését és a kijelentkezést.

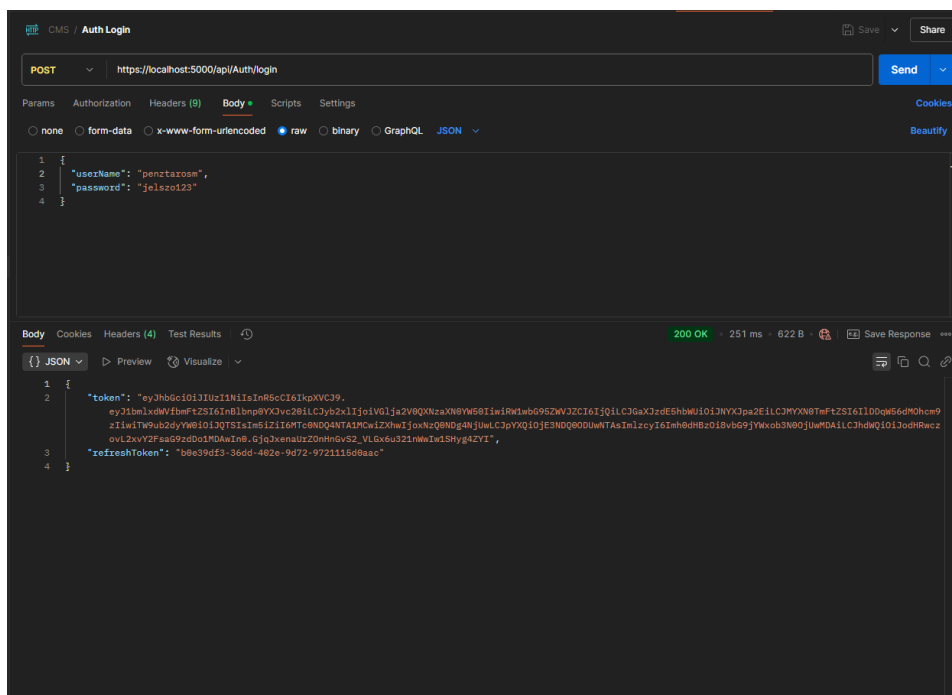
Elérhető végpontok:

-  ^A POST /api/Auth/register: Új dolgozói fiók létrehozására



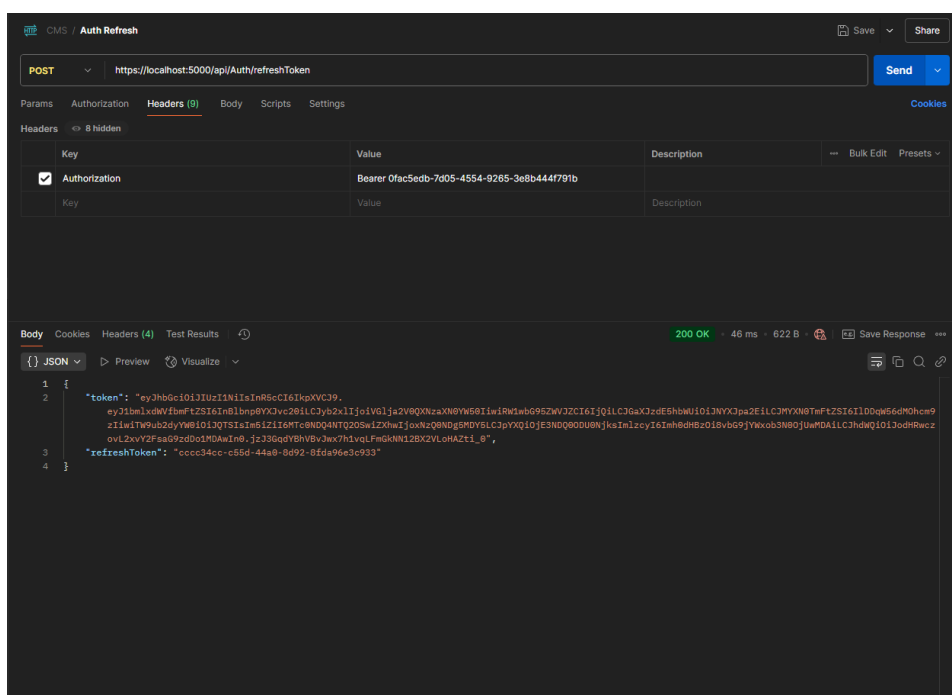
95. ábra – Auth/register POST végpont visszatérő értéke

-  POST /api/Auth/login: Bejelentkezés, JWT token és refresh token generálása, sikeres belépést követően.



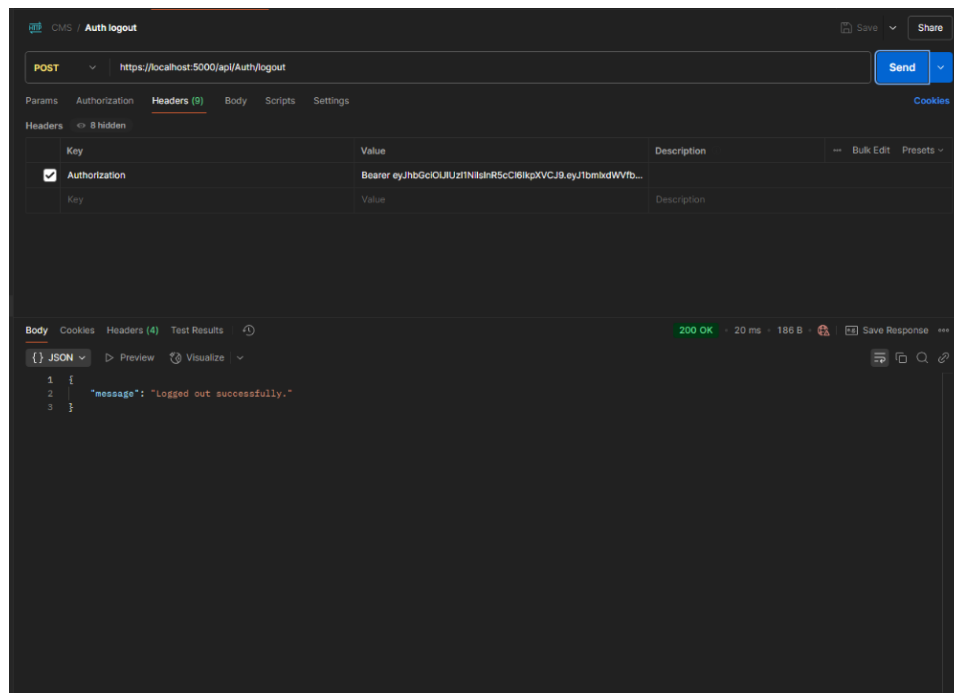
96. ábra – Auth/login POST végpont visszatérő értéke

-  POST /api/Auth/refreshToken: Frissítő token alapján új JWT token generálása.



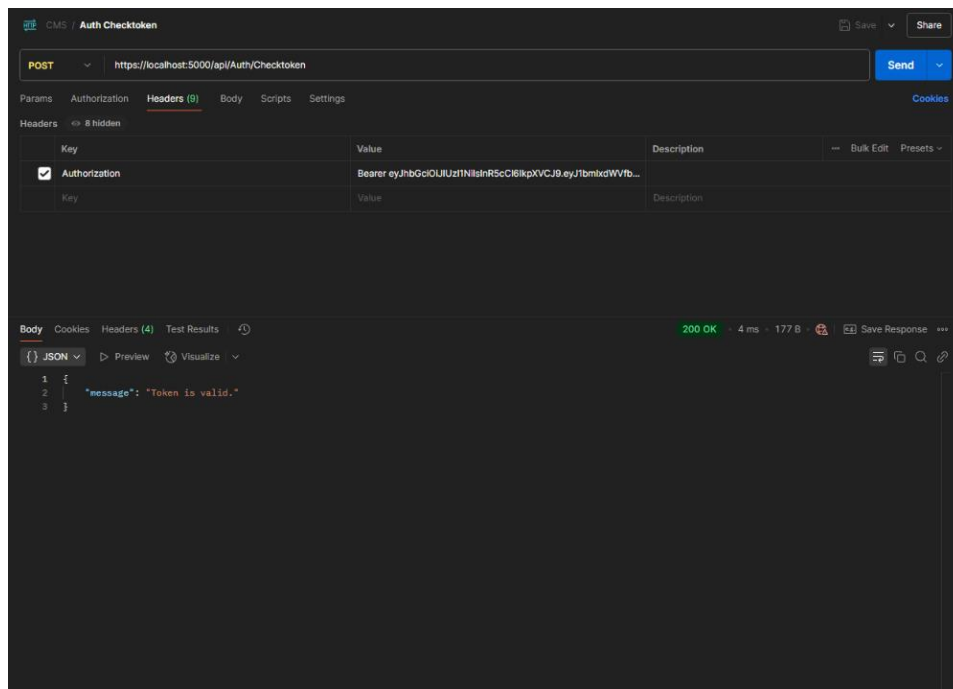
97. ábra – Auth/refreshToken POST végpont visszatérő értéke

- 🔒 POST /api/Auth/logout: Kijelentkezés, token és a refreshtoken érvénytelenítése.



98. ábra – Auth/logout POST végpont visszatérő értéke

- 🔒 POST /api/Auth/Checktoken: Leellenőrzi, hogy a token érvényes-e



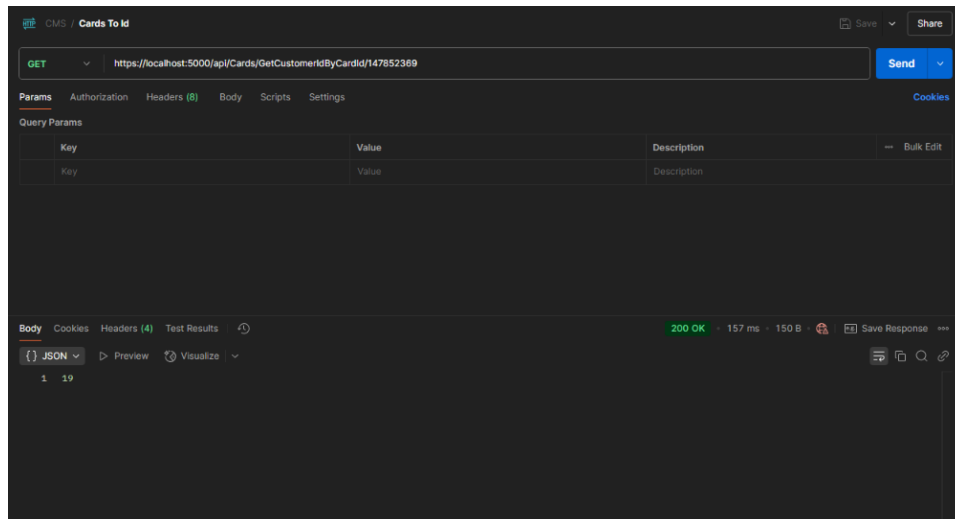
99. ábra – Auth/Checktoken POST végpont visszatérő értéke

CardController

A *CardController* egyetlen végpontot tartalmaz mely visszakeresi a vendég kártyája alapján az adatbázisban használt azonosítójára, ha aktív.

Elérhető végpontok:

- 🔒 A&RA GET /api/Cards/GetCustomerIdByCardId/{cardid}: Visszakeresi a vendég azonosítóját, ami az adott aktív kártyához van rendelve.



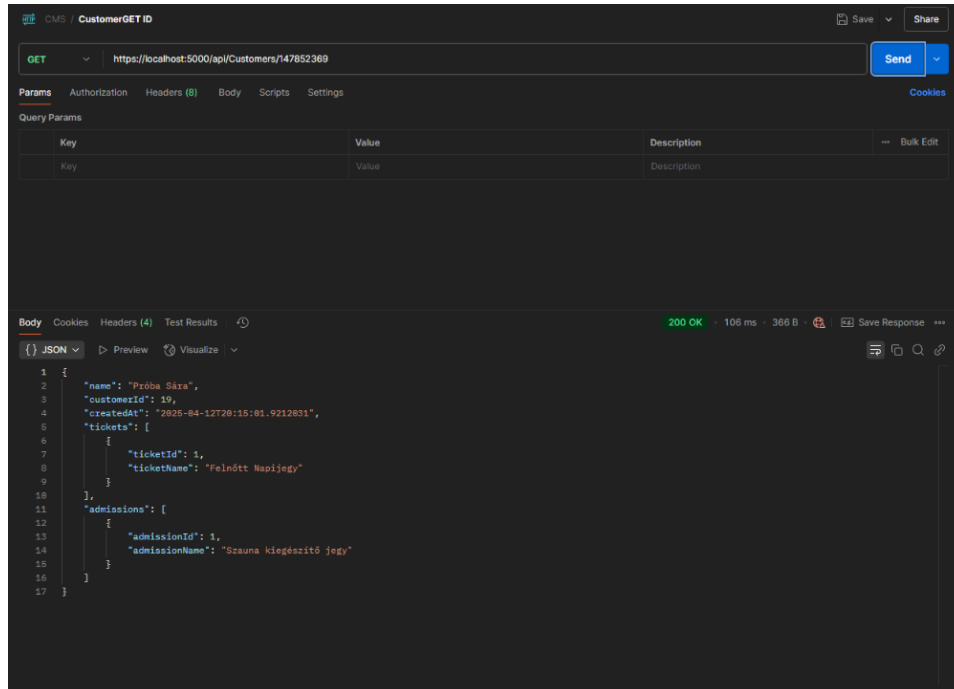
100. ábra – Cards/GetCustomerIdByCardId GET végpont visszatérő értéke

CustomerController

A *CustomerController* a vendégek kezeli a vendégek adatait módosító végpontokat.

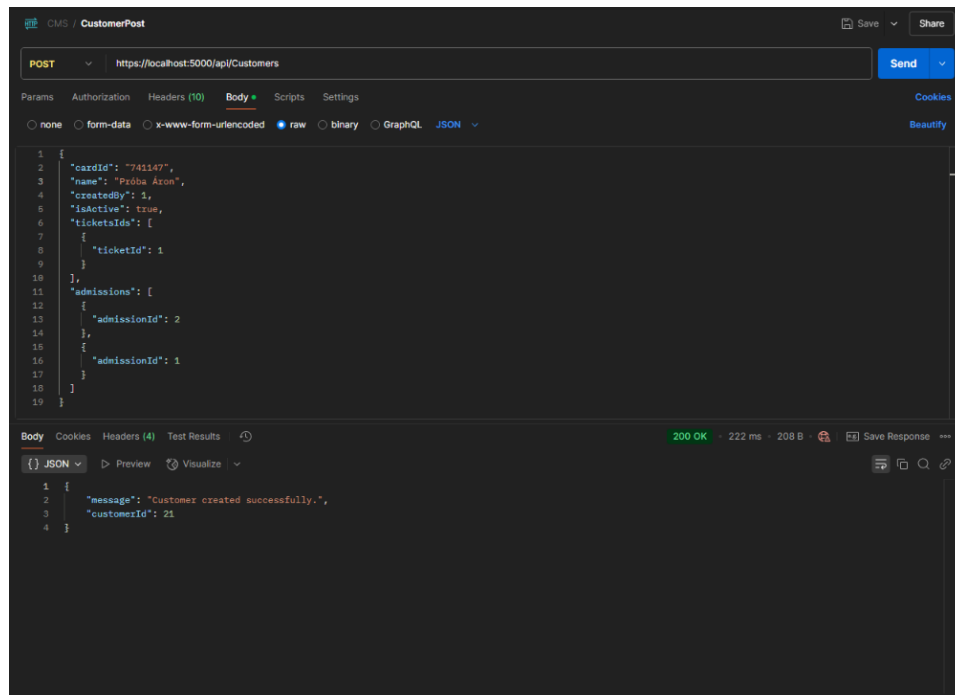
Elérhető végpontok:

-  GET /api/Customers/{cardid}: Visszakeresi a vendég jogosultságait, ami az adott kártyához van rendelve.



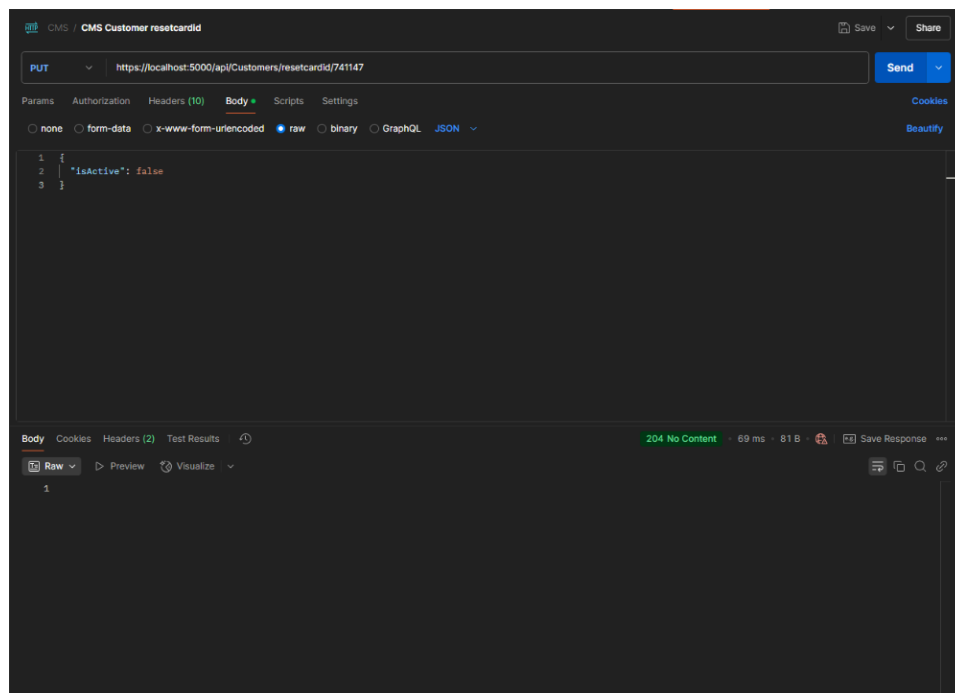
101. ábra – Customers/{cardid} GET végpont visszatérő értéke

-  A&TA POST /api/Customers: Új vendég létrehozása



102. ábra – Customers POST végpont visszatérési értéke

-  A&RA PUT /api/Customers/resetcardid/{cardid}: A vendég kijelentkezésekor használjuk, hogy inaktívvá változtatjuk a státuszát és a neve alól kikerül a kártya, így a kártya újra kiadásra kerülhet.



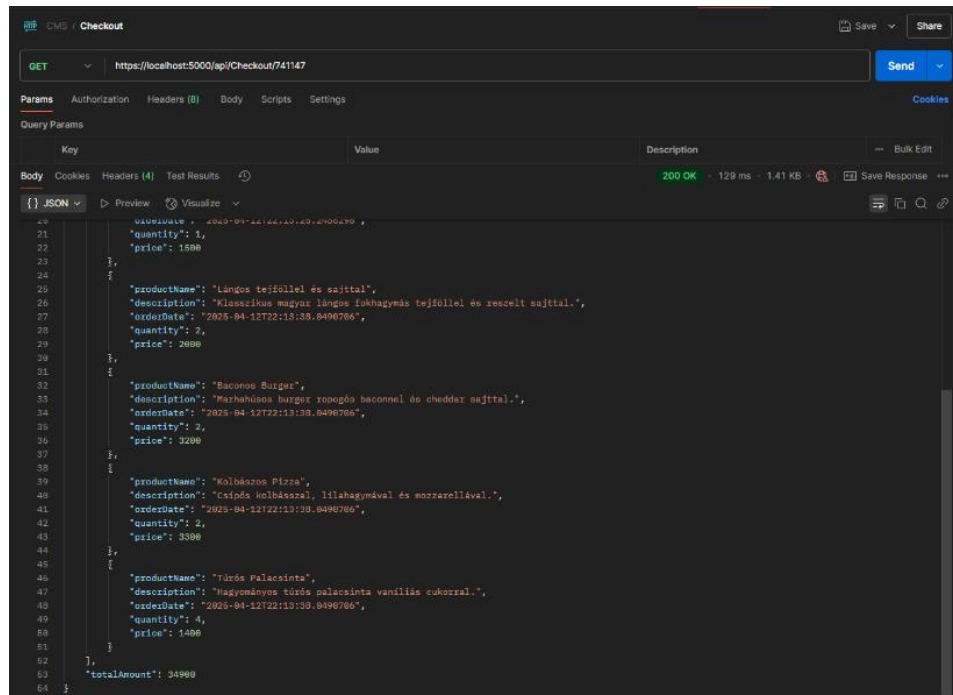
103. ábra – Customers/Resetcardid PUT végpont visszatérési értéke

CheckoutController

A CheckoutController a vendégek kijelentkeztetésekor kerül meghívásra.

Elérhető végpontok:

 A&TA GET /api/Checkout/{customerCardId}: Vendég fogyasztásainak lekérdezése kártya alapján.



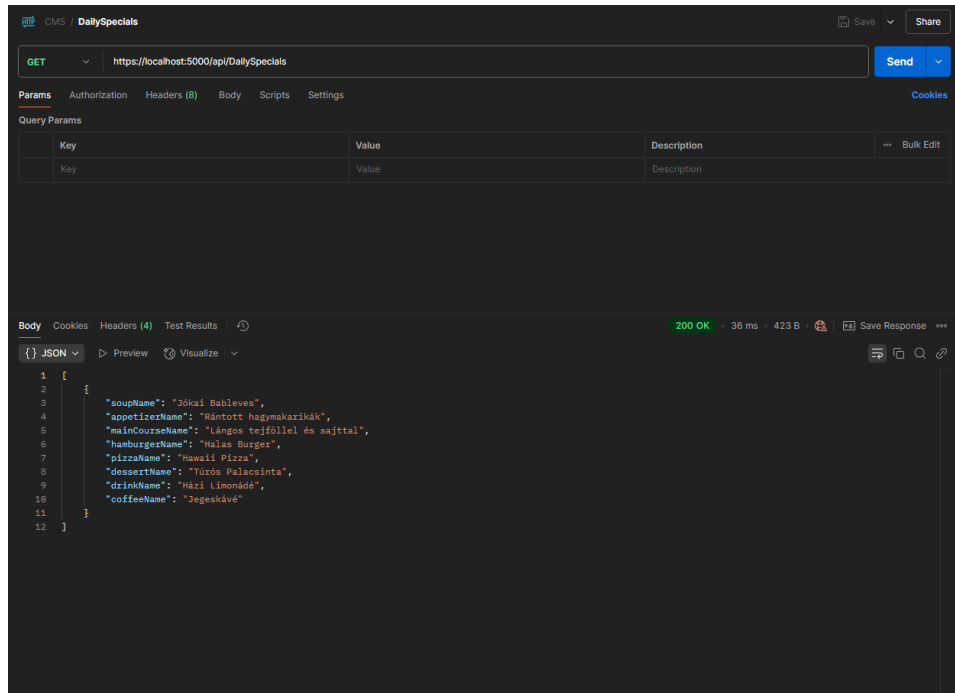
104. ábra – Checkout/{customerCardId} GET végpont visszatérési értéke

DailySpecialsController

A *DailySpecialsController* felel a napi ajánlatok kezeléséért és annak nyilvántartásáról.

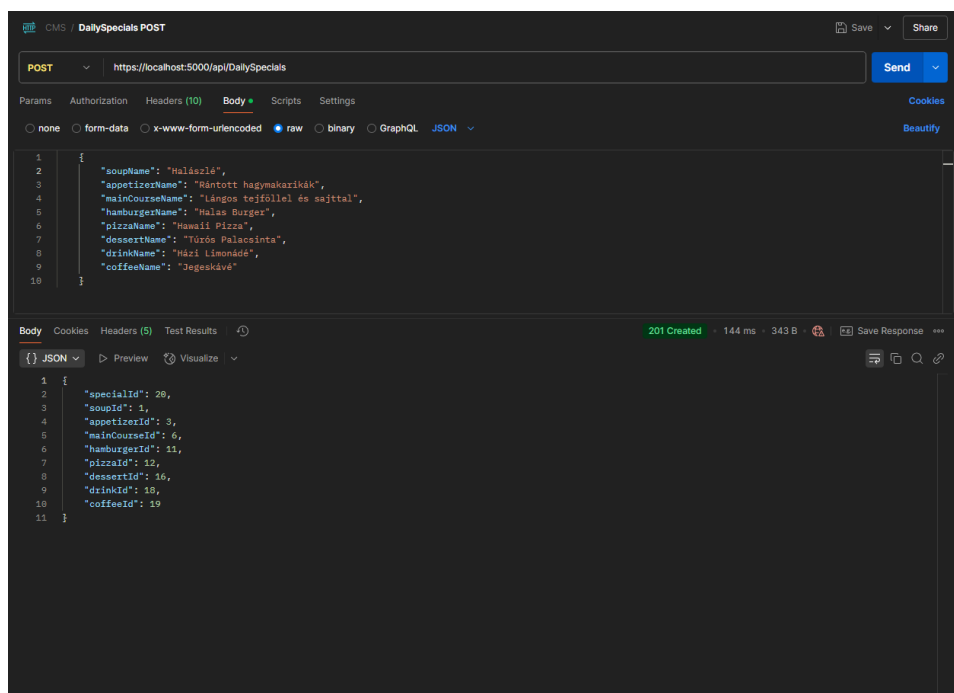
Elérhető végpontok:

-  A&RA GET /api/DailySpecials: Az aktuális napi ajánlat lekérdezése



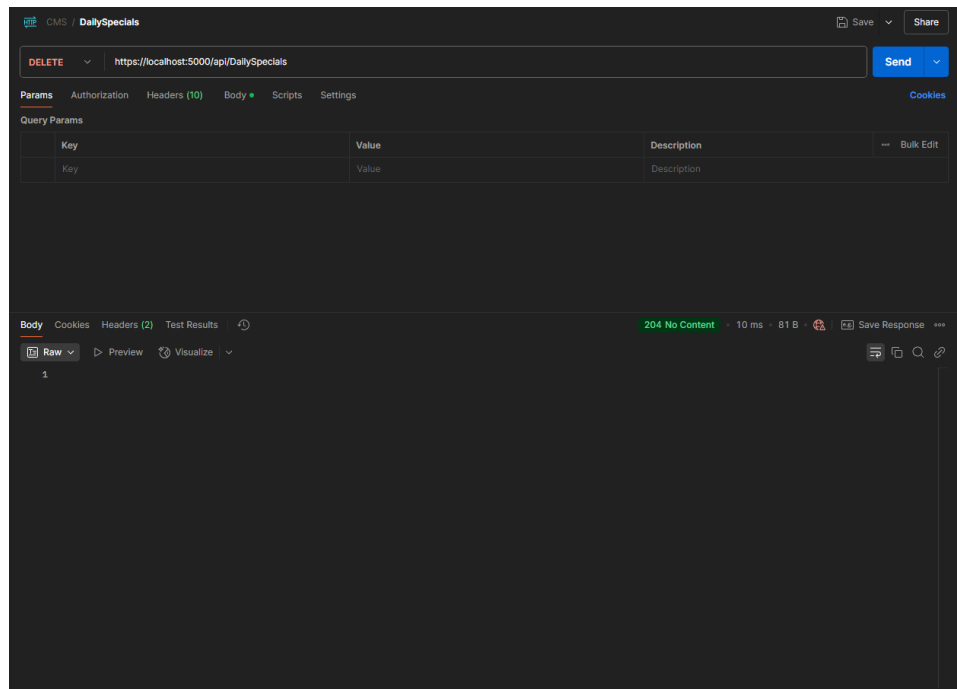
105. ábra – *DailySpecials* GET végpont visszatérési értéke

-  A POST /api/DailySpecials: Napi ajánlat készítése



106. ábra – *DailySpecials* POST végpont visszatérési értéke

-  ^A DELETE /api/DailySpecials: Napi ajánlat törlése (Összes rekord törlése)



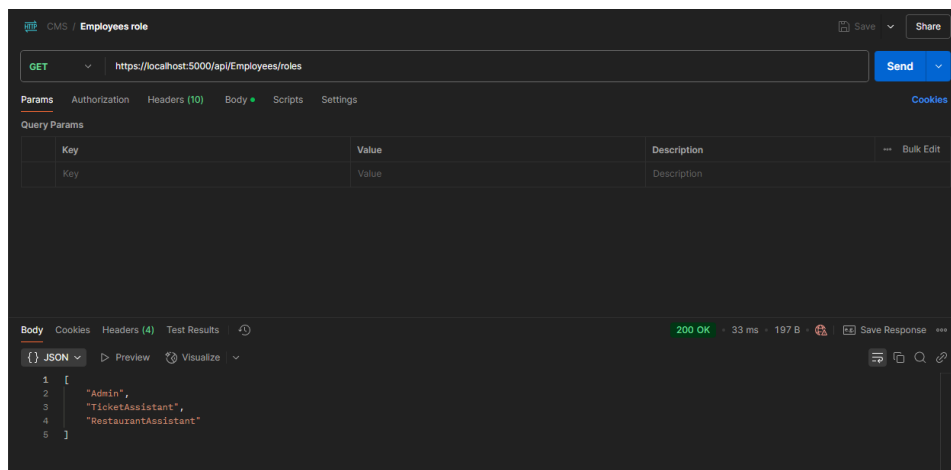
107. ábra – DailySpecials DELETE végpont visszatérési értéke

EmployeesController

Az *EmployeesController* dolgozókkal kapcsolatos adatokat közvetíti.

Elérhető végpontok:

-  A GET /api/Employees/roles: Az összes jogosultsági szint lekérdezése



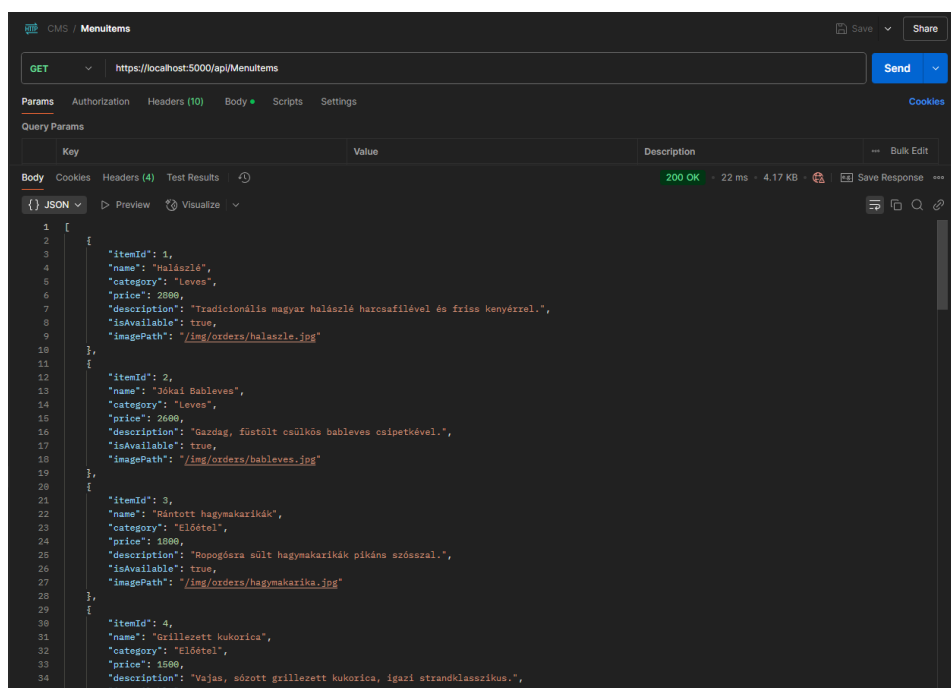
108. ábra – Employees/roles GET végpont visszatérési értéke

MenuItemsController

A *MenuItemsController* segítségével az éttermi kínálat adatait lehetséges módosítani.

Elérhető végpontok:

-  A&RA GET /api/MenuItems: Az összes éttermi cikk lekérdezése

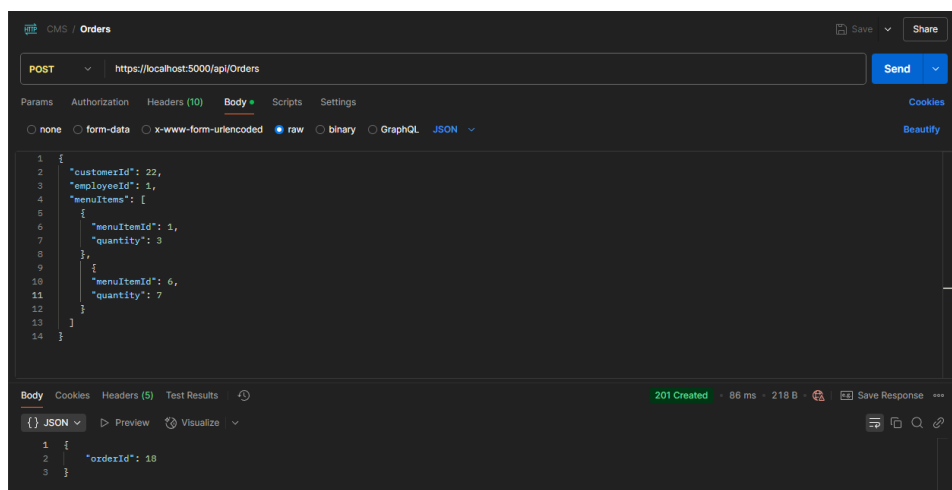


OrdersController

Az *OrdersController* az éttermi rendeléshez használatos végpontot tartalmaz.

Elérhető végpontok:

-  A&RA POST /api/Orders: Éttermi rendelés leadása



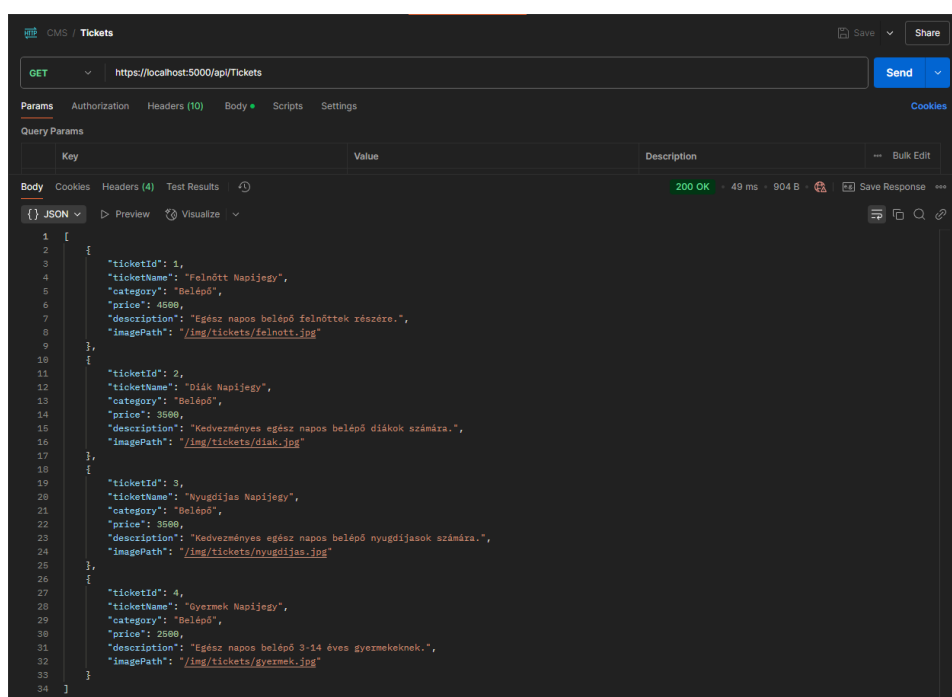
109. ábra – Orders POST végpont visszatérési értéke

TicketsController

A *TicketsController* a jegy értékesítési modulhoz

Elérhető végpontok:

-  A&TA GET /api/Tickets: Az összes jegy lekérése.



110. ábra – Tickets GET végpont visszatérési értéke

Általános Biztonsági Intézkedések és Validálás

Minden vezérlő külön figyelmet fordít az adatok validálására és az érvénytelen adatok elutasítására. Az Entity Framework ORM biztosítja a megfelelő adatbázis műveleteket és az adatintegritást. Az autentikáció alapvető biztonsági mechanizmusokkal rendelkezik, beleértve a JWT tokenek és a refresh tokenek kezelését.

A már említett DTO-k szerepe kulcsfontosságú az adatok áramlásában, mivel lehetővé teszik a bemeneti és kimeneti adatok validálását és tisztaságát, biztosítva ezzel a rendszer biztonságát és megbízhatóságát

3.3.9 Tesztelés

Az automata tesztelést *xUnit* segítségével hajtottuk végre. Az *xUnit* egy széles körben használt unit tesztelő keretrendszer a *.NET* alkalmazásokban. Célja, hogy egyszerűsíti és strukturáltan segíti a tesztelést, miközben számos hasznos funkcióval és lehetőséggel rendelkezik, amelyek a fejlesztők és tesztelők számára segítenek a kód minőségének és megbízhatóságának biztosításában. Az *xUnit* támogatja az aszinkron tesztelést, különböző típusú tesztelési életciklusokat, és az egység tesztelésének legjobb gyakorlatait, mint például a „*mockolás*”, az adat-injektálás, valamint az állapotkezelés és a különböző tesztesetek kezelése.

Miért érdemes *xUnit*-ot használni?

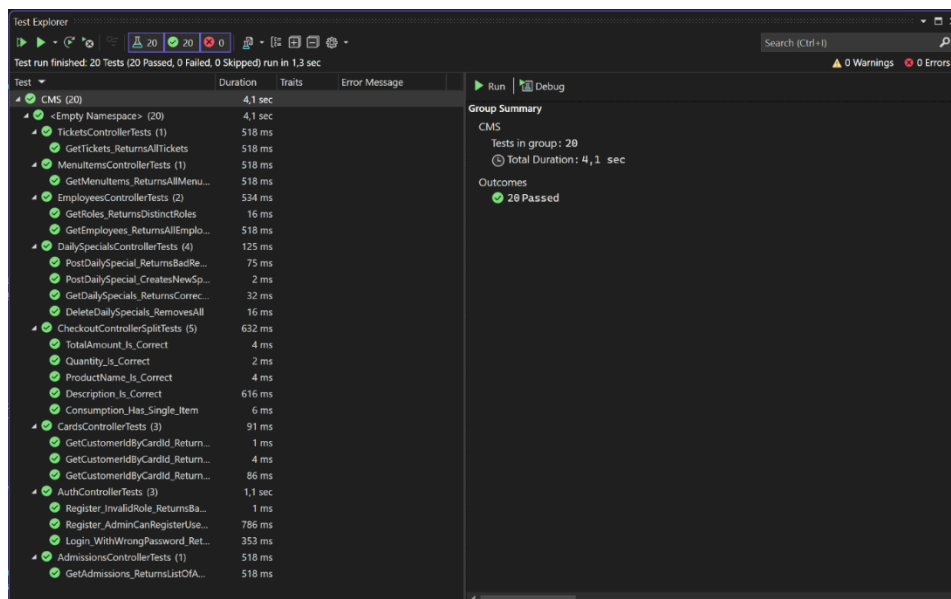
Könnyű telepítés és integráció: Az *xUnit* könnyen integrálható a *.NET* projektjeidbe, és különböző eszközökkel (például *Visual Studio*, *JetBrains Rider*, vagy parancssori eszközök) is használható.

Széles körű támogatás: Az *xUnit* támogatja az aszinkron tesztek és a különböző típusú függőségek kezelését, mint például *mock-olás*, *stubok*, illetve adatbázisok kezelését tesztelés során.

Rugalmasság és bővíthetőség: Az *xUnit* lehetőséget ad arra, hogy tesztekhez elő- és utófeltételeket (*setup* és *teardown*) definiáljunk, így biztosítva, hogy a tesztek tiszta és konzisztens környezetben fussanak.

Kód tisztaság és egyszerűség: Az *xUnit* egy egyszerű, de erőteljes tesztelési eszköz, amely megkönnyíti a tesztкод olvasását és karbantartását. Az *Assert* típusú hívások segítenek a tesztek pontosabb validálásában.

Párhuzamos futtatás: Az *xUnit* támogatja a tesztek párhuzamos futtatását, lehetővé téve a tesztelési idő csökkentését a különböző tesztkörnyezetekben



111. ábra – xUnit teszt eredménye

Az xUnit alapjai

Az *xUnit* tesztelés során a tesztelni kívánt kódot egy vagy több tesztmetódusba helyezzük el, amelyek különféle teszteseteket futtatnak. A tesztelés folyamata az alábbi elemeket tartalmazza:

Test Class: A tesztsz osztály egy vagy több tesztet tartalmaz. Az osztály neve általában a tesztelendő funkciót vagy komponenset tükrözi.

Test Method: A tesztmetódusok a tesztelt funkciók egy-egy tesztjeit képviselik. Minden metódus egy önálló tesztet tartalmaz.

Assert Statements: Az *Assert* osztály lehetővé teszi a tesztelni kívánt eredmények validálását. A leggyakoribb *Assert* típusok a *Assert.Equal()*, *Assert.NotNull()*, *Assert.True()*, és *Assert.Throws()*.

A tesztelés fázisai

Setup/Initialization (Kezdeti állapotok előkészítése): Minden teszt előtt szükség lehet a környezet előkészítésére, mint például *mock* objektumok, adatbázis inicializálása vagy tesztkörnyezet beállítása.

Test Execution (Teszt végrehajtása): A tesztelt metódus vagy funkció meghívása.

Assertion (Ellenőrzés): Az elvárt eredmények összehasonlítása a valós eredményekkel. Ha a két érték megegyezik, a teszt sikeres.

Teardown (Teszt utáni tisztítás): Az esetleges tesztadatok eltávolítása, a tesztkörnyezet visszaállítása, hogy ne zavarja a következő tesztet.

Backend tesztek futtatása a fejlesztői környezetben

A backend egységtesztek futtatása *Visual Studio* használatával az alábbi lépések mentén történik:

- A tesztelés megkezdéséhez először meg kell nyitni a megoldásfájlt (*CMS.sln*) a *Visual Studio* fejlesztői környezetben.
- A *Visual Studio* felső menüsorában található Test menüpont kiválasztását követően a *Run All Tests* opció segítségével indítható el az összes elérhető teszt automatikus futtatása.
- A tesztek lefutását követően a rendszer visszajelzést ad azok eredményéről, vizuálisan jelölve a sikeresen teljesített, illetve a hibásan lefutott teszteseteket.

Tesztpéldák

Az alábbiakban ismertetjük a mi esetünkben felmerülő főbb teszteket, tesztelési logikákat és azok célját.

A teszt az *AdmissionsController GetAdmissions* metódusát ellenőrzi. Először egy *in-memory* adatbázist hozunk létre, amelyben két belépési típust (*admission*) tárolunk. Ezután meghívjuk a *GetAdmissions* metódust, amelynek vissza kell adnia a két rekordot. A teszt során ellenőrizzük, hogy a metódus visszaadja a várt eredményt, azaz két belépési típus rekordot, és hogy az eredmény típusa és a rekordok száma megfelel-e az elvárásoknak.

```

12 [Fact]
13 public async Task GetAdmissions_ReturnsListOfAdmissions()
14 {
15     // Arrange: In-memory adatbázis létrehozása
16     var options = new DbContextOptionsBuilder<CMSContext>()
17         .UseInMemoryDatabase(databaseName: "TestDatabase")
18         .Options;
19
20     // Teszt adatok inicializálása
21     using (var context = new CMSContext(options))
22     {
23         context.Admissions.AddRange(
24             new Admission
25             {
26                 AdmissionId = 1,
27                 AdmissionName = "VIP Entry",
28                 Category = "VIP",
29                 Price = 500,
30                 Description = "Access to all areas",
31                 ImagePath = "/images/vip.jpg"
32             },
33             new Admission
34             {
35                 AdmissionId = 2,
36                 AdmissionName = "Standard Entry",
37                 Category = "Standard",
38                 Price = 100,
39                 Description = "Access to general areas",
40                 ImagePath = "/images/standard.jpg"
41             }
42         );
43         context.SaveChanges();
44
45         // Controller példányosítása
46         var controller = new AdmissionsController(context);
47
48         // Act: Meghívjuk a GetAdmissions metódust
49         var result = await controller.GetAdmissions();
50
51         // Assert: Eredmény ellenőrzése
52         var actionResult = Assert.IsType<ActionResult<IEnumerable<Admission>>>(result);
53         var model = Assert.IsType<IEnumerable<Admission>>(actionResult.Value);
54         Assert.Equal(2, model.Count());
55     }
56 }

```

112. ábra – xUnit teszt, *AdmissionController* számára

A teszt a *GetCustomerIdByCardId* metódus működését ellenőrzi, amely a megadott kártya ID alapján visszaadja az ügyfél azonosítóját. A teszt során egy aktív ügyfelet hozunk létre a tesztadatbázisban, akinek a kártya ID-ja "123ABC". Miután az ügyfél hozzáadása megtörtént, meghívjuk a metódust a kártya ID-val, és ellenőrizzük, hogy a válasz egy *OkObjectResult* típusú objektum. Ezen belül az ügyfél azonosítójának értékét is ellenőrizzük, amelynek meg kell egyeznie a létrehozott ügyfél ID-jával, azaz 1. A teszt célja annak biztosítása, hogy a metódus helyesen kezelje az aktív ügyfelek kártya ID-ját és visszaadja a megfelelő ügyfél azonosítót.

```
12 [Fact]
13 public async Task GetCustomerIdByCardId_ReturnsCustomerId_WhenCustomerExistsAndIsActive()
14 {
15     // Arrange: InMemory database
16     var options = new DbContextOptionsBuilder<CMSContext>()
17         .UseInMemoryDatabase("TestDatabase")
18         .Options;
19
20     using (var context = new CMSContext(options))
21     {
22         var customer = new Customer
23         {
24             CustomerId = 1,
25             CardId = "123ABC",
26             Name = "John Doe",
27             IsActive = true
28         };
29         context.Customers.Add(customer);
30         context.SaveChanges();
31
32         var controller = new CardsController(context);
33
34         // Act: Method invocation
35         var result = await controller.GetCustomerIdByCardId("123ABC");
36
37         // Assert: Előfeltétel
38         var okResult = Assert.IsType<OkObjectResult>(result.Result);
39         dynamic value = okResult.Value;
40         Assert.Equal(1, value.CustomerId);
41     }
42 }
```

113. ábra – xUnit teszt *CheckoutController* számára

Ez a teszt a *CheckoutController* osztály *GetCustomerConsumption* metódusának működését ellenőrzi, amely visszaadja egy ügyfél összes rendelését. A tesztek biztosítják, hogy a visszakapott adatok pontosak és helyesek, mint például az összeg, a termékek száma, a termékek neve, leírása és mennyisége.

A tesztek főbb pontjai:

- TotalAmount_Is_Correct: Ellenőrzi, hogy az összeg helyesen van kiszámítva ($3 * 12 = 36$).
- Consumption_Has_Single_Item: Ellenőrzi, hogy egyetlen termék szerepel az elfogyasztott listán.
- ProductName_Is_Correct: Ellenőrzi, hogy a termék neve helyes ("Burger").
- Description_Is_Correct: Ellenőrzi, hogy a termék leírása helyes ("Beef burger").
- Quantity_Is_Correct: Ellenőrzi, hogy a termék mennyisége helyes (3).

```
64 [Fact]
65 [Order(1)]
66 public async Task TotalAmount_Is_Correct()
67 {
68     var dbResult = await ExecuteTestController();
69     dynamic value = dbResult.Value;
70     Assert.Equal(36, (int)value.TotalAmount); // 3 * 12
71 }
72
73 [Fact]
74 [Order(2)]
75 public async Task Consumption_Has_Single_Item()
76 {
77     var dbResult = await ExecuteTestController();
78     dynamic value = dbResult.Value;
79     Assert.Single(value.Consumption);
80 }
81
82 [Fact]
83 [Order(3)]
84 public async Task ProductName_Is_Correct()
85 {
86     var dbResult = await ExecuteTestController();
87     dynamic value = dbResult.Value;
88     Assert.Equal("Burger", (string)value.Consumption[0].ProductName);
89 }
90
91 [Fact]
92 [Order(4)]
93 public async Task Description_Is_Correct()
94 {
95     var dbResult = await ExecuteTestController();
96     dynamic value = dbResult.Value;
97     Assert.Equal("Beef burger", (string)value.Consumption[0].Description);
98 }
99
100 [Fact]
101 [Order(5)]
102 public async Task Quantity_Is_Correct()
103 {
104     var dbResult = await ExecuteTestController();
105     dynamic value = dbResult.Value;
106     Assert.Equal(3, (int)value.Consumption[0].Quantity);
107 }
```

114. ábra – xUnit teszt, Különböző ellenőrzések a rendeléshez

PostDailySpecial_ReturnsBadRequest_WhenItemMissing: Ez a teszt azt ellenőrzi, hogy a rendszer helyesen reagál-e, ha a napi étkezési ajánlathoz tartozó egyik étel nem található a menüben. Ha például a *CoffeeName* értéke "Nem létezik", amely nem szerepel az adatbázisban, akkor a rendszer *BadRequest* választ ad, és a hibaüzenet a következő kell legyen: "One or more menu items not found." Ezzel a teszttel biztosítható, hogy az alkalmazás megfelelő hibaüzenetet adjon, ha az étkezési ajánlatok között egy étel neve nem

egyezik meg a meglévő elemekkel.

GetDailySpecials_ReturnsCorrectNames: Ez a teszt azt vizsgálja, hogy a napi étkezési ajánlatok lekérésekor a válaszban a megfelelő ételnevek szerepelnek-e. A tesztben létrehozott napi étkezési ajánlat például a "Halászlé" (leves) és a "Strand Burger" (hamburger), és a teszt azt ellenőrzi, hogy ezek a nevek pontosan megjelennek-e a válaszban. A teszt sikeres lefutása biztosítja, hogy a napi étkezési ajánlatok helyesen kerülnek visszaadásra a felhasználói kérésre.

```

90 [Fact]
91 [System.Diagnostics.CodeAnalysis.SuppressMessage("Performance", "CA1822:Mark members with SuppressMessageAttribute", Justification = "Test method")]
92 public async Task PostDailySpecial_ReturnsBadRequest_WhenItemMissing()
93 {
94     using var context = CreateTestContext();
95     var controller = new DailySpecialController(context);
96
97     var dto = new DailySpecialPostDto
98     {
99         SoupName = "Halászlé",
100         AppetizerName = "Mentés hagymakorika",
101         MainCourseName = "Langos tejföllel és sajttal",
102         HamburgerName = "Strand Burger",
103         PizzaName = "Hawaii Pizza",
104         DessertName = "Mentés Palacsinta",
105         DrinkName = "Mentés limonádé",
106         CoffeeName = "Nem létezik"
107     };
108
109     var actionResult = await controller.PostDailySpecial(dto);
110     var result = actionResult.Result;
111
112     var badRequest = Assert.IsType<BadRequestResult>(result);
113     Assert.Equal("One or more menu items not found.", badRequest.Value);
114 }
115
116 [Fact]
117 [System.Diagnostics.CodeAnalysis.SuppressMessage("Performance", "CA1822:Mark members with SuppressMessageAttribute", Justification = "Test method")]
118 public async Task GetDailySpecials_ReturnsCorrectNames()
119 {
120     using var context = CreateTestContext();
121
122     context.DailySpecials.Add(new DailySpecial
123     {
124         SoupId = 1,
125         AppetizerId = 2,
126         MainCourseId = 3,
127         HamburgerId = 4,
128         PizzaId = 5,
129         DessertId = 6,
130         DrinkId = 7,
131         CoffeeId = 8
132     });
133     await context.SaveChangesAsync();
134
135     var controller = new DailySpecialController(context);
136     var result = await controller.GetDailySpecials();
137
138     var actionResult = Assert.IsType<ActionResult<IEnumerable<DailySpecialGetDto>>>(result);
139     var specials = Assert.IsAssignableTo<IEnumerable<DailySpecialGetDto>>(actionResult.Value);
140
141     var special = specials.First();
142     Assert.Equal("Halászlé", special.SoupName);
143     Assert.Equal("Strand Burger", special.HamburgerName);
144 }

```

115. ábra – xUnit teszt *DailySpecialController* számára

A **GetEmployees_ReturnsAllEmployees** teszt azt ellenőrzi, hogy a *GetEmployees* metódus helyesen visszaadja-e az összes alkalmazottat az adatbázisból.

- **Arrange (Előkészítés):** Az *InMemory* adatbázis használatával létrehoz egy tesztadatbázist és két alkalmazottat ad hozzá (John Doe és Jane Doe), mindkettő különböző szerepkörökkel és jelszóval.
- **Act (Művelet):** A teszt meghívja a *GetEmployees* metódust, amely az összes alkalmazottat lekéri.
- **Assert (Ellenőrzés):** A teszt ellenőrzi, hogy a lekért alkalmazottak száma megegyezik-e a várttal (kettő alkalmazott), és hogy a két alkalmazott neve (John és Jane) megfelel-e az elvártnak.

A teszt célja annak biztosítása, hogy a *GetEmployees* metódus minden alkalmazottat helyesen visszaadjon az adatbázisból.

```
12 [Fact]
13 #pragma warning disable
14 public async Task GetEmployees_ReturnsAllEmployees()
15 {
16     // Arrange: InMemory adatbázis létrehozása
17     var options = new DbContextOptionsBuilder<CMSContext>()
18         .UseInMemoryDatabase("TestDatabase")
19         .Options;
20
21     using (var context = new CMSContext(options))
22     {
23         // Tesztadatok inicializálása
24         context.Employees.AddRange(
25             new Employee { EmployeeId = 1, FirstName = "John", LastName = "Doe", Role = "Manager", Username = "johndoe", PasswordHash = "hashedpassword1" },
26             new Employee { EmployeeId = 2, FirstName = "Jane", LastName = "Doe", Role = "Cashier", Username = "janedoe", PasswordHash = "hashedpassword2" }
27         );
28         context.SaveChanges();
29
30         var controller = new EmployeesController(context);
31
32         // Act: Metódus meghívása
33         var result = await controller.GetEmployees();
34
35         // Assert: Eredmények ellenőrzése
36         var actionResult = Assert.IsType<ActionResult<IEnumerable<Employee>>>(result);
37         var employees = Assert.IsType<List<Employee>>(actionResult.Value);
38         Assert.Equal(2, employees.Count);
39         Assert.Equal("John", employees[0].FirstName);
40         Assert.Equal("Jane", employees[1].FirstName);
41     }
42 }
```

116. ábra – xUnit teszt, dolgozók helyes visszatérésének teszteléséhez

A **GetMenuItems_ReturnsAllMenuItems** teszt célja annak ellenőrzése, hogy a *GetMenuItems* metódus helyesen adja-e vissza az összes menüpontot az adatbázisból.

- **Arrange (Előkészítés):** Létrehoz egy *InMemory* adatbázist, amelybe két menüpontot ad hozzá (Margherita Pizza és Cheeseburger). Mindezek tartalmazzák az alapvető információkat, mint a név, kategória, ár, leírás, elérhetőség és kép.
- **Act (Művelet):** Meghívja a *GetMenuItems* metódust, amely lekéri az összes menüpontot az adatbázisból.
- **Assert (Ellenőrzés):** A teszt ellenőrzi, hogy a válasz tartalmazza-e a két menüpontot, és hogy azok nevei pontosan egyeznek-e a várttal (Margherita Pizza és Cheeseburger).

A teszt célja annak biztosítása, hogy a *GetMenuItems* metódus az adatbázisban tárolt összes menüpontot megfelelően visszaadja.

```
12 [Fact]
13 public async Task GetMenuItems_ReturnsAllMenuItems()
14 {
15     // Arrange: InMemory adatbázis létrehozása
16     var options = new DbContextOptionsBuilder<CMContext>()
17         .UseInMemoryDatabase("TestDatabase")
18         .Options;
19
20     using (var context = new CMContext(options))
21     {
22         // Tesztadatok inicializálása
23         context.MenuItems.AddRange(
24             new MenuItem
25             {
26                 ItemId = 1,
27                 Name = "Margherita Pizza",
28                 Category = "Pizza",
29                 Price = 15,
30                 Description = "Classic Italian Pizza",
31                 IsAvailable = true,
32                 ImagePath = "/images/margherita.jpg"
33             },
34             new MenuItem
35             {
36                 ItemId = 2,
37                 Name = "Cheeseburger",
38                 Category = "Burger",
39                 Price = 10,
40                 Description = "Delicious Cheeseburger",
41                 IsAvailable = true,
42                 ImagePath = "/images/cheeseburger.jpg"
43             }
44         );
45         context.SaveChanges();
46
47         var controller = new MenuItemsController(context);
48
49         // Act: Metódus hívása
50         var result = await controller.GetMenuItems();
51
52         // Assert: Eredmények ellenőrzése
53         var actionResult = Assert.IsType<ActionResult<IEnumerable<MenuItem>>>(result);
54         var menuItems = Assert.IsType<IEnumerable<MenuItem>>(actionResult.Value);
55
56         Assert.Equal(2, menuItems.Count());
57         Assert.Equal("Margherita Pizza", menuItems[0].Name);
58         Assert.Equal("Cheeseburger", menuItems[1].Name);
59     }
60 }
```

117. ábra – xUnit teszt, a menüpontok helyes visszatérésének teszteléséhez

A *AuthControllerTests* osztály a *AuthController* működését teszteli, különös figyelmet fordítva a felhasználói regisztrációra. A kódban szereplő tesztek az adminisztrátori felhasználó által történő regisztráció sikerességét és az érvénytelen szerepkörök kezelését vizsgálják. Az *Register_AdminCanRegisterUser_ReturnsOk* teszt ellenőrzi, hogy egy admin felhasználó sikeresen regisztrálhat-e új felhasználót, míg a *Register_InvalidRole_ReturnsBadRequest* teszt azt biztosítja, hogy a rendszer megfelelően kezelje az érvénytelen szerepköröket a regisztrációs folyamatban. Emellett más tesztek is találhatók a kódban, amelyek további érvényesítéseket és válaszokat vizsgálnak.

```
[Fact]
public async Task Register_AdminCanRegisterUser_ReturnsOk()
{
    var context = CreateTestContext();
    var config = CreateTestConfiguration();
    var controller = new AuthController(context, config);
    controller.ControllerContext = new ControllerContext
    {
        HttpContext = new DefaultHttpContext { User = CreateAdminUser() }
    };
    var dto = new RegisterDto
    {
        Username = "newuser",
        Password = "password",
        FirstName = "John",
        LastName = "Doe",
        Role = "Admin"
    };
    var result = await controller.Register(dto);
    var okResult = Assert.IsType<OkObjectResult>(result);
    Assert.Equal("User registered successfully", ((dynamic)okResult.Value).message);
}

[Fact]
public async Task Register_InvalidRole_ReturnsBadRequest()
{
    var context = CreateTestContext();
    var config = CreateTestConfiguration();
    var controller = new AuthController(context, config);
    controller.ControllerContext = new ControllerContext
    {
        HttpContext = new DefaultHttpContext { User = CreateAdminUser() }
    };
    var dto = new RegisterDto
    {
        Username = "test",
        Password = "pass",
        FirstName = "test",
        LastName = "test",
        Role = "InvalidRole"
    };
    var result = await controller.Register(dto);
    var badRequest = Assert.IsType<BadRequestObjectResult>(result);
    Assert.Equal("Invalid role.", badRequest.Value);
}
```

118. ábra – xUnit teszt, új felhasználó sikeres létrehozásának teszteléséhez

A `TicketsControllerTests` osztály a `TicketsController` működését teszteli, különös figyelmet fordítva a jegyek listázására. A teszt egy in-memory adatbázist használ, amely tartalmazza a tesztadatokat: két különböző típusú jegyet (VIP és Regular). A `GetTickets_ReturnsAllTickets` teszt ellenőrzi, hogy a `GetTickets` metódus megfelelően visszaadja-e a jegyek listáját. A teszt során ellenőrzi, hogy a két jegy (VIP Ticket és Regular Ticket) helyesen szerepel-e a válaszban, és hogy a visszaadott jegyek száma megfelel-e a várt értéknek.

```

17 [Fact]
18 public async Task GetTickets_ReturnsAllTickets()
19 {
20     // Arrange: In-memory adatbázis létrehozása
21     var options = new DbContextOptionsBuilder<CRMContext>()
22         .UseInMemoryDatabase("testdbase1")
23         .Options;
24     using (var context = new CRMContext(options))
25     {
26         // Tesztadatok inicializálása
27         context.Tickets.AddRange(
28             new Ticket
29             {
30                 TicketId = 1,
31                 TicketName = "VIP Ticket",
32                 Category = "VIP",
33                 Price = 200,
34                 Description = "Access to all VIP areas",
35                 ImagePath = "/images/vip-ticket.jpg"
36             },
37             new Ticket
38             {
39                 TicketId = 2,
40                 TicketName = "Regular Ticket",
41                 Category = "Standard",
42                 Price = 100,
43                 Description = "General admission ticket",
44                 ImagePath = "/images/regular-ticket.jpg"
45             }
46         );
47         context.SaveChanges();
48     }
49     var controller = new TicketsController(context);
50     // Act: Metódus hívása
51     var result = await controller.GetTickets();
52     // Assert: Eredmények ellenőrzése
53     var actionResult = Assert.IsType<ActionResult<IEnumerable<Ticket>>>(result);
54     var tickets = Assert.IsType<List<Ticket>>(actionResult.Value);
55     Assert.Equal(2, tickets.Count());
56     Assert.Equal("VIP Ticket", tickets[0].TicketName);
57     Assert.Equal("Regular Ticket", tickets[1].TicketName);
58 }

```

119. ábra – xUnit teszt, a jegyek helyes visszatérésének teszteléséhez

Tesztelési stratégiák és jó gyakorlatok

Különböző tesztesetek tesztelése: Az egyes metódusok különböző eseteket és hibákat is tesztelnek, biztosítva, hogy minden lehetséges forgatókönyv megfelelően legyen kezelve.

Mock-olás használata: A tesztek során bizonyos adatbázis-interakciók (például `FirstOrDefaultAsync`, `ToListAsync`) mock-olása történt, annak érdekében, hogy az adatbázis valós használata elkerülhető legyen, és a tesztelési folyamat gyorsabban végrehajthatóvá váljon.

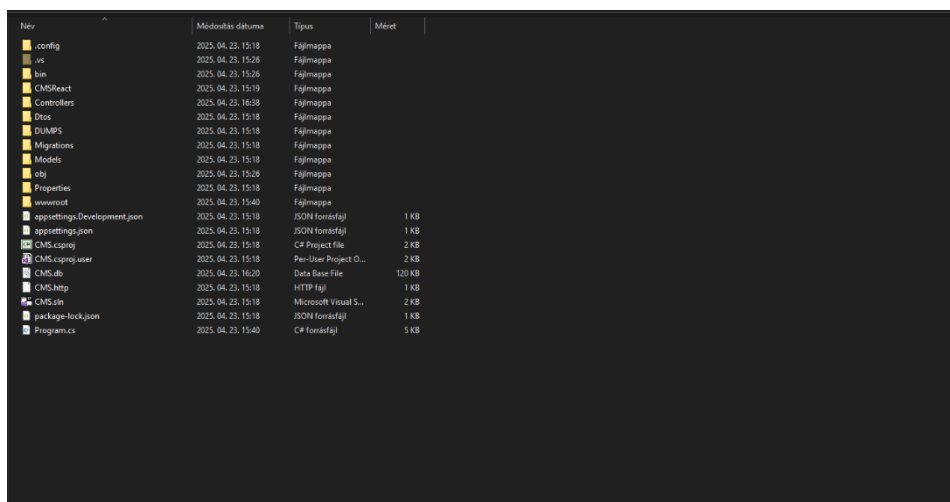
Egységtesztelés: Az egyes metódusok elszigetelten kerültek tesztelésre, hogy a hibák és anomáliák pontosan azonosíthatók legyenek.

Összegzés

Az xUnit egy rugalmas tesztelési keretrendszer, amely a .NET alkalmazások minőségének biztosítását szolgálja. A tesztek célja az ügyféladatokat validálása, a hibakezelés ellenőrzése, valamint a rendszer megfelelő működésének vizsgálata különböző bemenetek esetén. A struktúra támogatja a karbantarthatóságot és a bővíthetőséget.

3.4 Program futtatása fejlesztői módban

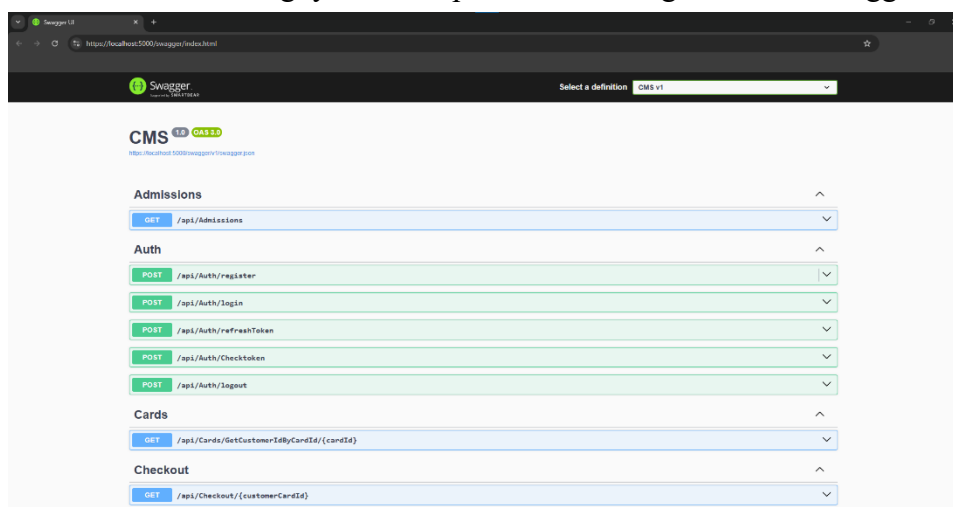
Az alkalmazás két fő komponensből áll: egy *.NET*-alapú backendből és egy React-alapú frontendből, amely az *npm* csomagkezelőt használja. Az alábbi lépések végrehajtásával a teljes rendszer elindítható és tesztelhető lokális környezetben.



120. ábra - CMS projekt mappa

3.4.1 Backend (ASP.NET) indítása

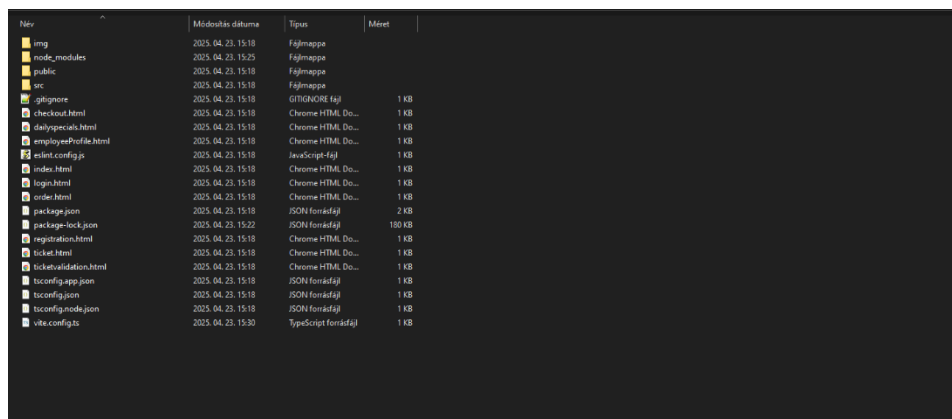
- 1) A projekt gyökérkönyvtárában található *CMS* nevű mappában első lépésként telepíteni szükséges a rendszer működéséhez elengedhetetlen függőségeket. Ez az alábbi parancs futtatásával végezhető el: *dotnet restore*
- 2) A függőségek telepítése után a *CMS.sln* megnyitásával a projekt futtatható. A rendszer a *localhost:5000* címen érhető el, mivel a backend alkalmazás dedikáltan az 5000-es portot használja.
- 3) Sikeres futtatás után, megnyílik az alapértelmezett böngészőben a Swagger.



121. ábra - Backend üzemel fejlesztői módban

3.4.2 Frontend (React + Vite) indítása

A felhasználói felület a projekt CMSReact mappájában található. Az alábbi lépések végrehajtásával a frontend komponens helyi fejlesztői környezetben elindítható:



Név	Módosítás dátuma	Típus	Méret
img	2025. 04. 23. 15:18	Fájlmappa	
node_modules	2025. 04. 23. 15:25	Fájlmappa	
public	2025. 04. 23. 15:18	Fájlmappa	
src	2025. 04. 23. 15:18	Fájlmappa	
gitignore	2025. 04. 23. 15:18	GitIGNORE fájl	1 KB
checkout.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
dailySpecials.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
employeeProfile.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
eslint.config.js	2025. 04. 23. 15:18	JavaScript-fájl	1 KB
index.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
login.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
order.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
package.json	2025. 04. 23. 15:18	JSON forrádfájl	2 KB
package-lock.json	2025. 04. 23. 15:22	JSON forrádfájl	180 KB
registration.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
ticket.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
ticketValidation.html	2025. 04. 23. 15:18	Chrome HTML, Do...	1 KB
tsconfig.app.json	2025. 04. 23. 15:18	JSON forrádfájl	1 KB
tsconfig.json	2025. 04. 23. 15:18	JSON forrádfájl	1 KB
tsconfig.node.json	2025. 04. 23. 15:18	JSON forrádfájl	1 KB
vite.config.ts	2025. 04. 23. 15:30	TypeScript forrádfájl	1 KB

122. ábra - CMSReact mappa tartalma

1) A CMSReact mappa megnyitása

- Első lépésként navigálni szükséges a CMSReact könyvtárba terminál segítségével, amely tartalmazza a frontend projekt fájljait.

2) Parancssor megnyitása

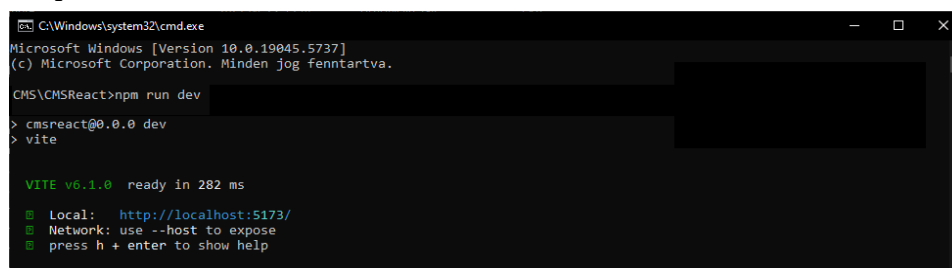
- A mappán belül parancssoros környezetet (pl. terminált vagy Visual Studio Code beépített terminálját) kell indítani.

3) Függőségek telepítése (szükség esetén)

- A mennyiben a *node_modules* mappa nem található a projektben a szükséges csomagokat az alábbi parancs segítségével kell telepíteni: *npm install*. Ez letölti és telepíti a projekt működéséhez szükséges összes Node.js függőséget.

4) A fejlesztői szerver elindítása

- Végül, a frontend alkalmazás indítása a következő parancs segítségével történik:
npm run dev



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. Minden jog fenntartva.

CMS\CMSReact>npm run dev

> cmsreact@0.0.0 dev
> vite

VITE v6.1.0 ready in 282 ms
  Local: http://localhost:5173/
  Network: use --host to expose
  press h + enter to show help
```

123. ábra - Frontend Vite szerveren való futtatása

4 Felhasználói dokumentáció

4.1 A program általános specifikációja, célja

A rendszerünk célja, hogy a fürdőkomplexumok adminisztrációs folyamatait egyszerűsítse és hatékonyabbá tegye. A program a fürdő vendégeinek adatait és fogyasztásait valós időben kezeli és tárolja, így lehetőséget biztosít a gyors és pontos nyilvántartásra. Az alkalmazás támogatja a vendégek regisztrálását egyedi azonosító segítségével, érkezési és távozási idejük nyomon követését, valamint az általuk igénybe vett szolgáltatások és fogyasztások részletes rögzítését. A rendszer képes a különböző szolgáltatások (pl. belépőjegy vásárlás, wellness kezelések, étkezés, egyéb kiegészítő szolgáltatások) díjainak valós időben történő rögzítésére és számlázására. Valós időben ad visszajelzést a felhasználó jogosultságairól és fogyasztásainak állapotáról. A komplexum dolgozói számára egy egyszerű, átlátható kezelőfelületet biztosít, amely lehetővé teszi az adminisztrációs feladatok gyors és pontos elvégzését.

4.2 Rendszerkövetelmények

4.2.1 Szoftver követelmények

A weboldal zavartalan működéséhez korszerű operációs rendszerekre és naprakész böngészőkre van szükség. Az oldal teljes funkcionalitása biztosított asztali számítógépeken, laptopokon, valamint mobil eszközökön is (Android és iOS).

Támogatott operációs rendszerek

- Windows 7 vagy újabb (ajánlott: Windows 11)
- macOS Mojave (10.14.6) vagy újabb (ajánlott: macOS Sonoma 14.3.1)
- Android 10 vagy újabb
- iOS 13 vagy újabb

Támogatott böngészőmotorok

- **Chromium**
 - *Google Chrome*
 - *Microsoft Edge*
 - *Brave*
 - *Opera*
 - *Safari*

- **Gecko**
 - *Mozilla Firefox*

Ajánlott böngészők a legjobb felhasználói élményhez

- *Google Chrome* (asztali és mobil verzió)
- *Safari* (Apple eszközökön)

4.2.2 Minimális kliensoldai hardver követelmény

A fejlesztett rendszer egy platformfüggetlen webalkalmazás, amely internetböngészőn keresztül érhető el. Ennek köszönhetően a kliensoldalon nincs szükség specifikus operációs rendszerre vagy külön telepített szoftverkörnyezetre. Bár a rendszer mobil eszközökről is elérhető, a megbízható működés és a kényelmes felhasználói élmény érdekében számítógép használata javasolt. Az alábbi minimális hardver- és szoftverkörnyezet ajánlott a stabil és zökkenőmentes működéshez:

- **Processzor:** Intel Core i3-7100 vagy ezzel egyenértékű teljesítményű központi feldolgozóegység, amely biztosítja a modern webböngészők megfelelő működését.
- **Háttértár:** Legalább 64 GB kapacitású merevlemez (HDD) vagy szilárdtest-meghajtó (SSD). Az SSD előnyösebb, mivel gyorsabb rendszerbetöltést és adatkezelést tesz lehetővé.
- **Memória (RAM):** Minimum 8 GB DDR3 vagy újabb típusú RAM a többfeladatos működés és a böngésző alapú alkalmazások zökkenőmentes futtatásához.
- **Operációs rendszer:** A rendszer platformfüggetlen, azonban a kliensoldali számítógépen javasolt egy naprakészen tartott operációs rendszer használata például *Windows 10 22H2*, *macOS Catalina*, valamint *Linux*-disztribúciók közül a *Debian* vagy az *Ubuntu* –, amelyek támogatják a modern böngészők futtatását. Hordozható eszközök, például okostelefonok és táblagépek esetén a rendszer megfelelő működéséhez legalább *iOS 15* vagy *Android 8* operációs rendszer szükséges, szintén frissített állapotban.
- **Képernyő:** A felhasználói élmény javítása érdekében legalább egy 24 colos, 16:9 képarányú monitor használata javasolt, amely megfelelő felületet biztosít az alkalmazás elemeinek áttekinthető megjelenítéséhez.

4.3 Program telepítése szerverre / kiszolgáló számítógépre

Az adatok kiszolgálását kezelő rendszer egy .exe fájl segítségével indul el, melynek adatai lokális .db (SQLite) kiterjesztésű fájlban tárolódnak. A kiszolgáló rendelkezzen aktív szélessávú interneteléréssel, valamint szükséges a megfelelő hálózati topológia, kliensekkel való kapcsolat. A megfelelő port engedélyezése szükséges, a távoli elérés érdekében. A szerver oldal kizárólag Windows operációs rendszeren fut, javasoljuk operációs rendszernek az alábbi Windows szerver konfigurációt:

Windows Server 2022 vagy újabb, Standard vagy Datacenter

- Hosszú távú támogatás (LTS)
- Továbbfejlesztett biztonsági funkciók (Secured-core)

4.3.1 Rendszer futtatása (.exe esetén build használata)

A rendszer használatához nincs szükség külön telepítésre vagy fejlesztői környezetre. A teljes alkalmazás egyetlen .exe fájlba lett integrálva, amely tartalmazza a szerveroldali (backend) és kliensoldali (frontend) elemeket is.

Indítás lépései:

1) A futtatható fájl megkeresése

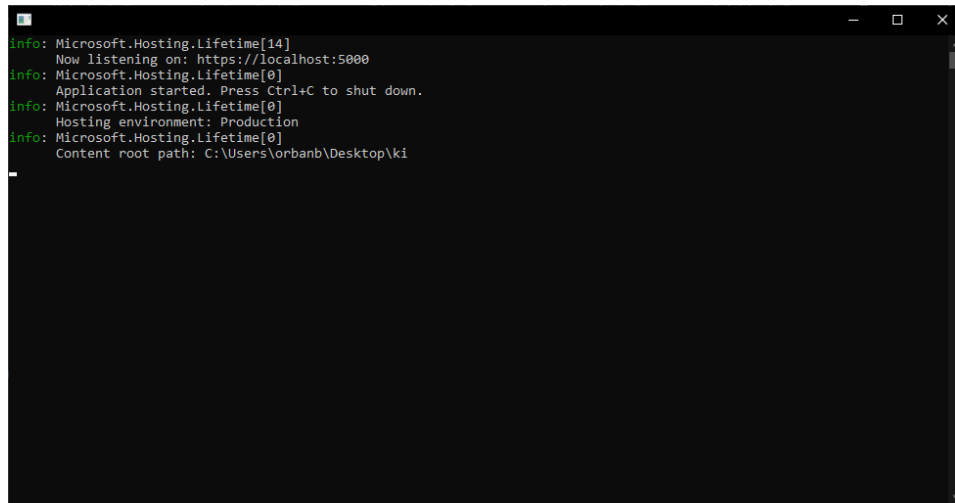
A rendszer indításához először meg kell nyitni azt a könyvtárat, amely tartalmazza a CMS.exe kiterjesztésű fájlt.

Név	Módosítás dátuma	Típus	Méret
tr	2025. 04. 23. 20:55	Fájlmappa	
uk	2025. 04. 23. 20:55	Fájlmappa	
uz-Cyrl-UZ	2025. 04. 23. 20:55	Fájlmappa	
uz-Latn-UZ	2025. 04. 23. 20:55	Fájlmappa	
vi	2025. 04. 23. 20:55	Fájlmappa	
wwwroot	2025. 04. 23. 20:55	Fájlmappa	
zh-CN	2025. 04. 23. 20:55	Fájlmappa	
zh-Hans	2025. 04. 23. 20:55	Fájlmappa	
zh-Hant	2025. 04. 23. 20:55	Fájlmappa	
appsettings.Development.json	2025. 04. 23. 15:18	JSON forrásfájl	1 KB
appsettings.json	2025. 04. 23. 15:18	JSON forrásfájl	1 KB
appsettings.production.json	2025. 04. 23. 20:55	JSON forrásfájl	1 KB
Azure.Core.dll	2024. 02. 26. 16:24	Alkalmazáskiterjesztés	392 KB
Azure.Identity.dll	2024. 06. 10. 18:43	Alkalmazáskiterjesztés	335 KB
BCrypt.Net-Next.dll	2022. 03. 03. 17:10	Alkalmazáskiterjesztés	23 KB
CMS.db	2025. 04. 23. 21:04	Data Base File	120 KB
CMS.deps.json	2025. 04. 23. 20:55	JSON forrásfájl	130 KB
CMS.dll	2025. 04. 23. 20:55	Alkalmazáskiterjesztés	138 KB
CMS.exe	2025. 04. 23. 20:55	Alkalmazás	136 KB
CMS.pdb	2025. 04. 23. 20:55	Program Debug D...	51 KB
CMS.runtimeconfig.json	2025. 04. 23. 20:55	JSON forrásfájl	1 KB
CMS.staticwebassets.endpoints.json	2025. 04. 23. 20:55	JSON forrásfájl	180 KB
dotnet-aspnet-codegenerator-design.dll	2024. 10. 31. 3:18	Alkalmazáskiterjesztés	54 KB
Humanizer.dll	2022. 01. 29. 16:04	Alkalmazáskiterjesztés	348 KB
Microsoft.AspNetCore.Authentication.Jwt...	2025. 01. 17. 7:41	Alkalmazáskiterjesztés	51 KB
Microsoft.AspNetCore.Razor.Language.dll	2023. 10. 19. 6:55	Alkalmazáskiterjesztés	987 KB
Microsoft.Bcl.AsyncInterfaces.dll	2022. 10. 18. 18:14	Alkalmazáskiterjesztés	19 KB
Microsoft.Build.dll	2023. 10. 19. 22:53	Alkalmazáskiterjesztés	1 875 KB
Microsoft.Build.Framework.dll	2023. 10. 19. 22:53	Alkalmazáskiterjesztés	299 KB
Microsoft.CodeAnalysis.AnalyzerUtilities...	2020. 06. 06. 5:37	Alkalmazáskiterjesztés	531 KB
Microsoft.CodeAnalysis.CSharp.dll	2023. 11. 08. 18:56	Alkalmazáskiterjesztés	6 350 KB
Microsoft.CodeAnalysis.CSharp.Features...	2023. 11. 08. 18:58	Alkalmazáskiterjesztés	1 967 KB
Microsoft.CodeAnalysis.CSharp.Workspa...	2023. 11. 08. 18:56	Alkalmazáskiterjesztés	863 KB

124. ábra - CMS.exe, a futtatandó fájl a projekthez

2) Futtatás

A fájl futtatása, mely automatikusan elindítja a háttérfolyamatokat. És a megjelenő parancssorba látszik, hogy a szolgáltatás fut az 5000-es porton

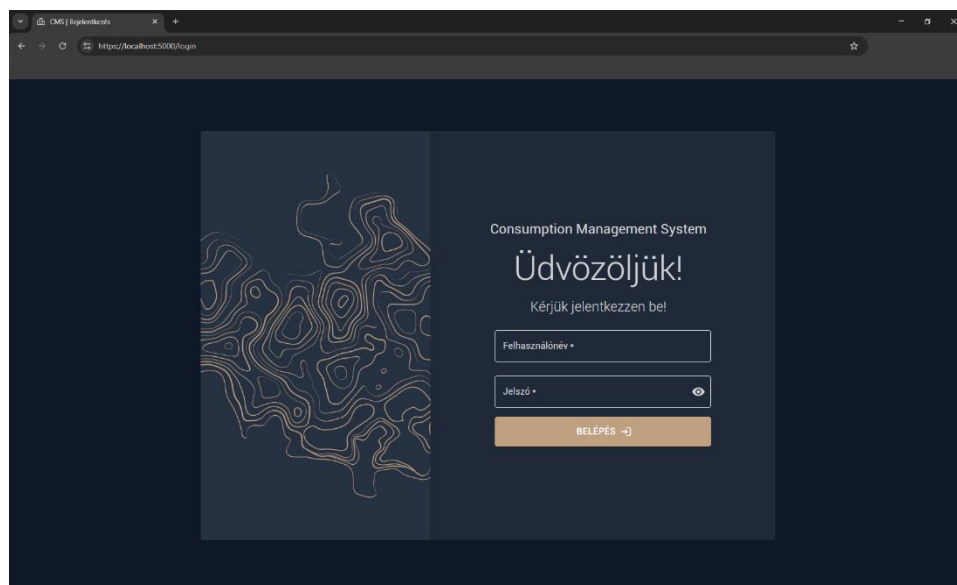


```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\orbanb\Desktop\ki
```

125. ábra - CMS.exe működés közben

3) Elérés böngészőn keresztül

A <https://localhost:5000> cím felkeresése után, már is elérhető a projekt fejlesztői eszközök nélkül



126. ábra - Böngészőben futó alkalmazás

4.3.2 Hardverkövetelmény

- **Processzor:** Legalább 8 magos, 2.6 GHz+ (pl. Intel Xeon Gold vagy AMD EPYC)
- **Memória:** Minimum 32 GB RAM (ajánlott: 64 GB vagy több, ha cache-elés vagy microservice szintű elosztás van)
- **Tárhely:**
 - SSD tárhely, legalább 512 GB (ajánlott: NVMe)
 - RAID 10 vagy hasonló redundáns megoldás ajánlott

A futtatható fájl kezeli a backend és a frontend szolgáltatást, így nem szükséges külön webservert üzemeltetése a frontend megjelenítéséhez.

A szerver futtatásához szükség lesz a .Net 8 vagy újabb keretrendszerre és azok függőségeinek a telepítésére.

4.4 A program használatának részletes leírása

Jogosultságok

A rendszerben különböző jogosultsági szintek kerültek kialakításra, amelyek meghatározzák a felhasználói felület megjelenését és a funkcionalitáshoz való hozzáférést. A jogosultsági rendszer nem csupán a felhasználói felület elemeit szabályozza, hanem a háttérben futó folyamatokhoz való hozzáférést is korlátozza.

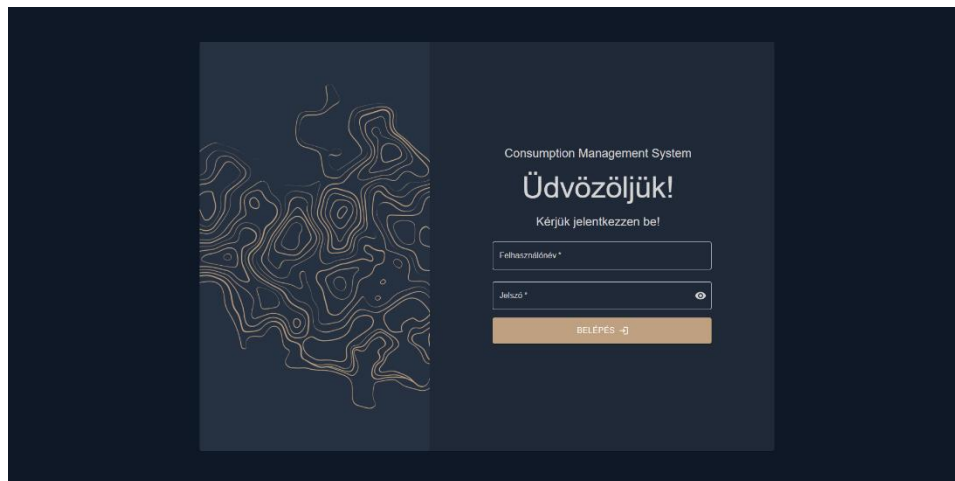
A rendszerben három különböző jogosultsági szint került implementálásra:

- **Admin** - Teljes körű hozzáféréssel rendelkezik
- **Restaurant assistant** - Éttermi funkcionalitásokhoz kapcsolódó jogosultságokkal
- **Ticket assistant** - Jegykezeléshez kapcsolódó jogosultságokkal

Bejelentkező felület

Az oldal megnyitásakor a felhasználót egy bejelentkező felület fogadja, amelyen a program használatához szükséges felhasználónév és jelszó megadására szolgáló mezők találhatók. A megfelelő felhasználónév és jelszó páros után a belépés gombra kattintva lehet a rendszerhez hozzáférni.

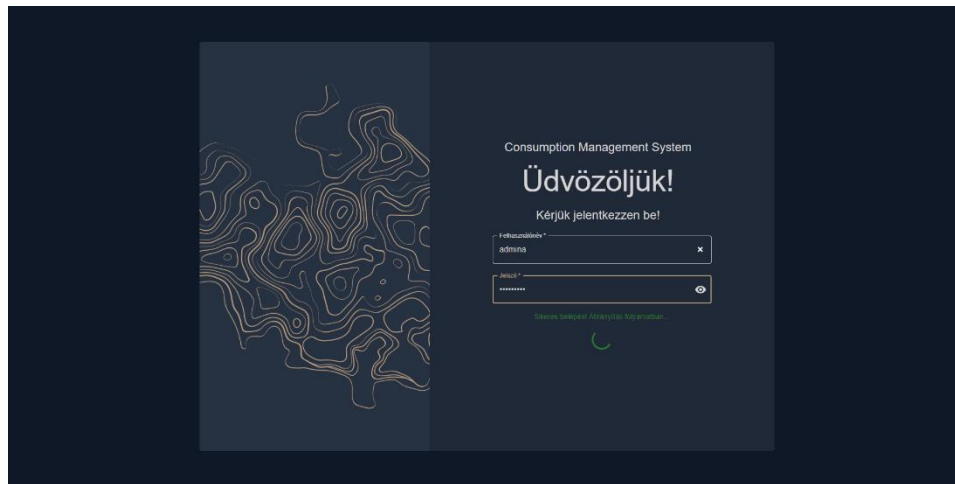
Funkciók és lépések



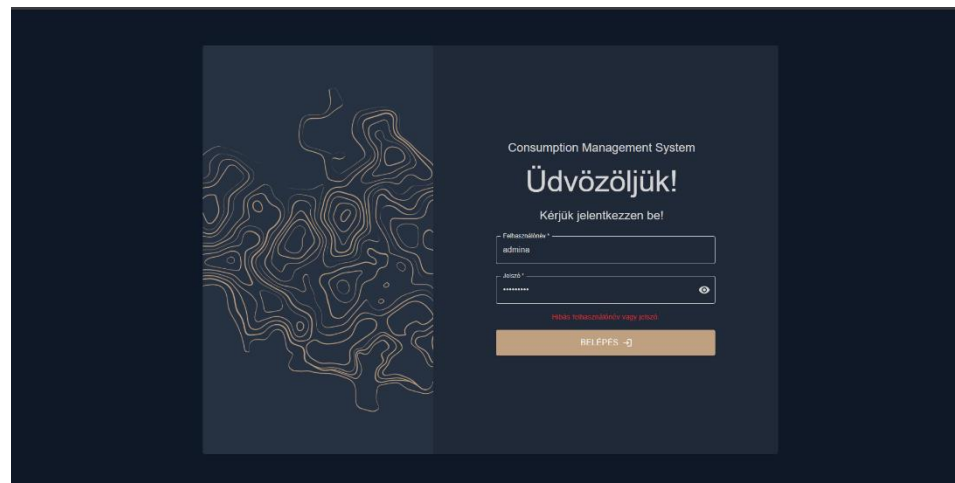
127. ábra – Bejelentkező oldal felülete

1. Felhasználónév és jelszó mező kitöltése
2. Belépés gombra való kattintás

Sikeres azonosítás után, átirányít a program a kezdőképernyőre Amennyiben sikertelen volt a bejelentkezési folyamat, hibaüzenet jelzi a probléma okát.



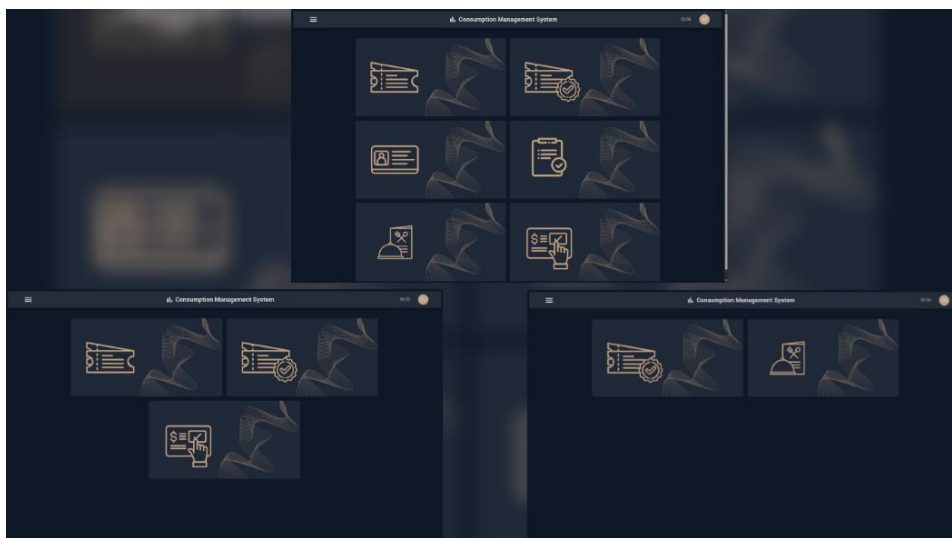
129. ábra – Sikeres bejelentkezés



128. ábra – Sikertelen bejelentkezés

Nyitó oldal

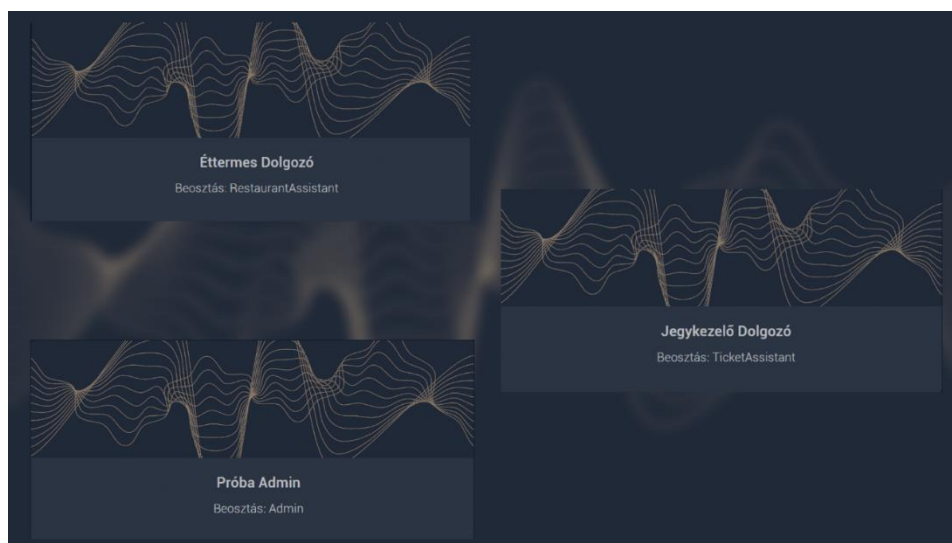
A felületen kártyák és navigációs gombok jelennek meg, melyek száma felhasználói jogosultságoktól függ. Az oldal felületének jobb felső sarkában egy időzítő található, amely valós időben jelzi, hogy mennyi idő van még hátra a felhasználó automatikus kijelentkeztetéséig inaktivitás esetén. Emellett egy különálló navigációs menüsor is rendelkezésre áll, amely megkönnyíti a felhasználó számára a profiloldalhoz való hozzáférést, valamint lehetőséget biztosít a programból történő manuális kijelentkezésre.



130. ábra – Nyitó oldal felülete

Profil

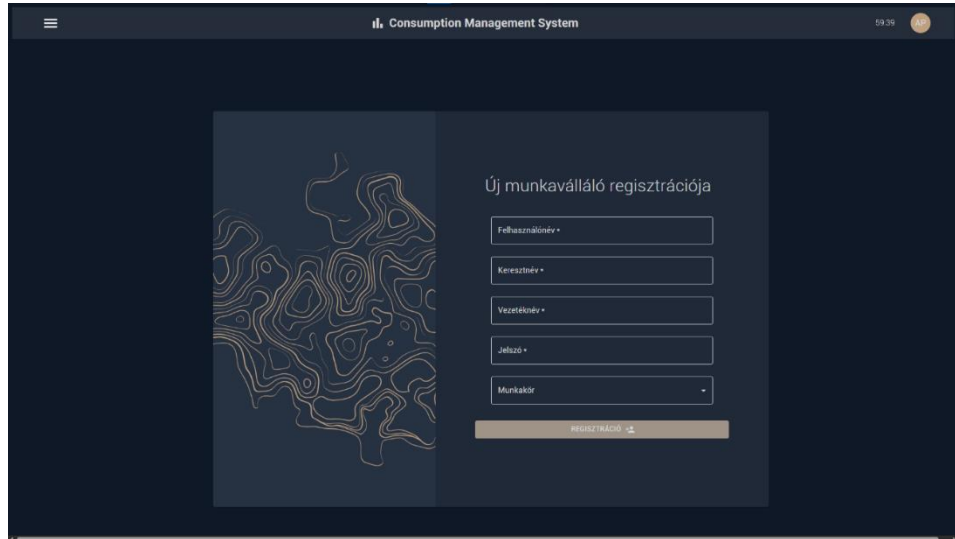
A kezdőoldalról a profil gombra kattintva, átirányít a program a saját felhasználói oldalra, ahol a dolgozó megtudja nézni a nevét, valamint, hogy milyen jogosultsággal rendelkezik.



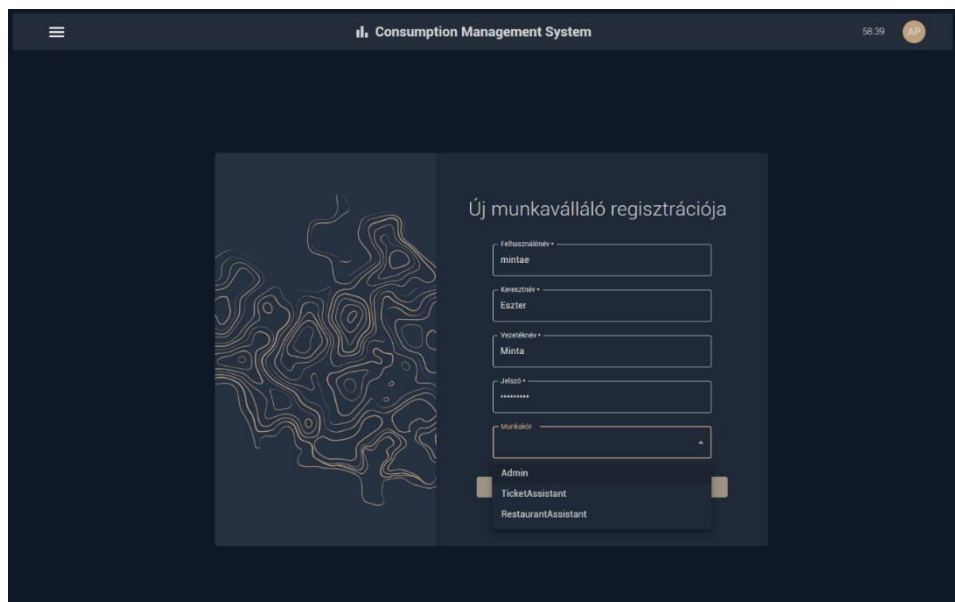
131. ábra – Profil oldal felülete

Regisztráció

A regisztrációs oldalt kizárólag *Admin* jogosultságú felhasználók érhetik el. Itt lehet új dolgozót regisztrálni a rendszerbe.



133. ábra – Regisztrációs oldal felülete

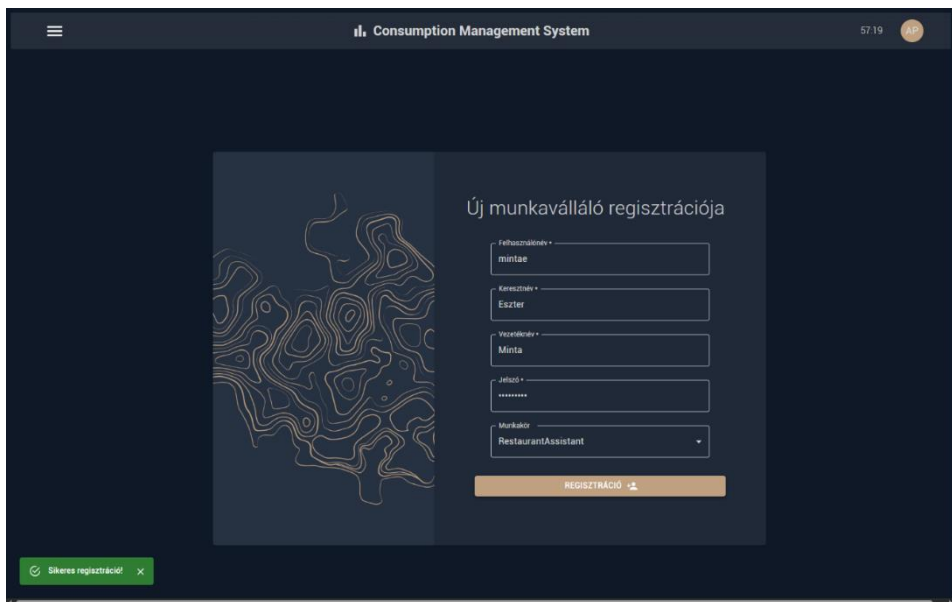


132. ábra – Regisztrációs felület helyes kitöltése

Funkciók és lépések

1. Minden beviteli mező helyes kitöltése
2. Regisztráció gombra való kattintás

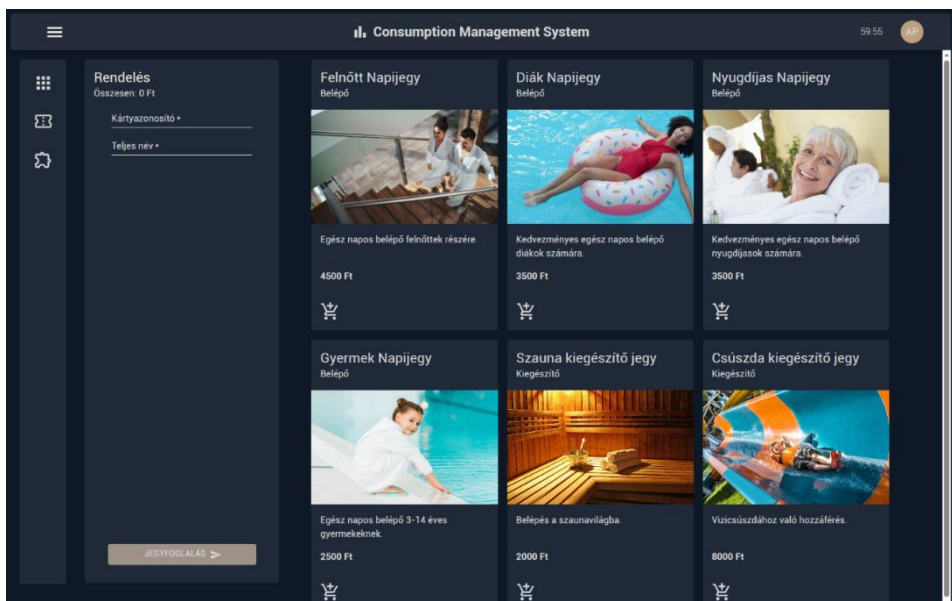
Ha minden érték helyesen került kitöltésre, a rendszer visszajelzi a sikerességet.



134. ábra – Sikeres regisztráció

Jegyvásárlás

Ezen az oldalon, a látogatók részére különféle fürdőbelépők és kiegészítő jegyek vásárolhatók, valamint a vásárlásokat vendégekhez (kártyához) lehet rendelni. Az oldal célja, hogy egyszerű és gyors módot biztosítson a jegyfoglalásra.

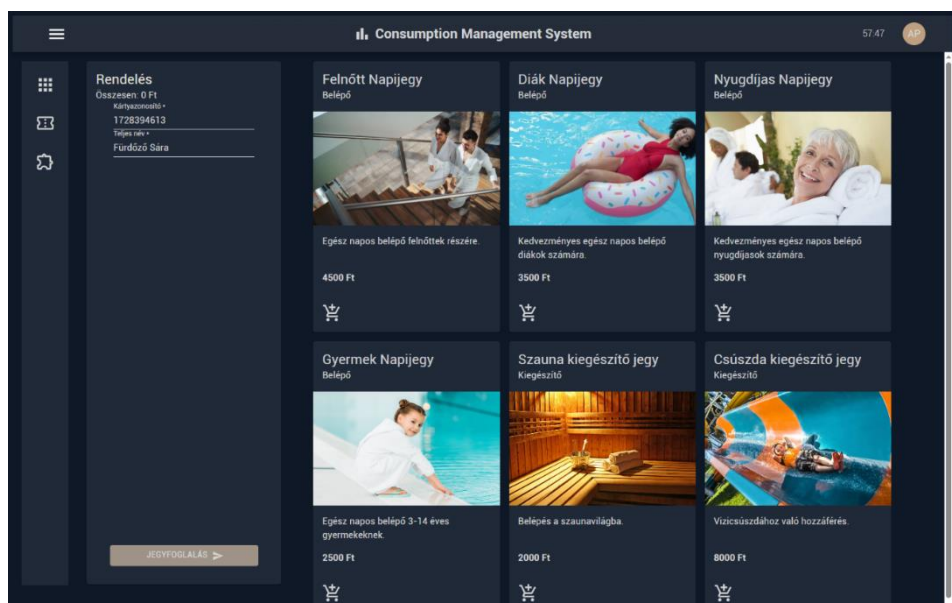


135. ábra – Jegy vásárló felület

Funkciók és lépések

1. Vendég felvétele

A bal oldali panelen található a *Rendelés* szekció, ahol meg kell adni a vendég adatait:



136. ábra – Jegyvásárló felület helyes kitöltése

- **Kártyazonosító:** A vendég RFID-kártyájának azonosítója (pl. beléptetőkártya).
- **Teljes név:** A vendég neve.

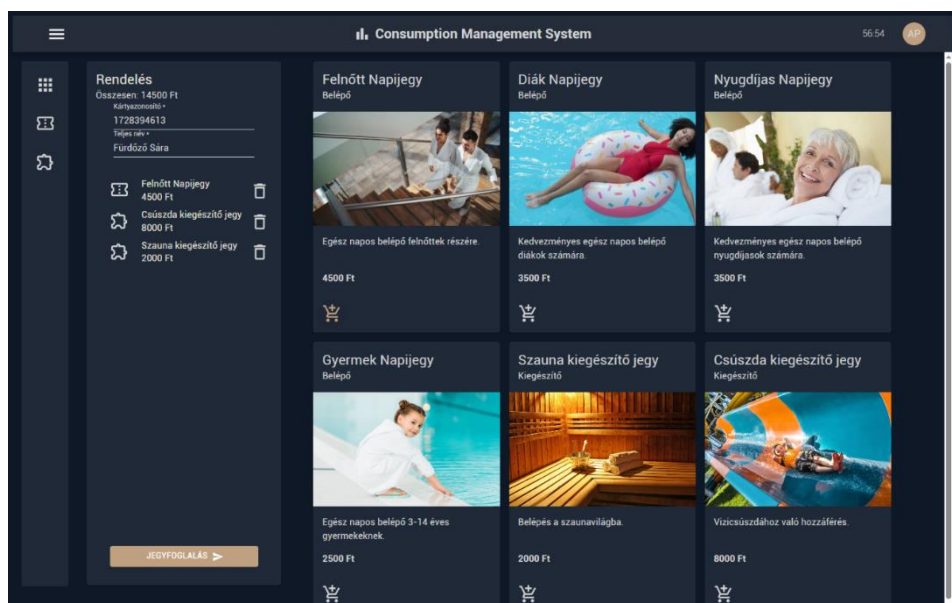
Ezek az adatok kötelezőek a jegyfoglalás véglegesítéséhez.

2. Jegytypus kiválasztása

A jobb oldalon különféle jegytypusok közül lehet választani. Minden jegykategória tartalmaz képet, leírást és árat.

3. Kosárba helyezés

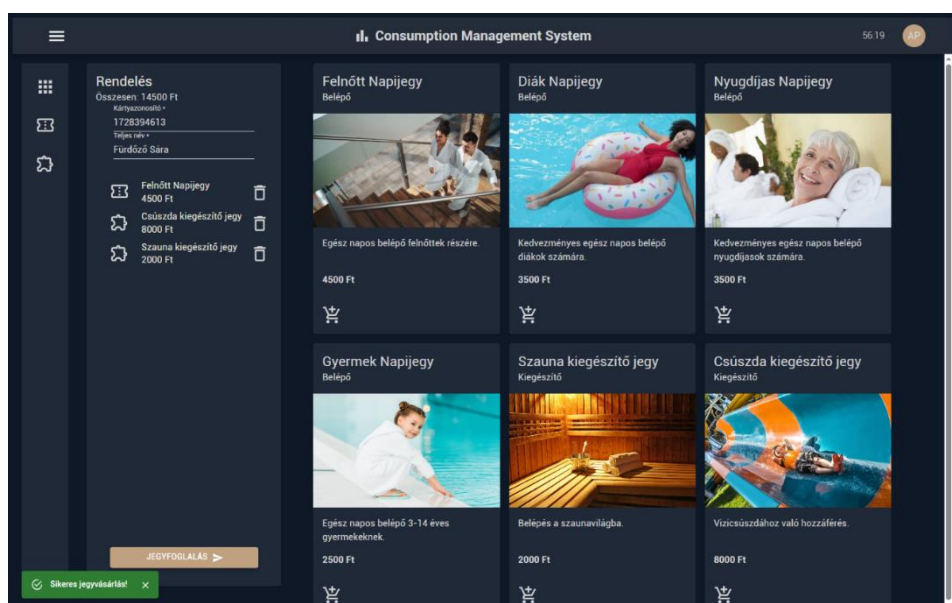
Minden jegy mellett található egy plusz ikon, amellyel az adott jegy hozzáadható a vásárlási listához és egy mínusz ikon, amellyel a jegy eltávolítható a kosárból.



137. ábra – Jegyek hozzáadása a kosárhoz

4. Jegyfoglalás véglegesítése

Miután a szükséges jegyek a kosárba kerültek, és a vendég adatait kitöltöttük, a bal alsó sarokban található **"JEGYFOGLALÁS"** gombbal lehet véglegesíteni a rendelést. A gomb csak akkor aktív, ha minden szükséges mező ki van töltve és van kiválasztott jegy.



138. ábra – Sikeres rendelés

Használati példa

A vendég megérkezik az esemény helyszínére, és a kezelő a rendszerben új jegyet szeretne rögzíteni.

1. A kezelő a főmenüben kiválasztja a "*Jegyvásárlás*" lehetőséget.
2. A megjelenő képernyőn megadja a vendég **kártyaazonosítóját** (pl.:1) a bal felső keresősávban.
3. A rendszer automatikusan **regisztrálja** az új jegyet, amelyhez a háttérben generál egy azonosítót (pl.: #19).
4. A vendég adatai és az eseményhez tartozó jegy mentésre kerülnek.
5. Ezután a vendég már használhatja a kártyáját fogyasztásra – az elfogyasztott tételek rögzítésre kerülnek a profiljához, amit később a kijelentkezés során lehet elszámolni.

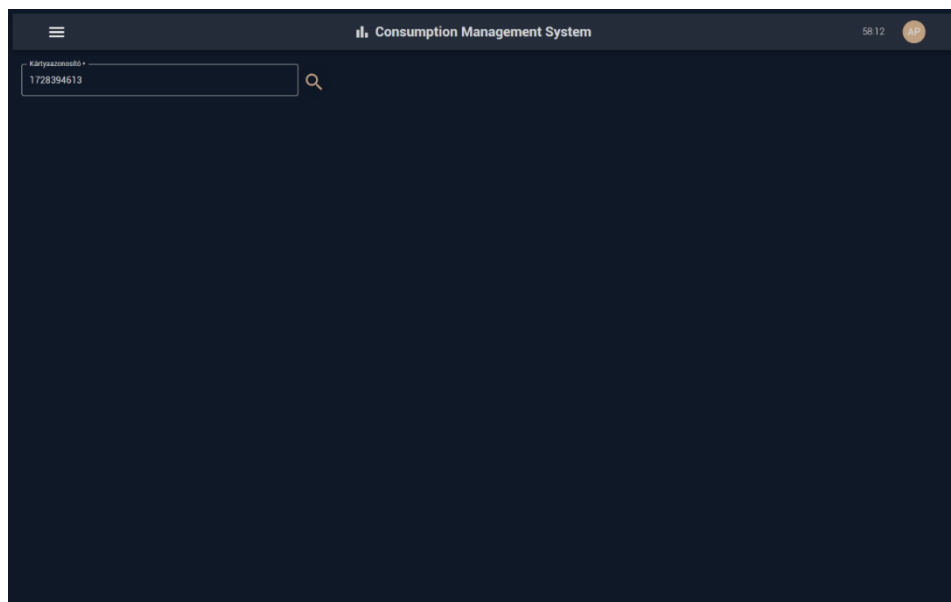
Jegyellenőrzés kártyaazonosító alapján

Ez a felület lehetővé teszi a fürdőbe érkező vendégek jegyének gyors ellenőrzését RFID kártya segítségével.

Funkciók és lépések

1. Kártyaazonosító megadása

A bal felső sarokban található beviteli mezőbe írjuk be a vendég **RFID kártyaazonosítóját** (pl. 123456), majd nyomjuk meg a nagyító ikont.



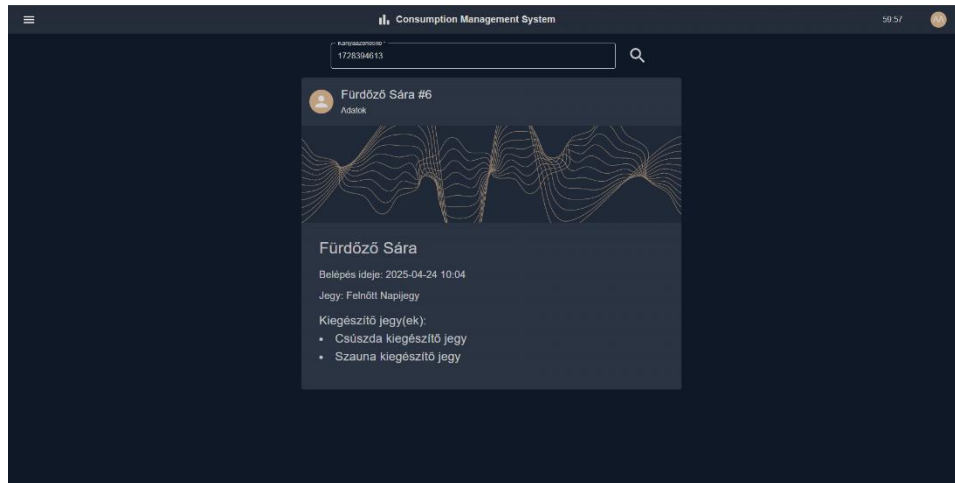
139. ábra – Jegyellenőrzés felülete

2. Vendégadatok és jegyinformációk megjelenítése

A rendszer automatikusan lekérdezi és megjeleníti a vendéghez tartozó aktuális belépési adatokat:

- **Vendég neve** és azonosítója (pl. *Fürdőző Sára #22*)
- **Belépés ideje** (pl. *2025-04-13 12:32*)
- **Megvásárolt jegytípus** (pl. *Felnőtt Napijegy*)
- **Kiegészítő jegyek** (Ha belépéskor vásárolt a vendég)

Ez a felület lehetővé teszi az azonnali vizuális ellenőrzést, hogy a vendég érvényes jeggyel rendelkezik-e a belépéshez.



140. ábra – Jegy ellenőrzése

Használati példa

1. Fürdő beléptetéskor az operátor beolvassa vagy beírja a kártyaszámot.
2. A képernyőn megjelenik, hogy a vendég milyen típusú belépőt vásárolt, mikor lépett be, és van-e hozzá kiegészítő jegye (pl.: *szauna*, *csúszda*).
3. Ha a vendég jegye érvényes, beléphet az adott zónába.

Vendég kijelentkeztetése és fogyasztás áttekintése

Ez a képernyő részletesen megjeleníti a vendég által a tartózkodása során igénybe vett szolgáltatásokat és termékeket, valamint lehetőséget biztosít a **kijelentkezésre**.

Fő elemek:

1. Vendégazonosítás

A vendég neve és azonosítója megjelenik a fogyasztási lista felett (pl. *Fürdőző Sára #19*).

2. Összes fogyasztás

A felületen látható az eddigi fogyasztások **összesített értéke** (pl. *Összes fogyasztás: 21500 Ft*).

3. Fogyasztási tételek listája

Minden tétel külön-külön jelenik meg, tartalmazva:

- Termék nevét
- Leírását
- Mennyiséget
- Egységárat
- Rendelés idejét

Műveleti gombok

- **VÉGLEGESÍTÉS**

Ezzel a gombbal történik a vendég **kijelentkeztetése a rendszerből**, ami lezárja a fogyasztását, és megindítja az elszámolási folyamatot.

- **PDF MEGTEKINTÉSE**

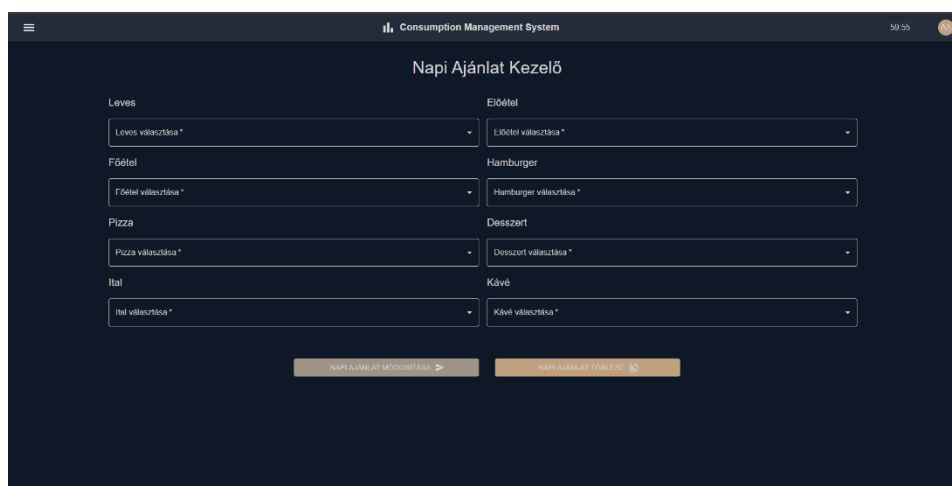
Letölthető és megtekinthető a fogyasztási összesítő PDF formátumban.

Használati példa

A vendég távozásakor a kezelő beolvassa a kártyát, majd ezen a képernyőn áttekinti a fogyasztásokat, végül a **Véglegesítés** gombra kattintva kijelentkezteti a vendéget.

Napi ajánlat kezelő oldal

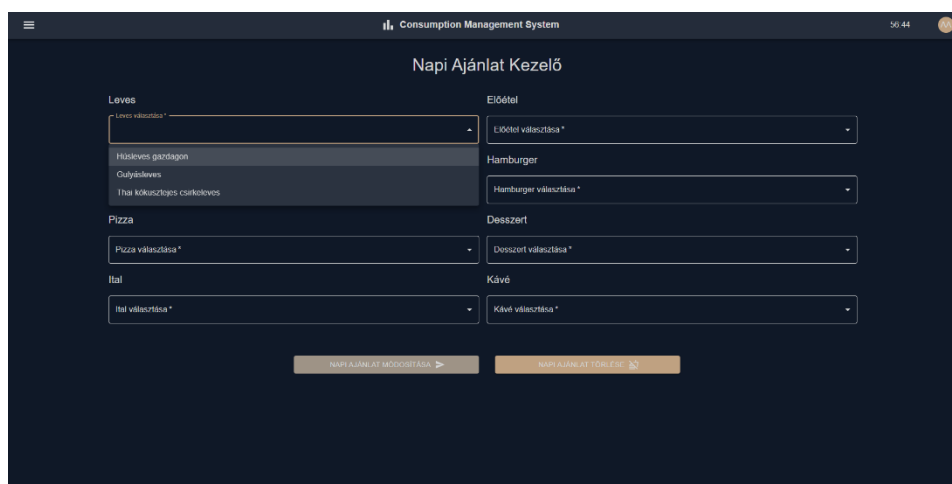
Ez a képernyő lehetőséget biztosít az étterem napi ajánlatának egyszerű és gyors módosítására, kategóriánként csoportosítva. A felhasználó különböző étel és italfajták közül választhat, amelyeket a rendszer napi kínálatként rögzít.

Fő elemek:

141. ábra – Napi ajánlat kezelő felülete

1. Étél- és italválasztó mezők

A felületen nyolc különálló legördülő menü található, amelyek segítségével kiválasztható a napi ajánlat minden egyes eleme. Minden kategória mellett egy lenyíló lista található, amelyben az adott típushoz tartozó elérhető termékek szerepelnek.



142. ábra – Napi ajánlat kezelő helyes kitöltése

2. Műveleti gomb

Az oldal alján elhelyezett „KÉSZ” gombbal véglegesíthető a kiválasztott napi ajánlat. A rendszer ezután frissíti az ajánlatot, és az új beállítások érvénybe lépnek.

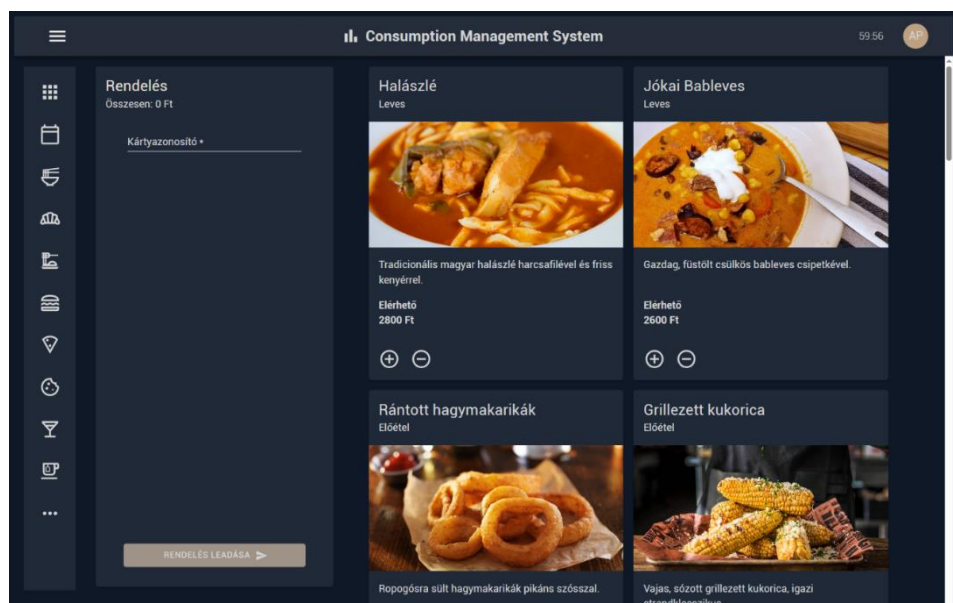
Használati példa

A felhasználó minden kategóriában kiválasztja a napi kínálatra szánt ételt vagy italt (pl. Leves: Gulyásleves, Főétel: Rántott csirkemell, Desszert: Somlói galuska), majd a „KÉSZ” gombra kattint. A rendszer menti a kiválasztott ajánlatot, és ez lesz az aktuálisan megjelenített napi ajánlat az éttermi dolgozók felületén.

143. ábra – Sikeres napi ajánlat felvitel

Éttermi rendszer

Ez a képernyő lehetőséget biztosít a vendég rendeléseinek leadására, és a fogyasztás véglegesítésére.



144. ábra – Étterem rendelő felület

Funkciók és lépések

1. Vendég felvétele

A felszolgáló a képernyő bal oldalán található Rendelés szekcióban beolvassa a vendég RFID-kártyáját, vagy manuálisan beírja a kártyaazonosítót (pl. 0001234).

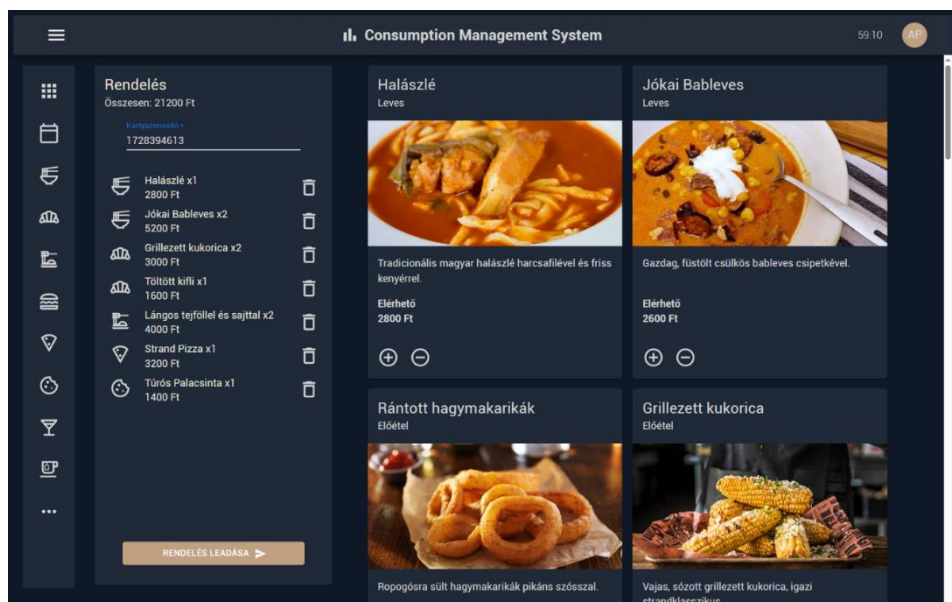
Ez kötelező lépés minden rendelés leadása előtt, mivel a rendszer ehhez kapcsolja a fogyasztást.

2. Rendelés összeállítása

A jobb oldali menüben a pultos kiválasztja a vendég által kért ételeket és italokat. A termékek szűrhetők kategória szerint (pl.: Előétel, Főétel ...)

3. Kosárba helyezés

Minden termék mellett található + és – ikon. A + gombbal a termék a kosárba kerül, a – gombbal pedig eltávolítható. A kosár folyamatosan mutatja az aktuális összeget és a rendelt tételeket.



145. ábra – Ételek hozzáadása a kosárhoz

4. Rögzítés

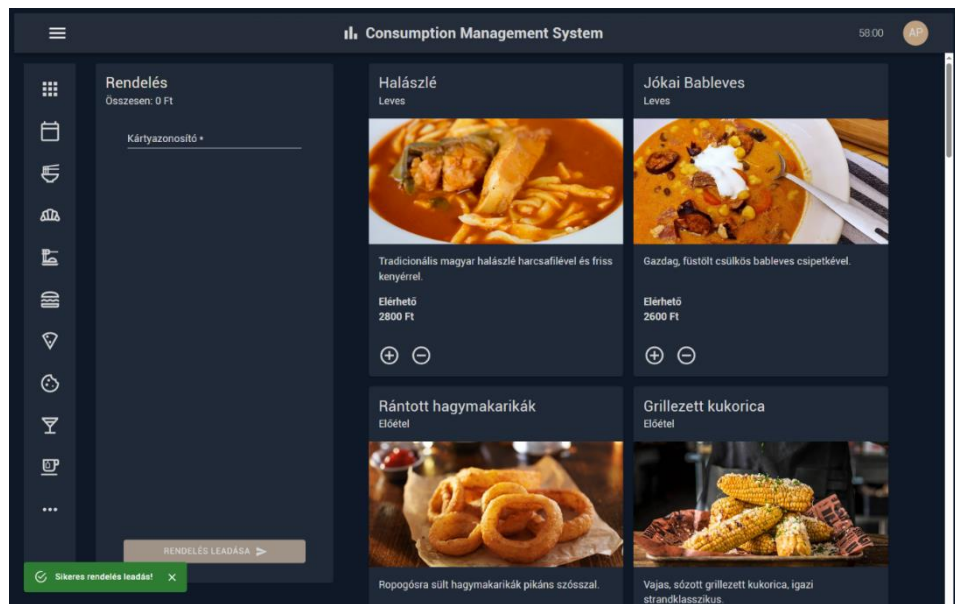
Miután a vendég kiválasztotta a kívánt ételeket, és az azonosító is megadásra került, a Rendelés leadása gomb (a bal alsó sarokban) elérhetővé válik.

Használati példa

A vendég megérkezik az esemény helyszínére, és a kezelő a rendszerben új jegyet szeretne rögzíteni.

1. A kezelő a főmenüben kiválasztja a "Étterem" lehetőséget.
2. Az éttermi alkalmazott a vendég kéréseit rögzíti a kosárban
3. A rendelés befejezése után a „Rendelés leadása” gombra kattintva kerül rögzítésre a vendég adatai mellé a fogyasztás
4. Sikereség esetén egy üzenet jelzi a sikerességet.

5. Végül a rendszer készen áll a következő vendégre fogadására.



146. ábra – Sikeres rendelés leadás

5 Összegzés

A Consumption Management System megalkotása során egy valódi, gyakorlati problémára kerestünk korszerű, megbízható megoldást. Célunk nem csupán az volt, hogy megfeleljünk a vizsgaremek követelményeinek, hanem hogy olyan rendszert hozzunk létre, amely technológiailag is helytálló, és a valós életben is könnyedén alkalmazható.

A rendszer – a legmodernebb webes technológiákra építve – lehetőséget kínál arra, hogy komplex szolgáltatóhelyeken egyszerűvé, átláthatóvá és hatékonyá váljon a vendégek fogyasztásának nyilvántartása. A React + TypeScript alapú frontend és az ASP.NET Core Web API-s backend együttese nemcsak stabil, de jövőálló alapokat is teremtett.

A projekt során megszerzett ismeretek, az alkalmazott technológiák és a csapatmunka eredményeként egy olyan szoftver született, amelyre büszkék lehetünk. A fejlesztés során külön figyelmet fordítottunk a biztonságra, a reszponzív felhasználói élményre, valamint az átlátható, jól dokumentált kódszerkezetre is.

5.1 Fejleszthetőségi lehetőségek

A kitűzött célok, mint az átláthatóság, rugalmasság és jövőbeni bővíthetőség, megvalósultak. A rendszer stabil alapként szolgál a további fejlesztésekhez, amik a következők lehetnek:

Többnyelvűség támogatása

A fürdők gyakran fogadnak külföldi látogatókat, akik nem beszélnek magyarul, ezért fontos, hogy a rendszer több nyelven is elérhető legyen. Ez nemcsak a felhasználói élményt javítja, hanem csökkenti a személyzet idegennyelv-ismeretére nehezedő nyomást is. A többnyelvűség megvalósítása során a felület nyelvét a felhasználó egy lenyíló menüből választhatja ki (például a navigációs sávban elhelyezve). Ez a fejlesztés lehetővé tenné, hogy a rendszer szélesebb közönséget szolgáljon ki, és nemzetközi környezetben is hatékonyan működjön.

Dinamikus szűrési lehetőségek bővítése

A jelenlegi rendszer kizárólag ételtípusok szerinti szűrést tesz lehetővé. A felhasználói élmény további javítása érdekében érdemes megfontolni a szűrési lehetőségek kibővítését, például alapanyagok, allergének vagy egyéb szempontok alapján. Ez különösen hasznos lehet speciális étrendet követő vagy ételérzékenységgel élő felhasználók számára, és hozzájárulhat a rendszer használhatóságának és személyre szabhatóságának növeléséhez.

Import / export funkciók bővítése

A rendszer jelenlegi formájában egy adott szolgáltatóhelyre lett kialakítva, azonban a jövőben lehetőség nyílhat arra, hogy más helyszíneken is alkalmazhatóvá váljon. Ennek elősegítése érdekében érdemes lenne megvalósítani olyan importálási funkciókat, amelyek lehetővé teszik az ételek és egyéb adatok egyszerű átvitelét más rendszerekbe, ezáltal csökkentve az új helyszínek bevezetéséhez szükséges időráfordítást.

Emellett az exportálási lehetőségek bevezetése, például CSV vagy más általánosan elfogadott formátumban, hasznos lenne a rendelések statisztikai elemzéséhez, kimutatások készítéséhez és további adatelemzési célokra. Ez támogatná az üzleti döntéshozatalt, valamint az üzemeltetés átláthatóságát és optimalizálását.

Új jegytípusok és termékek kezelése a webes felületen

A jelenlegi rendszerben kizárólag rendszergazdai jogosultsággal, közvetlenül az adatbázis szintjén lehetséges új jegytípusok, valamint ételek és italok felvétele. A felhasználói élmény és a rugalmasság növelése érdekében célszerű lenne egy webes adminisztrációs felület kialakítása, amelyen keresztül az adminisztrátor jogosultsággal rendelkező felhasználók is képesek lennének új elemek létrehozására, módosítására vagy törlésére. Ez a fejlesztés lehetővé tenné a gyorsabb és hatékonyabb tartalomkezelést, valamint csökkentené a technikai közreműködés szükségességét a mindennapi működés során.

Kioszk és mobilalkalmazás lehetősége

A jelenlegi rendszer kizárólag a szolgáltatóhely munkatársai által használható, így a rendelések kezelése teljes mértékben a pultos személyzetre hárul. Ez nemcsak erőforrás-igényes, de megnövekedett vendégforgalom esetén torlódást is okozhat. A folyamat hatékonyabbá tételére a jövőben érdemes lenne önkiszolgáló kioszkok bevezetését megfontolni, amelyek lehetővé tennék, hogy a vendégek közvetlenül leadják rendeléseiket anélkül, hogy sorban kellene állniuk. Emellett egy mobilalkalmazás fejlesztése szintén megkönnyítené a rendelési folyamatot, és kényelmes, korszerű felhasználói élményt nyújtana. Mindkét megoldás célja az ügyfélkiszolgálás gyorsítása, a dolgozók leterheltségének csökkentése, valamint a szolgáltatás minőségének emelése.

Ezúton köszönjük mindazok támogatását, akik hozzájárultak a projekt sikeréhez.

6 Ábrajegyzék

1. ábra - Trello munkafelület	7
2. ábra - Github repository development branch (2025.04.21. állapot szerint).....	7
3. ábra - CMS.db adatbázis tábláink kapcsolatai DBeaver alkalmazásban	8
4. ábra - Bejelentkező oldal minden mező kitöltése kötelező	11
5. ábra - Bejelentkező oldal, rossz felhasználónév vagy jelszó hibakezelése	12
6. ábra – Bejelentkező oldal, Backenddel nem képes kialakítani kapcsolatot	12
7. ábra - Böngésző local storage tartalma sikeres bejelentkezés után	13
8. ábra - Bejelentkező oldal iPhone 12 Pro készüléken.....	13
9. ábra - Főoldal megjelenési formái különféle jogosultságokkal	14
10. ábra - Jogosultsági szint kinyerése a tokenből.....	14
11. ábra - Az „Étterem” nevű kártya megjelenéséért felelős kódrészlet	15
12. ábra - Regisztrációs oldal, hibakezelése, üres mező esetén.....	16
13. ábra - Regisztrációs oldal, Új dolgozó sikeresen létrejött	16
14. ábra - RegistrationPage.tsx-ben használt register végpont tokennel	17
15. ábra - Regisztrációs oldal, Backend elérésének hibája esetén	17
16. ábra - Regisztrációs oldal, megjelenése iPhone 12 Pro készüléken	18
17. ábra - Jogosultság hiány miatti átirányítás a restaurant oldalról	19
18. ábra - Éttermi oldal, Nem elérhető termék kártyája	19
19. ábra - Éttermi oldal, Elérhető termék kártyája	19
20. ábra - Éttermi oldal felépítése.....	20
21. ábra – OrderPage.tsx, Menüelem végpont meghívása	20
22. ábra - Éttermi oldal, Napi ajánlat fül, ha van beállítva ajánlat.	21
23. ábra - Éttermi oldal, Napi ajánlat fül, ha nincs beállítva ajánlat	21
24. ábra - OrderPage.tsx, Napi ajánlat lekérdezése	22
25. ábra - Éttermi oldal, Folyamatban lévő rendelés	23
26. ábra - Éttermi oldal, Sikeresen éttermi rendelés.....	23

27. ábra - OrderPage.tsx, Rendelés elküldéséhez használt végpont.....	24
28. ábra - Éttermi oldal, Nem létező kártyaazonosító használta esetén	24
29. ábra - Éttermi oldal, iPhone 12 Pro készüléken.....	25
30. ábra - Navigációs sáv nagy képernyős eszközökön	26
31. ábra - Navigációs sáv kisebb kijelzőkön	26
32. ábra - Navbar.tsx, Visszaszámláló funkció	26
33. ábra - Navigációs sáv, Felhasználói funkciók	27
34. ábra - Navigációs sáv, Navigálási funkciók	27
35. ábra - Navbar.tsx - Kijelentkeztetési algoritmus	27
36. ábra - Jogosultság hiány miatti átirányítás a tickets oldalról.....	28
37. ábra - Jegyértékesítő oldal, minta egy kiegészítő típusú jegyre	28
38. ábra - Jegyértékesítő oldal, minta egy belépő típusú jegyre.....	28
39. ábra - Jegyértékesítő oldal, Felülete	29
40. ábra – TicketPage.tsx, Jegyek lekérése a két GET végpontról.....	29
41. ábra – Jegyvásárló oldal felülete	30
42. ábra - TicketsPage.tsx, Új vendég létrehozása	30
43. ábra – Jegyértékesítő oldal, Sikeres jegy vásárlás.....	31
44. ábra - Jegyértékesítő oldal, Backendről az adatok nem érkeznek meg	31
45. ábra - Jegyellenőrző oldal felülete.....	32
46. ábra - TicketValidationPage.tsx, Vendég adatainak lekérdezése GET végpont segítségével.....	32
47. ábra – Jegyellenőrző oldal, Kártya adatainak lekérdezése	33
48. ábra - Jegyellenőrző oldal, Kártya nem létezik	33
49. ábra - Jegyellenőrző oldal, iPhone 12 Pro készüléken	34
50. ábra - Napi ajánlat készítő oldal felülete	35
51. ábra – Ételek lekérdezése GET végpont segítségével	35
52. ábra – Adatok törlése DELETE végpont segítségével	36

53. ábra – Napi ajánlat kezelő visszajelző, Sikeres módosítás	36
54. ábra – Napi ajánlat kezelő oldal, iPhone 12Pro készüléken	37
55. ábra – Véglegesítés oldal felülete	38
56. ábra – Fogyasztási bizonylat, PDF formátumban.....	39
57. ábra – PDF generálás React-PDF segítségével.....	39
58. ábra – Véglegesítés oldal, Kártya nem létezik	40
59. ábra – Véglegesítés oldal, iPhone 12Pro készüléken	40
60. ábra – Profil oldal, Felülete	41
61. ábra – EmployeeProfilPage.tsx Adatok kinyerése tokenből	41
62. ábra – Wireframe-ek tervezéshez	43
63. ábra – OrderPage.tsx Napi ajánlat lekérése GET végponttal	44
64. ábra – OrderPage.tsx Elem hozzáadó algoritmus	44
65. ábra – OrderPage.tsx Elem mennyiség csökkentő algoritmus	45
66. ábra – OrderPage.tsx Elem törlő algoritmus	45
67. ábra – OrderPage.tsx Végösszeg kiszámító algoritmus.....	46
68. ábra – OrderPage.tsx Rendelés leadó algoritmus	46
69. ábra – Jogosultság kezelés JWT token segítségével.....	47
70. ábra – Tesztelés Jest keretrendszer segítségével	51
71. ábra – jest.config.mjs, default export	52
72. ábra – Jest teszt, MediaCard komponens helyes megjelenése.....	52
73. ábra – Jest teszt, Hibakezelés sikertelen bejelentkezés esetén	52
74. ábra – Jest teszt, Sikeres bejelentkezés kezelése	53
75. ábra – A Jest teszt sikeres	53
76. ábra – Nuget csomag hozzáadása Visual Studiohoz	56
77. ábra – appsettings.json, ConnectionString és JWT konfiguráció	58
78. ábra – Admission.cs, Kiegészítő jegy tábla felépítése.....	59
79. ábra – Customer.cs, Vendégek tábla felépítése	60

80. ábra – CustomerAdmission.cs, Kapcsolótábla a Customer és az Admission táblához .	61
81. ábra – CustomerTicket.cs, Kapcsolótábla a Customer és a Ticket táblához	62
82. ábra – DailySpecial.cs, Napi menü tábla felépítése.....	63
83. ábra – Employee.cs, Dolgozó tábla felépítése	64
84. ábra – MenuItem.cs, Termékek tábla felépítése	65
85. ábra – MenuItemOrder.cs, Kapcsolótábla a MenuItem és az Order táblához	66
86. ábra – Order.cs, Rendelés tábla felépítése	67
87. ábra – RefreshToken.cs, Token tábla felépítése	68
88. ábra – Ticket.cs, Jegy tábla felépítése	69
89. ábra – CMSContext.cs, Adatbázis kapcsolatok kezelése	70
90. ábra – CMSContext.cs, Függőségi befeckskendezés	70
91. ábra – CMSContext.cs, Táblák reprezentálása DbSet segítségével	71
92. ábra – CMSContext.cs, Metódus és Entitás kapcsolatok konfigurálása.....	71
93. ábra – MenuItemPostDto.cs, POST kérés során megjelenő adatok listája.....	72
94. ábra – Admissions GET végpont visszatérő értéke	75
95. ábra – Auth/register POST végpont visszatérő értéke.....	76
96. ábra – Auth/login POST végpont visszatérő értéke.....	77
97. ábra – Auth/refreshToken POST végpont visszatérő értéke	77
98. ábra – Auth/logout POST végpont visszatérő értéke.....	78
99. ábra – Auth/Checktoken POST végpont visszatérő értéke.....	78
100. ábra – Cards/GetCustomerIdByCardId GET végpont visszatérő értéke	79
101. ábra – Customers/{cardid} GET végpont visszatérő értéke.....	80
102. ábra – Customers POST végpont visszatérési értéke	81
103. ábra – Customers/Resetcardid PUT végpont visszatérési értéke.....	81
104. ábra – Checkout/{customerCardId} GET végpont visszatérési értéke	82
105. ábra – DailySpecials GET végpont visszatérési értéke	83
106. ábra – DailySpecials POST végpont visszatérési értéke	83

107. ábra – DailySpecials DELETE végpont visszatérési értéke	84
108. ábra – Employees/roles GET végpont visszatérési értéke.....	85
109. ábra – Orders POST végpont visszatérési értéke.....	86
110. ábra – Tickets GET végpont visszatérési értéke.....	86
111. ábra – xUnit teszt eredménye	89
112. ábra – xUnit teszt, AdmissionController számára	90
113. ábra – xUnit teszt CheckoutController számára	91
114. ábra – xUnit teszt, Különböző ellenőrzések a rendeléshez	92
115. ábra – xUnit teszt DailySpecilasController számára	93
116. ábra – xUnit teszt, dolgozók helyes visszatérésének teszteléséhez	94
117. ábra – xUnit teszt, a menüpontok helyes visszatérésének teszteléséhez	95
118. ábra – xUnit teszt, új felhasználó sikeres létrehozásának teszteléséhez.....	96
119. ábra – xUnit teszt, a jegyek helyes visszatérésének teszteléséhez	97
120. ábra - CMS projekt mappa	98
121. ábra - Backend üzemel fejlesztői módban.....	98
122. ábra - CMSReact mappa tartalma.....	99
123. ábra - Frontend Vite szerveren való futtatása.....	99
124. ábra - CMS.exe, a futtatandó fájl a projekthez	102
125. ábra - CMS.exe működés közben	103
126. ábra - Böngészőben futó alkalmazás	103
127. ábra – Bejelentkező oldal felülete	105
128. ábra – Sikertelen bejelentkezés.....	106
129. ábra – Sikeres bejelentkezés	106
130. ábra – Nyitó oldal felülete	107
131. ábra – Profil oldal felülete	107
132. ábra – Regisztrációs felület helyes kitöltése.....	108
133. ábra – Regisztrációs oldal felülete.....	108

134. ábra – Sikeres regisztráció.....	109
135. ábra – Jegy vásárló felület	109
136. ábra – Jegyvásárló felület helyes kitöltése	110
137. ábra – Jegyek hozzáadása a kosárhoz.....	111
138. ábra – Sikeres rendelés	111
139. ábra – Jegyellenőrzés felülete.....	112
140. ábra – Jegy ellenőrzése	113
141. ábra – Napi ajánlat kezelő felülete	115
142. ábra – Napi ajánlat kezelő helyes kitöltése.....	115
143. ábra – Sikeres napi ajánlat felvitel.....	116
144. ábra – Étterem rendelő felület	117
145. ábra – Ételek hozzáadása a kosárhoz.....	118
146. ábra – Sikeres rendelés leadás	119

7 Irodalmi jegyzék, hivatkozásjegyzék

.NET Documentation – Microsoft Learn

<https://learn.microsoft.com/en-us/dotnet/>

A *backend* fejlesztés során használt keretrendszer hivatalos dokumentációja.

Entity Framework Core & SQLite

<https://learn.microsoft.com/en-us/ef/core/providers/sqlite/>

Adatbázis-kezelés *SQLite* segítségével *.NET* környezetben.

React TypeScript – React official documentation

<https://react.dev/>

<https://www.typescriptlang.org/docs/handbook/react.html>

A *frontend* fejlesztés alapjául szolgáló *React* + *TypeScript* technológia.

Material UI (MUI)

<https://mui.com/>

A felhasználói felület komponenseinek kialakításához használt *UI* keretrendszer.

Tailwind CSS

<https://tailwindcss.com/docs>

Utility-first CSS keretrendszer a testreszabható, reszponzív stílusokhoz.

W3Schools Web Tutorials

<https://www.w3schools.com/>

Alapvető *HTML*, *CSS*, *TypeScript* ismeretek megerősítéséhez használt referenciaforrás.

GitHub

<https://github.com/>

Verziókezelés, csapatmunka és forráskód tárolása.

YouTube – oktatóvideók, tutorialok

Több, fejlesztéshez kapcsolódó technológia (pl. *React*, *.NET*, *MUI*) használatának elsajátításához.

Chrome DevTools – Reszponzív teszteléshez

<https://developer.chrome.com/docs/devtools/device-mode/>

A mobilnézet és különböző kijelzőméretek szimulációjára használt eszköz.

Microsoft .NET – JWT Authentication in ASP.NET Core

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwtbearer/>

Hivatalos dokumentáció *JWT* alapú autentikáció implementálásához *ASP.NET Core* alkalmazásban.

Freepik – Graphic Resources for Web Design

<https://www.freepik.com/>

Képek, ikonok, illusztrációk és *UI*-elemek forrása. A felületi tervezéshez használt inspirációk és grafikai elemek részben innen származnak.

GitHub Copilot – AI-alapú kódgeneráló eszköz

<https://github.com/features/copilot>

A *GitHub Copilot* egy mesterséges intelligencia alapú eszköz, mely segítette a tanulást és a hibakeresést a fejlesztési folyamat során. A rendszer gyors javaslatokat tett a kódrészletekhez, így elősegítve a programozási nyelvek és technológiák elsajátítását.